



TrackWise Training Program:

Fundamentals of TrackWise Web Services

Manual Version: March 2015

Information in this document is subject to change without notice and does not represent a commitment on the part of Sparta Systems, Inc. The software described in this document is furnished under a license agreement or nondisclosure agreement. The software may be used or copied only in accordance with the terms of those agreements. No part of this publication may be reproduced, stored in a retrieval system, or transmitted in any form or any means electronic or mechanical, including photocopying and recording for any purpose other than the purchaser's own use, without the written permission of Sparta Systems, Inc.

© 1995-2013 Sparta Systems Inc. All Rights Reserved.

TrackWise®, TrackWise Web TeamAccess, and TrackWise Coordinator are registered trademarks owned by Sparta Systems. Sparta Systems is also the exclusive owner of the trademarks ConfigForm(s), FlexField, FlexTabs, StateMachine, Parallel StateMachine, SavePlus, DateAssist, and InstantAction. All other registered trademarks or trademarks are the property of their respective holders.

Sparta Systems, Inc.
Holmdel Corporate Plaza
2137 Highway 35
Holmdel, New Jersey 07733

(888) 261-5948
support@sparta-systems.com

Sparta Systems Europe, Ltd.
1 Hachoshlim Street
Herzliya, Israel 46724

972.9.970.6000
europe-support@sparta-systems.com

Sparta Systems Asia Pacific Ltd.
503 Silvercord Tower
230 Canton Road, TST, Hong Kong

+852.2376.9345
asia-sales@spartasystems.com

Table of Contents

Preface.....	5
1. Introduction	6
1.1 Introduction to TrackWise Web Services.....	6
1.2 Key Concepts Overview	8
1.3 Business Case and Web Services in this Course.....	12
2. Installation and Environment Setup	16
2.1 Eclipse Installation	16
2.2 Inbound Web Services (IWS) Setup	16
2.3 Outbound Web Services (OWS) Setup.....	30
2.4 Debugging in Eclipse	51
3. TW WS Best Practices & Conventions.....	53
3.1 Introduction	53
3.2 Code Convention.....	53
3.3 Logging	55
3.4 Connection / Session Handling	56
3.5 TrackWise Configuration.....	57
3.6 Properties Files, Parameter Intake	59
3.7 Usage (when to use a web service vs. TWC or TW Utilities).....	59
4. Inbound Web Services (IWS)	61
4.1 Introduction to IWS.....	61
4.2 Connection Logic.....	61
4.3 IWS Data Types	64
4.4 Querying Records.....	67
4.5 Updating Records.....	68
4.6 New Record Creation	70
4.7 Post an Activity.....	72
4.8 IWS Limitations	73
4.9 Security And User Permissions In IWS	73
5. Outbound Web Services (OWS).....	74
5.1 Introduction to OWS	74
5.2 Building the Interface Contract.....	75
5.3 Axis2 and AXIOM.....	78
5.4 Search.....	84
5.5 Populate	92
5.6 Populate into Grid.....	96

5.7	Search and Populate Inside of Grid.....	98
5.8	OWS Limitations.....	103
5.9	OWS Security	103
6.	Putting it all Together (IWS/OWS).....	105
6.1	Introduction	105
6.2	OWS Transfer	105
6.3	IWS Child Creation	107
7.	Repeatable Process for Development.....	109
7.1	Introduction	109
7.2	Process	109
8.	TW Web Services Efforts	110
9.	Troubleshooting Web Services	111
9.1	Introduction	111
9.2	Inbound Web Services	111
9.3	Outbound Web Services	112
10.	Appendix	113
10.1	Glossary of Terms.....	113
10.2	TrackWise Field WSDL Definitions for IWS	115
10.1	TrackWise Fields XML Structure for OWS.....	120
10.2	Using Log4J to write to a log file	124
10.3	Reading parameters from a file	126
10.4	Code Example : Convert date format.....	127
10.5	SoapUI	127
10.6	Workflows	130
10.7	Course ProductService.wsdl File	132
10.8	Course tws-course.properties File :	135
		135

Preface

Course Objective:

The objective of this course is to introduce the capabilities and application of TrackWise Web Services through the use of real-life business scenarios where web services are typically implemented. In the beginning of the course, the student will be presented with background information in order to understand the business need for web services as well as a high level overview of the technologies that are typically used. The course will then move into iterative development of both Inbound and Outbound web services, first starting with simple use cases and building knowledge along the way.

This course is intended to focus on the capabilities of TrackWise Web Services only – prior knowledge and experience with TrackWise, Java, XML, Eclipse IDE, and SOAP web services is assumed.

The Course Workbook:

In order to focus on topics specific to TrackWise web services, references to additional materials are provided throughout the manual. With this guide, students will be able to become familiar with the core components of TrackWise web services and their role in current implementations.

Workbook Exercises

Throughout the book you will find Workbook Exercises, represented by the following symbol:



Workbook Exercises allow the student to follow the instructor step by step through an example.

On Your Own Exercises

You will also find On-Your-Own Exercises throughout the course, represented by the following:



On-Your-Own Exercises allows the student to apply what they've learned to complete a problem and build their own code.

1. Introduction

1.1 Introduction to TrackWise Web Services

Web Services for TrackWise are a means for an external program to interact with TrackWise programmatically through a common set of Application Programmable Interfaces (APIs). From a TrackWise perspective, there are two different categories of Web Services that differ both in their functionality and the way in which they interact with TrackWise: Inbound and Outbound Web Services.

Although the examples in the course all leverage the Java programming language, the nature of web services allows the programmer to select their language of choice provided that communication with TrackWise leverages the SOAP protocol.

1.1.1 TW Web Services Capabilities

- Inbound
 - New Record Creation and update
 - Search / Update existing data
 - Customer / Vendor and active directory data updates/creation
 - Activity Performance
 - Outbound
 - Full user-interface-based **configurable** functionality
 - TrackWise Workflow triggers a call to one or more web services
 - Transfer of data from TrackWise
 - Master data lookup/validation
 - Flexibility in enterprise master data management
 - Extension of workflow driven out-bound calls to support synchronous operations
- The Web Services capabilities are agnostic and allow TrackWise to “talk” with external systems using Service Oriented Architecture based web services.

1.1.2 Inbound Web Services (IWS)

IWS provides an interface that exposes a TrackWise API which a third party application can access programmatically to perform actions against the TrackWise database, API Example:

- **postActivity**
- **createLogin**
- **createPerson**
- **updatePR**

Common usage scenarios for IWS are for interfaces with other systems where records have to be synchronized but the standard TrackWise integration manager utilities (PR Import, Person Import) are insufficient.

For example, the business requirement for an interface may specify functionality that would otherwise not be able to be accomplished via the PR Import utility such as real-time data exchange, augmented audit trail entries, an acknowledgement to the source system that the sync was complete, etc. In this case, a web service would be able to provide a more robust solution than would otherwise be available.

In order to properly make use of IWS, the web service application must be designed against the WSDL made available by TrackWise, which specifies all available IWS operations.

1.1.3 Outbound Web Services (OWS)

OWS provides a means for an action in TrackWise to trigger a programmatic call out to a third party program that takes some action. Specifically, the types of OWS operations are:

- **Search** – Send information from one or more TrackWise fields to an external program after a user selects a field or clicks a button; fill a returned value into the field that was searched. A search operation can have many input parameters mapped to data fields, but only one output parameter mapped. The same data field must be mapped to both an input parameter and an output parameter.
- **Populate** – Send information from one or more fields to an external program after a user selects a field or clicks a button; populate one or more fields with information returned. A populate operation can have as many input and output parameters mapped as necessary.
- **Transfer** – send information to an external application upon the performance of a specified action. A transfer operation can have one or multiple input parameters, but does not return any output parameters.

The most common usage for OWS is to lookup information in another system and populates it back into TrackWise. Although this can be accomplished with the TrackWise native external fields functionality, an OWS can be designed to be more flexible and have more than one entry point rather than being restricted to a dependent field relation in TrackWise. Common uses for the transfer operation are to notify another system or to push information out for an IWS web service to take action on. An example that we will look at later in the course is using TrackWise to push a grid out to a program via OWS, which will then pass the grid to a program which will use IWS to create children records in the system.

In order to properly make use of OWS, the web service application must make a WSDL available to TrackWise which specifies all available operations.

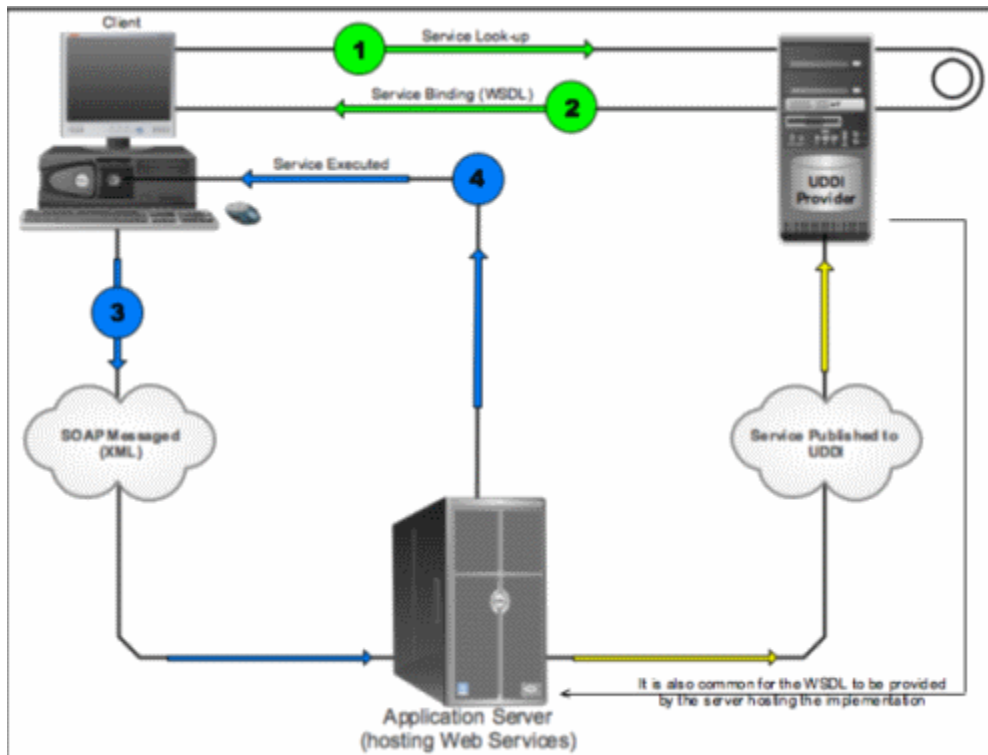
1.2 Key Concepts Overview

The focus of this course is specific to programming for TrackWise web services only. Prior knowledge and experience with the terms introduced in this section is assumed. This section will review some key terms and concepts for context and provide links to further reading material for reference.

1.2.1 Web Services

Web services extend the World Wide Web infrastructure to provide the means for software to connect to other software applications. Applications access web services via ubiquitous web protocols and data formats such as HTTP, XML, and SOAP, with no need to worry about how each Web service is implemented.

Web services combine the best aspects of component-based development and the Web.



1.2.2 SOAP

SOAP is a lightweight protocol intended for exchanging structured information in a decentralized, distributed environment. Both IWS and OWS leverage the SOAP protocol to communicate between TrackWise and the custom developed web service application. It is important to note that TrackWise web services do not support REST based web services.

SOAP is an XML-based protocol that functions over HTTP, allowing different applications based on different languages / technologies to communicate with one another. The structure of the XML document that comprises a SOAP message consists of the following elements:

- Envelope
- Header
- Body
- Fault

For the purposes of the TrackWise web services course, we will leverage SOAP toolkits which will handle the formatting /overhead necessary for SOAP interactions. In your own development you may choose not to leverage a SOAP toolkit, in which case understanding of SOAP, XML, and XML Schema is required. For the interested student, further information on the SOAP protocol can be found online. Some helpful references are provided below:

- SOAP Specification: <http://www.w3.org/TR/soap/>
- W3Schools SOAP Tutorial: <http://www.w3schools.com/soap/default.asp>
- Wikipedia: <http://en.wikipedia.org/wiki/SOAP>

SOAP Example:

Call to method: `String orderItem (String RetailerID, String ItemNumber, String ItemName, String ItemDesc)`

Request example:

```
<SOAP-ENV:Envelope
  xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
  SOAP-ENV:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
  <SOAP-ENV:Body>
    <m:OrderItem xmlns:m="Some-URI">
      <RetailerID>557010</RetailerID>
      <ItemNumber>1050420459</ItemNumber>
```

```
<ItemName>AMF Night Hawk Pearl M2</ItemName>

<ItemDesc>Bowling Ball</ItemDesc>

</m:OrderItem>

</SOAP-ENV:Body>

</SOAP-ENV:Envelope>
```

Response example:

```
<SOAP-ENV:Envelope xmlns:SOAP-
ENV="http://schemas.xmlsoap.org/soap/envelope/"

  SOAP-ENV:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">

  <SOAP-ENV:Body>

    <m:OrderItemResponse xmlns:m="Some-URI">

      <OrderNumber>561381</OrderNumber>

    </m:OrderItemResponse>

  </SOAP-ENV:Body>

</SOAP-ENV:Envelope>
```

1.2.3 WSDL

WSDL is the Web Services Description Language, an XML language for describing Web services. A WSDL document specifies the location of a web service and the operations that the service makes available. A WSDL must be developed to deploy an OWS interface for TrackWise, and a WSDL is provided for the TrackWise IWS interface that exposes the TrackWise API.

The structure of a WSDL document consists of the following major XML elements:

- types
- message
- portType
- binding

Together, these elements specify all facets of the web service, including how the service is called, what parameters the service requires, and what data structures the service returns. We will take a deeper look into the design of the WSDL when we discuss Outbound Web Services. For the interested student, further information on WSDL can be found online. Some helpful references are provided below:

- WSDL Specification: <http://www.w3.org/TR/wsd120-adjuncts/>
- W3Schools WSDL Tutorial: <http://www.w3schools.com/wsd/default.asp>
- Wikipedia: <http://en.wikipedia.org/wiki/Wsd1>

See [WSDL example](#) .

1.2.4 Apache Axis2

Apache Axis2 is a Web Services / SOAP / WSDL engine. Axis2 supports SOAP and WSDL based web services as well as other variants. Hot deployment makes it possible to deploy new services without the need for a restart. For more information on Axis2, please refer to one of the references below:

- Axis2/Java Page: <http://axis.apache.org/axis2/java/core/>
- Axis2 Tutorial: <http://www.journaldev.com/255/axis2-web-services-tutorial>
- Wikipedia: http://en.wikipedia.org/wiki/Apache_Axis2

1.2.5 XML

extensible Markup Language (XML) is a markup language for encoding information into a structured format that is readable by both humans and machines. XML makes use of various user defined elements and attributes to store data. Unlike HTML, XML must be well formed in order to be processed correctly. XML can also be validated against schema which define the structure and tags that must appear in the document.

XML is widely used in TrackWise web services and will be leveraged throughout the course. There are a variety of resources available to learn more about XML, the ones listed below can be considered a starting point into further exploration into the topic.

- XML Specification: <http://www.w3.org/TR/REC-xml/>
- W3C XML Homepage: <http://www.w3.org/XML/>
- Wikipedia: <http://en.wikipedia.org/wiki/XML>

1.2.6 Apache Tomcat

Apache Tomcat is an open source software implementation of the Java Servlet and JavaServer Pages technologies. TrackWise and the TrackWise web services interfaces are deployed on top of the Tomcat web server.

- Tomcat Page: <http://tomcat.apache.org/>
- Tomcat 6 & 7 Tutorial: <http://www.coreservlets.com/Apache-Tomcat-Tutorial/>
- Wikipedia: http://en.wikipedia.org/wiki/Apache_Tomcat

1.2.7 Other Terms / Technologies

Java

Java is an object oriented programming language which we will be leveraging for development throughout the course.

- Java API: <http://docs.oracle.com/javase/6/docs/api/>
- Java Tutorials: <http://docs.oracle.com/javase/tutorial/>
- Wikipedia: [http://en.wikipedia.org/wiki/Java_\(programming_language\)](http://en.wikipedia.org/wiki/Java_(programming_language))

Eclipse IDE

An integrated development environment to aid developers with coding

- Eclipse Documentation: <http://eclipse.org/documentation/>
- Wikipedia: [http://en.wikipedia.org/wiki/Eclipse_\(software\)](http://en.wikipedia.org/wiki/Eclipse_(software))

1.3 Business Case and Web Services in this Course

The TrackWise system that we will be working on in this course represents a simplified Complaints handling system. In this system, Complaint records are tracked from origination through review, investigation, and root cause analysis. Finally a determination is made as to whether a CAPA is needed to address the root cause of the issue. Commonly a Complaint record only tracks a single product lot, as reflected in the TrackWise configuration.

In the cases where an event or issue corresponds to multiple product lots, a Product Issue record is used to capture the issue and make a determination about whether a series of Complaints should be generated. If Complaints are needed, one is created for each product lot listed on the Product Issue record. Throughout this course, we will examine several web services that are typically utilized in a complaints system like the one described above. The different services and their use cases are outlined in the following sections.

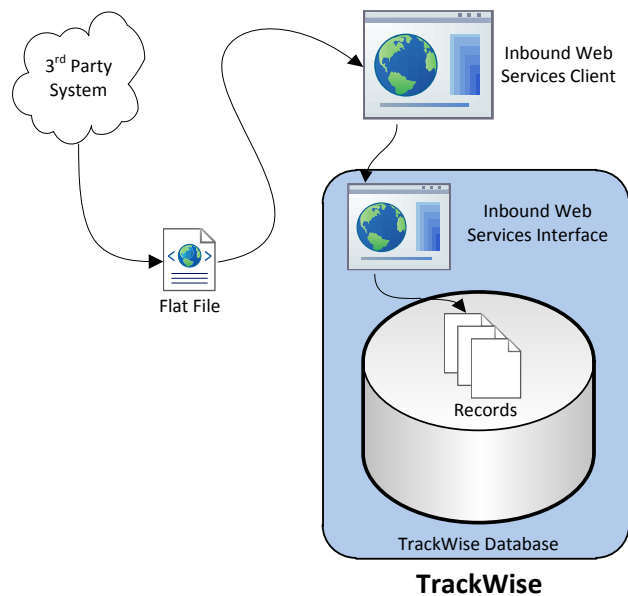
1.3.1 IWS Record Creation / Interaction

In some cases, a company may interface with a third party system that generates complaint records, but opt to track the records within their QMS, TrackWise. Inbound web services can be set up to receive information from these systems and create or update complaint records inside of TrackWise.

In this course, you will work with an inbound web service from a third party call center that will:

- Query Complaint records
- Create Complaint records
- Update Complaint records
- Perform activities against Complaint records

Throughout these exercises the information will be passed into the web service client via a flat file. In this scenario the file represents output from the 3rd party system. In a real world situation this may be an XML file, a call directly to the web service client, or some other method of communication. The web service client will process the file and take the appropriate actions against the TrackWise database by interacting with the Inbound Web Services Interface. Note that XML files are only one way of passing information, various other methods may be used and will vary greatly depending on the requirements for the interface.



1.3.2 OWS Master Data Lookup

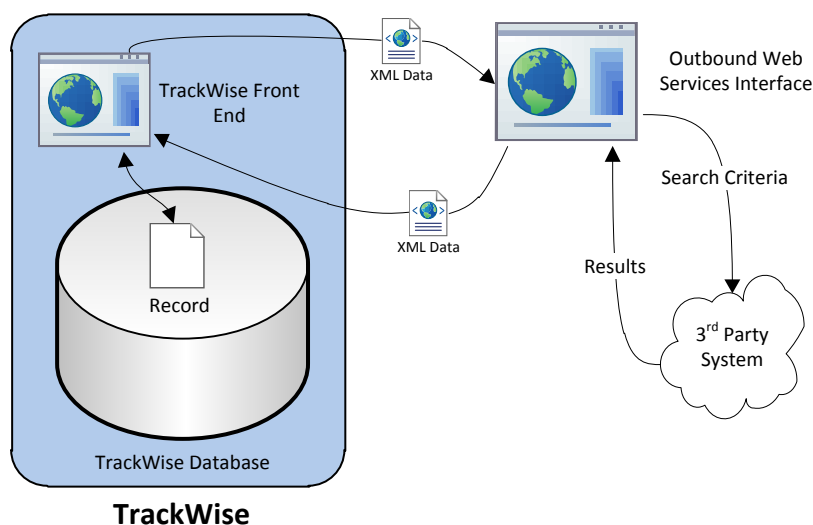
One of the most common uses for web services is to integrate to third party systems in a master data lookup capacity. When entering Complaints, Deviations, or other quality records, there are times when information stored in other systems needs to be referenced. In order to avoid duplication of this information in the TrackWise database, potentially leading to a maintenance problem, and ensure the accuracy while entering the data TrackWise can leverage web services to communicate with the external system and retrieve relevant information.

This course will highlight several methods of retrieving product master data from external sources by leveraging Outbound Web Services:

- First, on the Complaint you will learn to search the product name and pull back results into a single TrackWise field.
- Next, the results from this search will be daisy chained and leveraged for helping to filter search results on a second field – Lot number.
- We will then examine how to populate other product information fields on the form when a Lot number is selected.

- Grid fields must be worked with in a different manner than fields outside of a grid, to educate you on how to work with grids, we will step through populating a 'Materials' grid based on the product information that was populated earlier
- Finally, on the Product Issue record we will step through performing the search and populate operations completely within a product information grid to accomplish the same functionality that we set up on the Complaint record

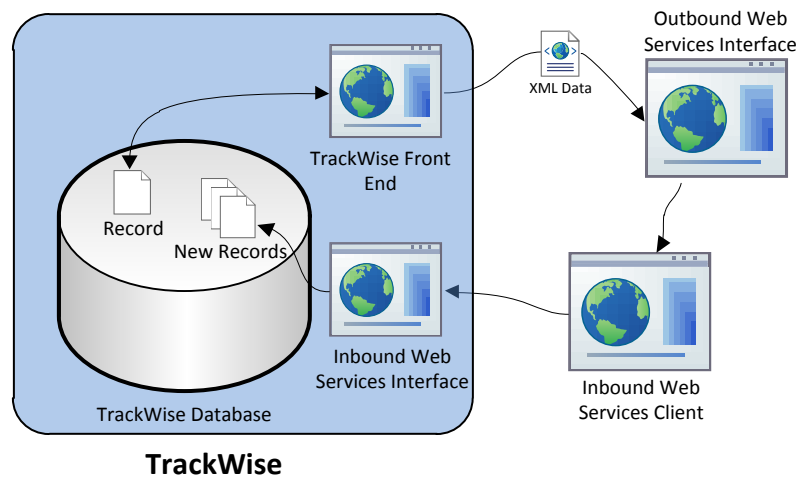
The general model for the interaction of outbound web services for populate and search operations is shown below. An event within the TrackWise front end will trigger an outbound web service call, passing XML information to the developed Outbound Web Services Interface. This interface will process the information received and will query a third party system for information as applicable. Information received back from the 3rd party system may be further processed before being sent back to the TrackWise front end in XML format. The user in the system must save the record that is being worked on in order for the information populated in the record to be committed to the database.



1.3.3 Child Record Creation

Another common use of TrackWise web services is to create children records corresponding to a row in a grid. This is an example of coupling Inbound and Outbound web services to one another to perform a business need.

In this course, this is represented by having a Product Issue record with multiple products and lots specified. When it is determined that the Issue is a Complaint, the TrackWise front end performs an OWS transfer operation to transfer the product information grid to an Outbound Web Services Interface. This interface then does processing and hands the information to an Inbound Web Services Client, which will process the grid and interact with the TrackWise Inbound Web Services Interface to create children Complaint records for each row and then perform an activity to close out the original Product Issue record.



2. Installation and Environment Setup

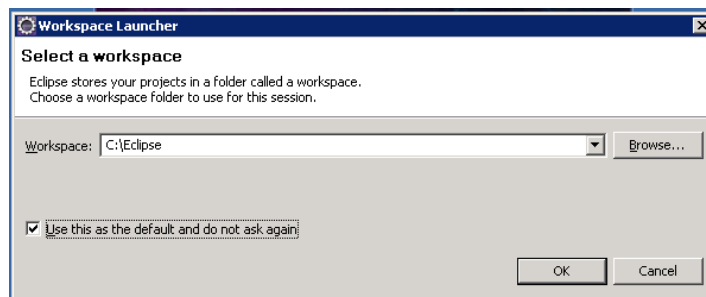
This chapter will take you through the installation and setup of the web services development environment. In the course of the setup, we will test a sample inbound and outbound web service to verify functionality.

2.1 Eclipse Installation

The integrated development environment (IDE) that we will work in for the entirety of this course will be Eclipse. This section will take you through installation of Eclipse.



1. Extract the contents of **C:\ Course Resources\WebService Resources\eclipse-jee-kepler-SR2-win32-x86_64** to your **C:\Program Files** folder. The eclipse.exe will be installed inside of the eclipse folder. You may create a shortcut to this executable file for convenience.
2. Open **Eclipse.exe** and specify your workspace where all files and projects will be stored. For the purposes of this course, we will use the **C:\Eclipse directory**.



2.2 Inbound Web Services (IWS) Setup

The TrackWise IWS interface must be installed in order to leverage TrackWise Inbound Web Services. This interface makes the TrackWise API available to third party applications to use. The following sections will step through installation of the interface and testing of a sample client.

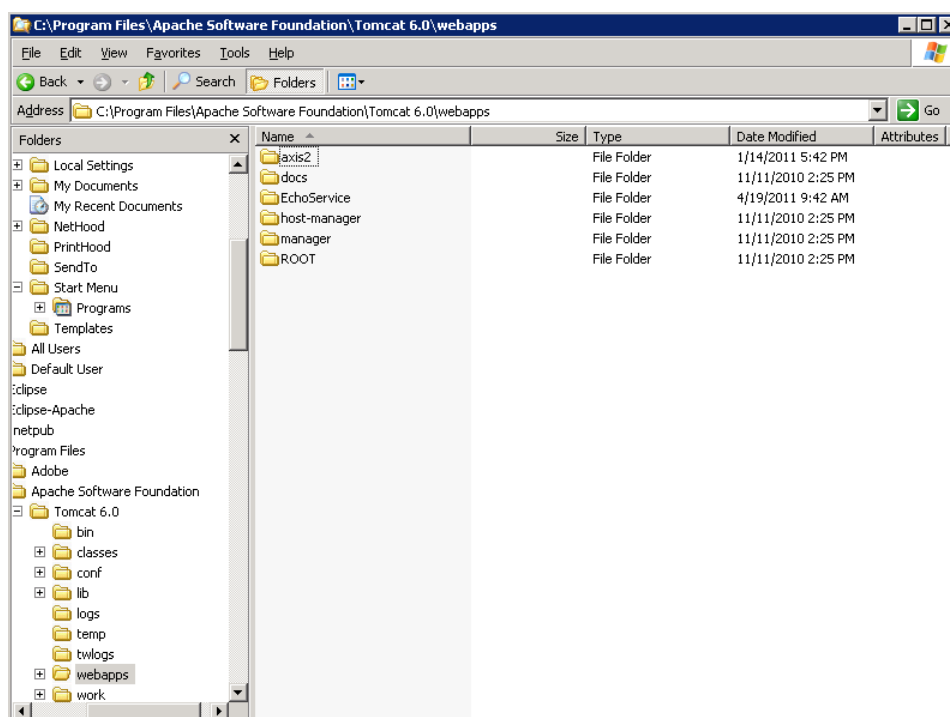
2.2.1 Installing the IWS Interface

Installation of IWS involves deploying the TWS package as a web service and using the appropriate Web Services-enabled TrackWise license.



1. The TrackWise web services interface for Inbound Web Services will need to be installed. In order to do this, the web application will need to be deployed in Tomcat. From the TrackWise Installation CD under **\TrackWise WebServices** (in this course, from **C:\Course Resources\TWS service.zip**), extract the contents of **TWS service.zip** to your local **C:\Program Files\Apache Software Foundation\Tomcat 6.0 webapps** folder. This should create an **axis2** folder.

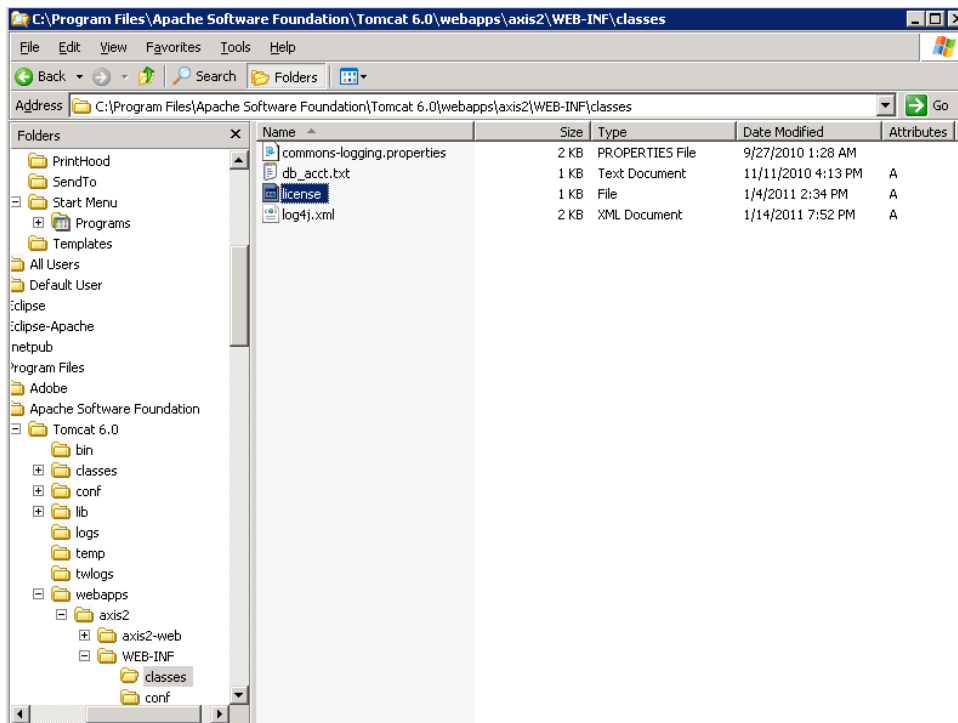
Note: The *axis2* folder can be renamed, if desired



2. Place a TWS enabled license file (located in the root of the C:\ drive) and the db_acct.txt (located in C:\TrackWiseWeb\tw1_TrackWise) file in **C:\Program Files\Apache Software Foundation\Tomcat 6.0 webapps\axis2\WEB-INF\classes**

Note: These files are case sensitive and must be in lower case or TWS will not function.

Note: The db_acct.txt file is optional and overrides the user name and password parameters in the services.xml file. If you do not use a db_acct.txt file, fill in the parameters for user name and password in the services.xml file below.



3. Edit the **services.xml** file in the **C:\Program Files\Apache Software Foundation\Tomcat 6.0\webapps\axis2\WEB-INF\services\TWS\META-INF** directory.
4. Set the following parameters based on TrackWise database:
 - a. jdbc.brand (“Oracle” or “MS SQL Server”) – **MS SQL Server**
 - b. jdbc.host – **localhost**
 - c. jdbc.sid – **tw1**
 - d. jdbc.port – **1433**
5. If configuring notifications on activities performed by web services, then fill out the mail server and mail server authentication parameters (if authentication is required):
 - a. mail.host
 - b. mail.userid
 - c. mail.password

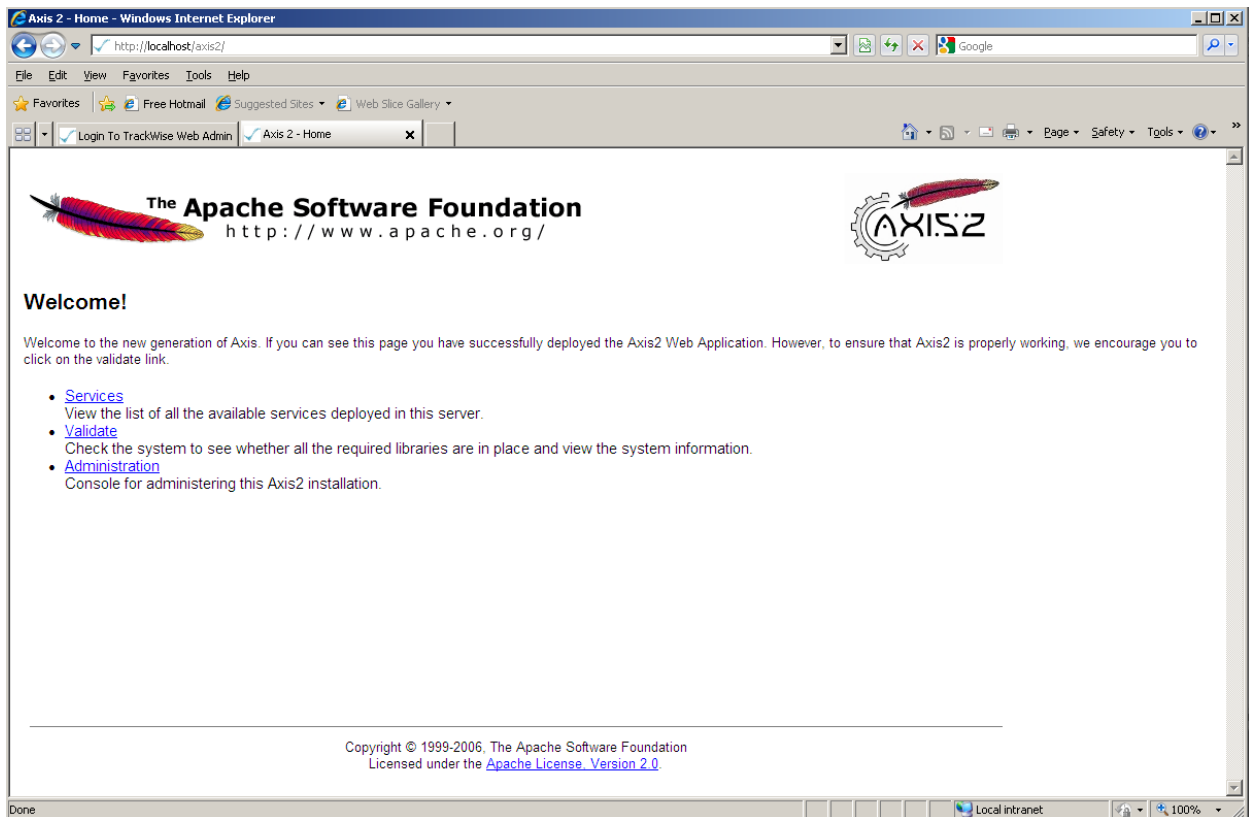
```

services.xml - Notepad
File Edit Format View Help
<?xml version="1.0" encoding="UTF-8"?>
<!-- This file was auto-generated from WSDL -->
<!-- by the Apache Axis2 version: 1.4.1 Built on : Aug 13, 2008 (05:03:35 LKT) -->
<serviceGroup>
  <service name="Trackwise" class="com.sparta.webservices.axis2.Axis2ServiceLifeCycle">
    <messageReceivers>
      <messageReceiver mep="http://www.w3.org/ns/wsdl/in-out" class="com.sparta.webservices.axis2.Trackwise" />
      <messageReceiver mep="http://www.w3.org/ns/wsdl/robust-in-only" class="com.sparta.webservices.axis2.Trackwise" />
    </messageReceivers>
    <parameter name="ServiceClass">com.sparta.webservices.axis2.TrackwiseSkeleton</parameter>
    <parameter name="useOriginalWSDL">true</parameter>
    <parameter name="modifyUserWSDLPortAddress">true</parameter>
    <!-- TWS parameters (only edit the parameters marked here) -->
    <parameter locked="false" name="jdbc.brand">MS SQL Server</parameter>
    <parameter locked="false" name="jdbc.host">localhost</parameter>
    <parameter locked="false" name="jdbc.port">1433</parameter>
    <parameter locked="false" name="jdbc.sid">tw1</parameter>
    <parameter locked="false" name="jdbc.userid"></parameter>
    <parameter locked="false" name="jdbc.password"></parameter>
    <parameter locked="false" name="jdbc.initconnections">5</parameter>
    <parameter locked="false" name="jdbc.minconnections">5</parameter>
    <parameter locked="false" name="jdbc.maxconnections">20</parameter>
    <parameter locked="false" name="jdbc.idle.cleanup.seconds">300</parameter>
    <parameter locked="false" name="mail.host"></parameter>
    <!--parameter locked="false" name="mail.userid"></parameter>
    <parameter locked="false" name="mail.password"></parameter>-->
    <parameter locked="false" name="lock.sleep.millis">1000</parameter>
    <parameter locked="false" name="lock.attempts">15</parameter>
    <parameter locked="false" name="session.timeout.millis">1800000</parameter>
    <parameter locked="false" name="max.object.resolution.depth">2</parameter>
    <!-- end: TWS parameters -->
    <operation name="getAttachment" mep="http://www.w3.org/ns/wsdl/in-out" namespace="http://services.webserv

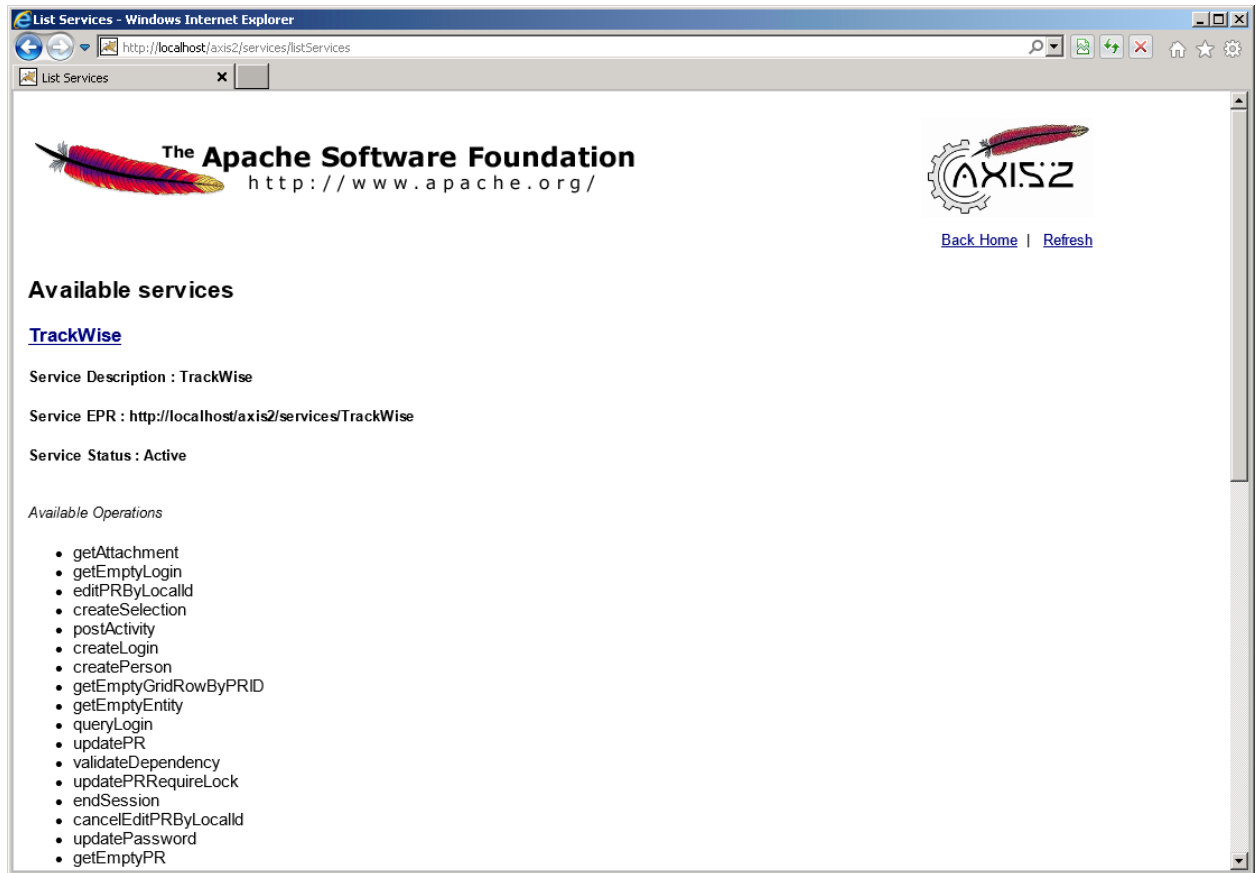
```

6. Restart Tomcat.

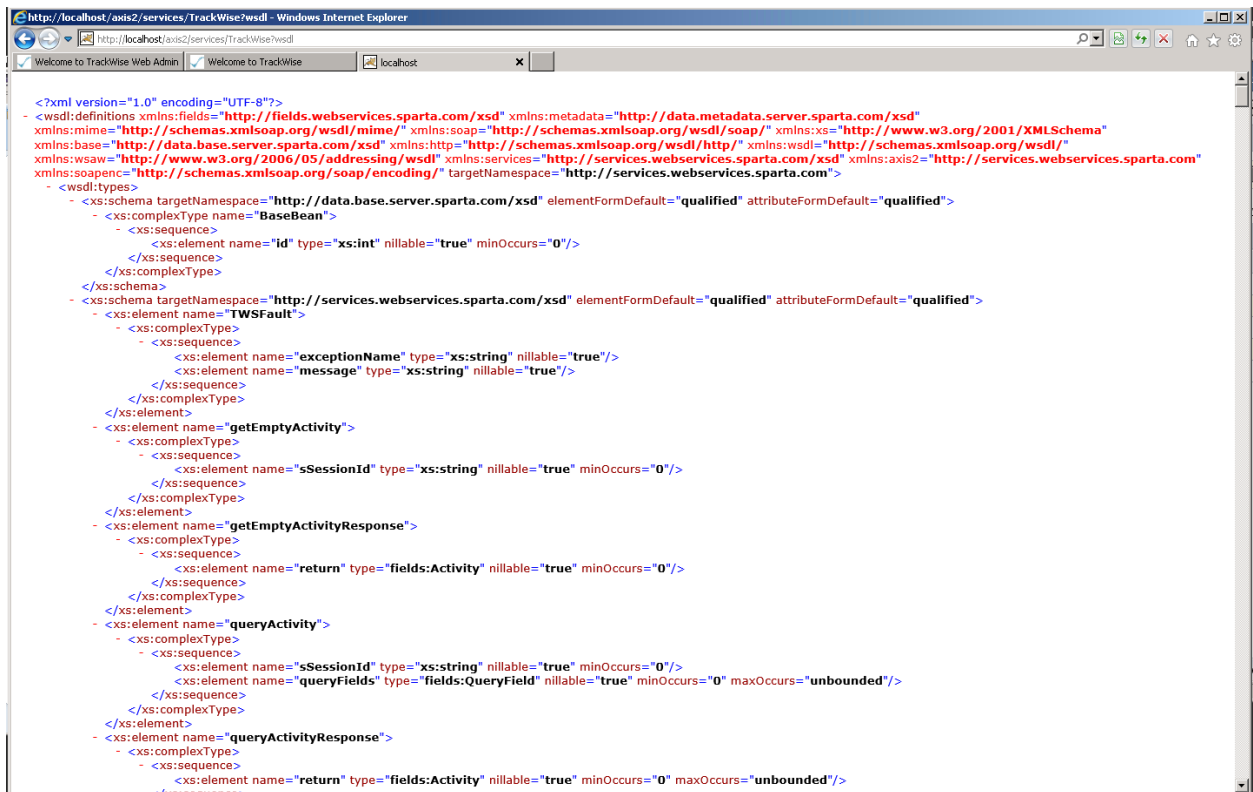
7. Test the installation by going to **<http://localhost/axis2>**. You should see the following screen



8. Click the **Services** link



9. Verify the 'TrackWise' appears. From this link you can view the list of deployed web services and the WSDL.
10. Click on 'TrackWise' link, you will see the content of the WSDL.



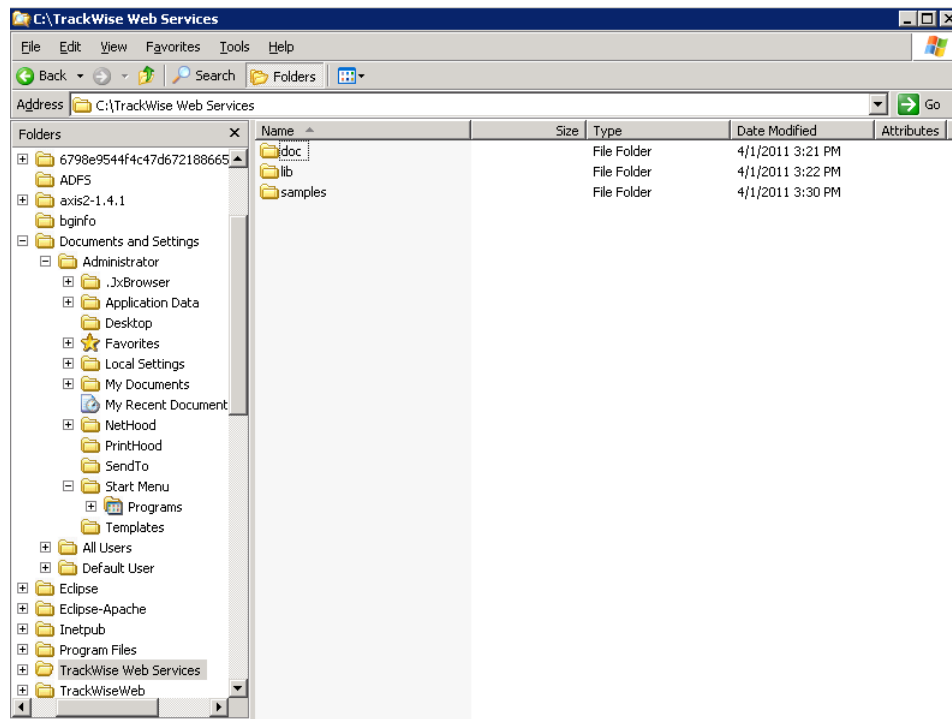
Note: The generic form of the URL is `http://<web server>:<port>/<web app folder name>`

2.2.2 Extracting and setting up the Sample Client

To verify that TWS is functioning appropriately, we will install, configure and run the sample java client provided on the TrackWise CD. This section will extract and set up the client for run in Eclipse.



1. From C:\Course Resources, extract the contents of **TWS sample-java-client.zip** to C:\TrackWise Web Services.
2. Under C:\TrackWise Web Services\samples, extract the content of **sample-src.zip** to C:\TrackWise Web Services\samples\samples-src.



3. The samples.properties file is a properties file leveraged by the sample web service client. Edit this file in **C:\TrackWise Web Services\samples\samples-src** folder as follows:

- **tw.s.url** – The URL for the web services installation in the form of *http://<host>:<port>/axis2/services/TrackWise*
- **user.name** – The login id of the TrackWise user account that will be used by this web service
- **user.password** – The password for the TrackWise user account that will be used by this web service
- **querypr.prid** – The PR ID that this web service will query for

C:\TrackWise Web Services\samples.properties - Notepad++

File Edit Search View Encoding Language Settings Macro Run TextFX Plugins Window ?

samples.properties

```

1  # This is the complete URL to the TrackWise Web Service
2  tws.url=http://localhost:8080/axis2/services/TrackWise
3
4  # Name of the TrackWise user
5  user.name=admin
6
7  # Password of the TrackWise user
8  user.password=admin
9
10 # TimeZone of the TrackWise user location
11 user.timezone=America/New_York
12
13 # The id of the PR to query for
14 # Used by QueryPR.java
15 querypr.prid=1001
16
17 # This is the PR title to search for
18 # Used by QueryPRByFields.java
19 queryprbyfields.prtitle=PR 1
20
21 # The Division to create the PR in
22 # Used by CreatePR.java
23 createpr.divisionname=Division 1
24
25 # The Project to create the PR in
26 # Used by CreatePR.java
27 createpr.projectname=Project 1
28
29 # The Entity id to update

```

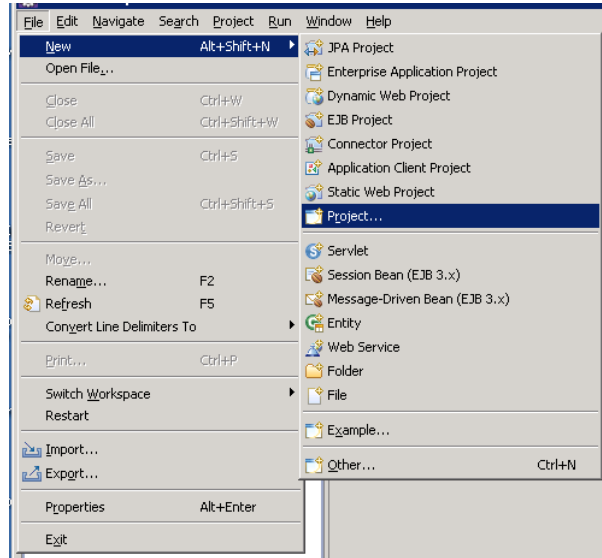
Properties file length: 1249 lines: 51 Ln: 15 Col: 18 Sel: 0 Dos\Windows ANSI INS

4. Access the **twos.url** through Internet Explorer to confirm that it is valid.

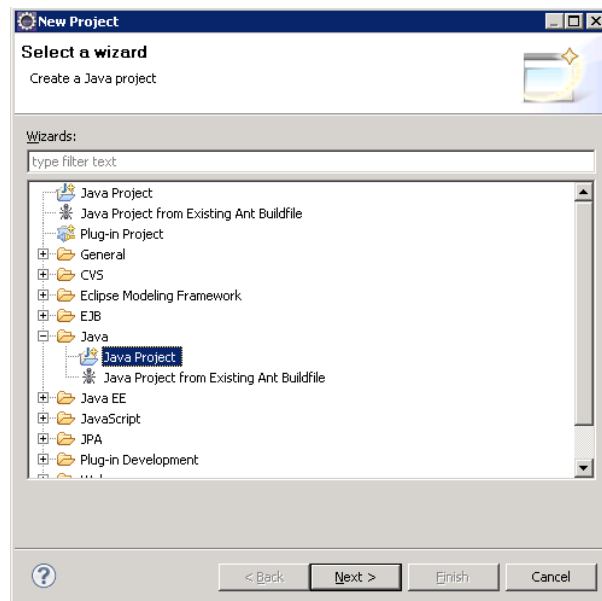
Note: The full Endpoint Reference (/services/TrackWise) does not interact with a web browser. Either use http://<host>:<port>/axis2, or append ?wsdl to the URL. In the samples.properties file use http://<host>:<port>/axis2/services/TrackWise.

2.2.3 Creating and Testing the IWS Project in Eclipse

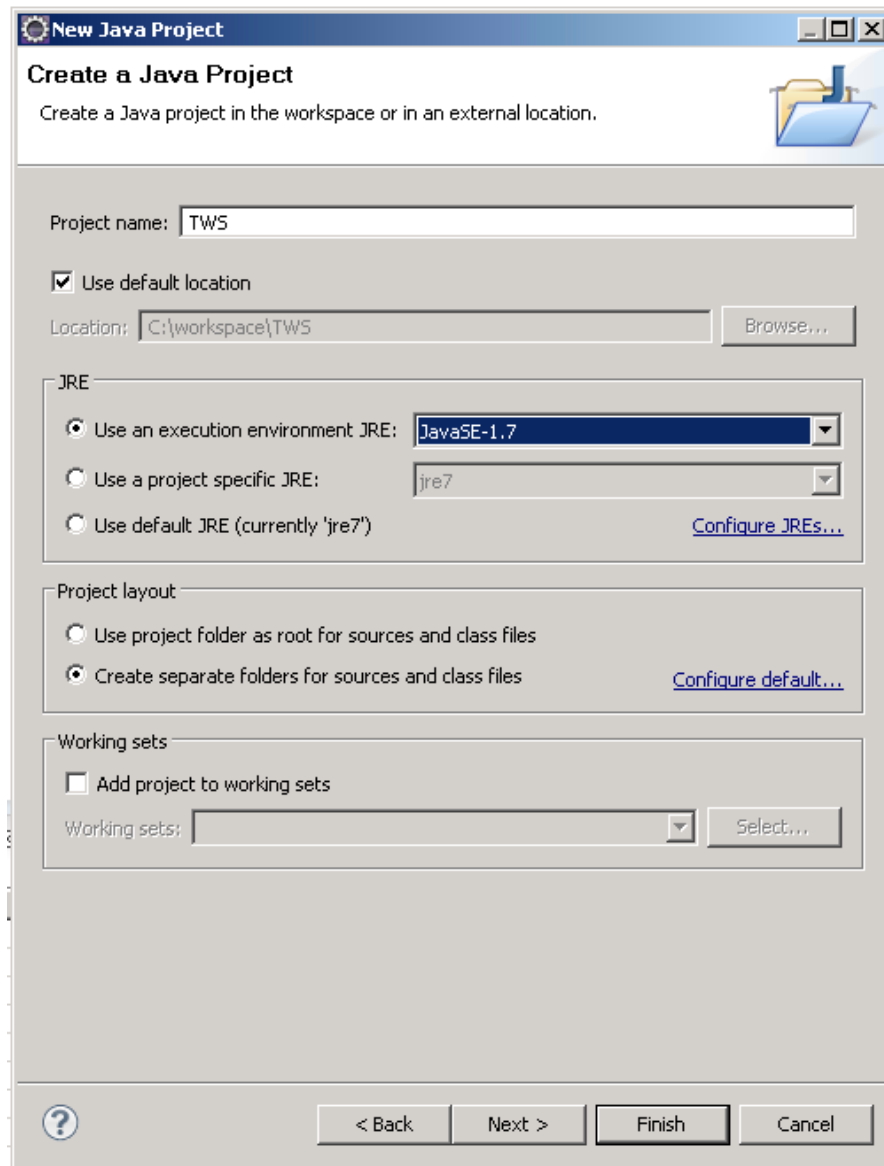
- Once Eclipse is loaded, we will import the provided sample client into a new project. Create a new project by selecting **New-Project...** from the **File** menu.



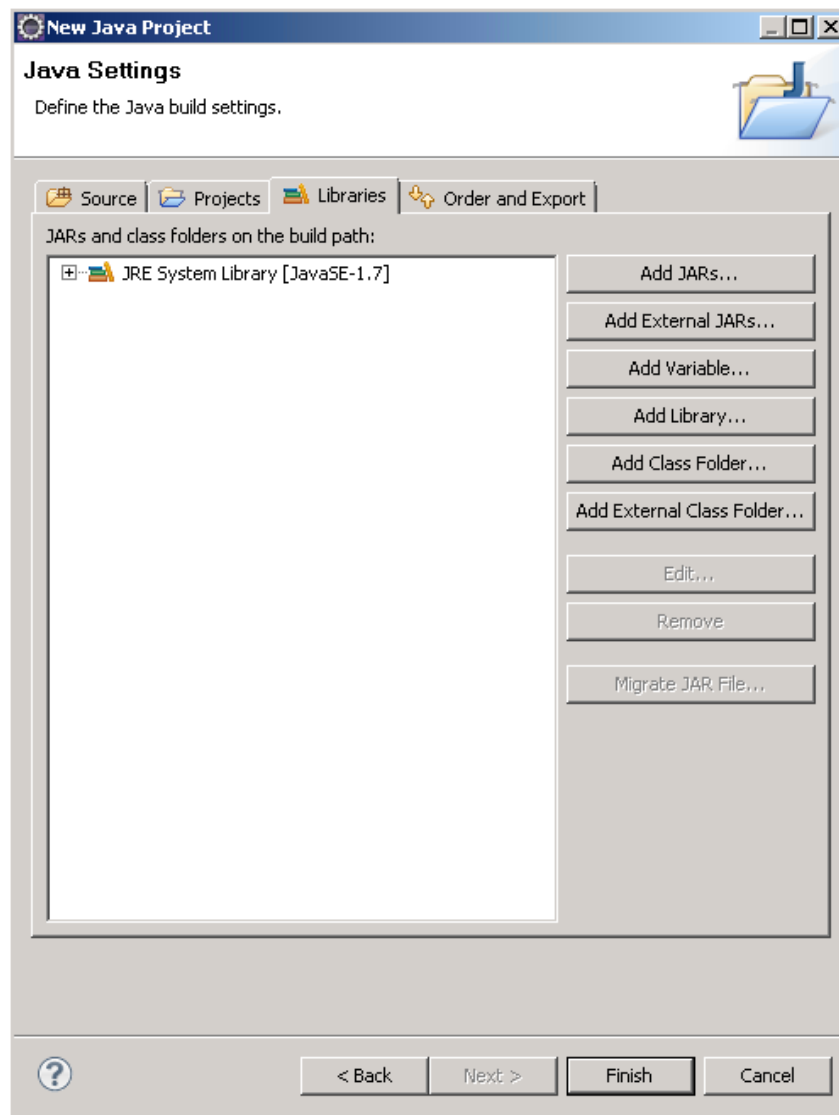
- Select **Java Project** and click **Next**.



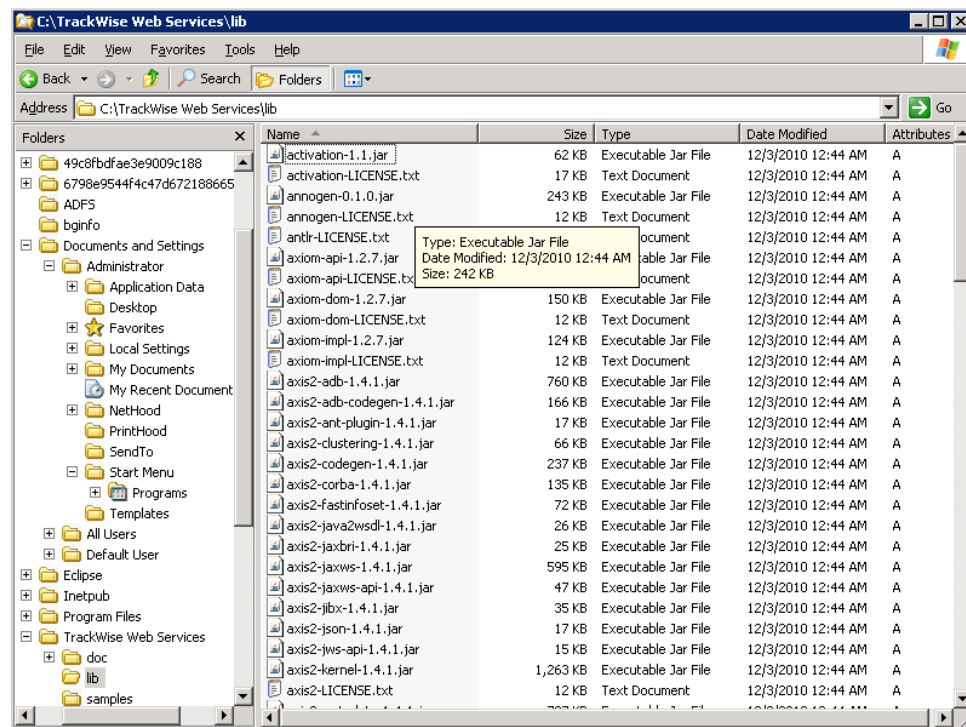
3. Name your project **TWS** and click **Next**.



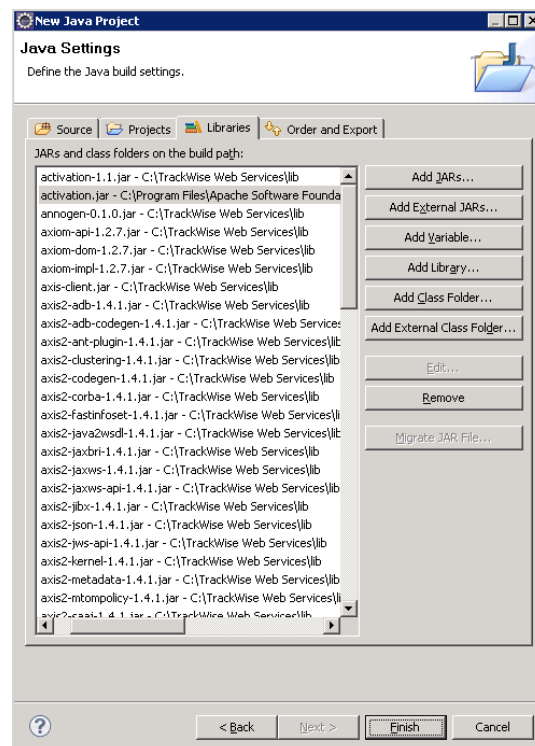
4. We need to now import the libraries necessary for the sample code. On the **Libraries** tab, select **Add External JARs**.



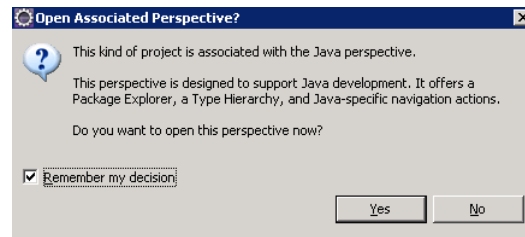
5. Add all of the .jar files located in the lib folder of the extracted TWS sample-java-client C:\TrackWise Web Services\lib.



6. Click **Finish** to create the project.

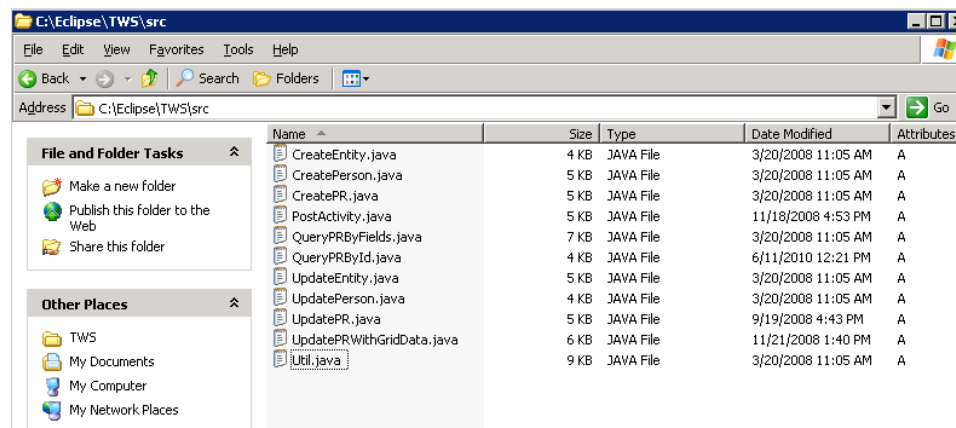


7. Eclipse will prompt you to open the Java perspective when creating this type of project. Select **Remember my decision** and click **Yes**.

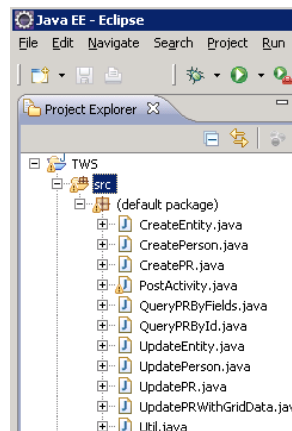


8. To import the source code from the samples, copy all of the .java files from the extracted files in the previous section (located in the **C:\TrackWise Web Services\sample\samples-src** folder) to your **<Eclipse workspace>\<Eclipse project>\src** folder.

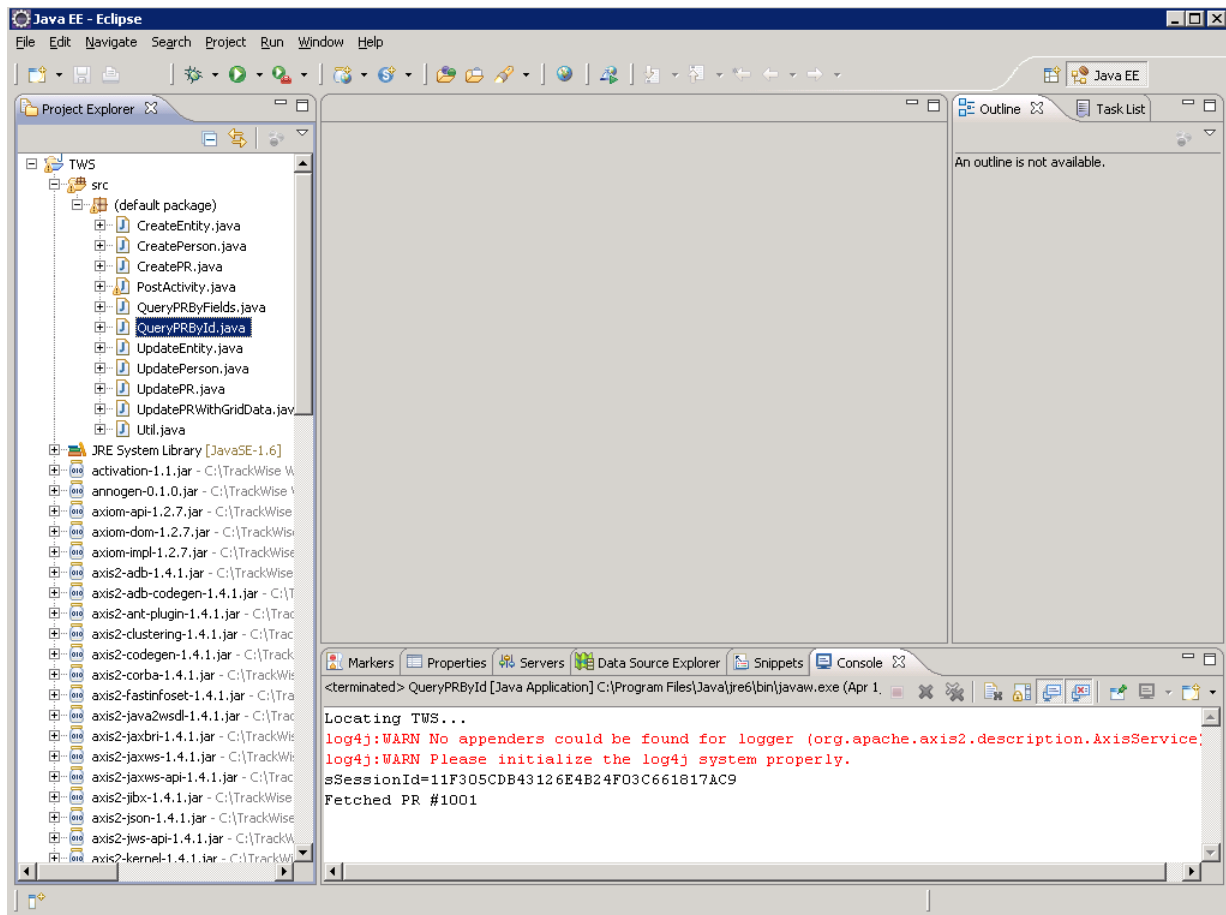
9.



10. Refresh the Eclipse project and see that the source files are now included in Eclipse.



11. Select **QueryPRById.java** and click **Run**. The output will be written to the console window.



The java sample client has been imported into a project successfully. Moving forward, the code used in these samples can be leveraged in other programs that will interact with TrackWise Inbound Web Services.

2.3 Outbound Web Services (OWS) Setup

Several steps are needed to ready the eclipse environment to write and test OWS programs. The following sections will take you through the process.

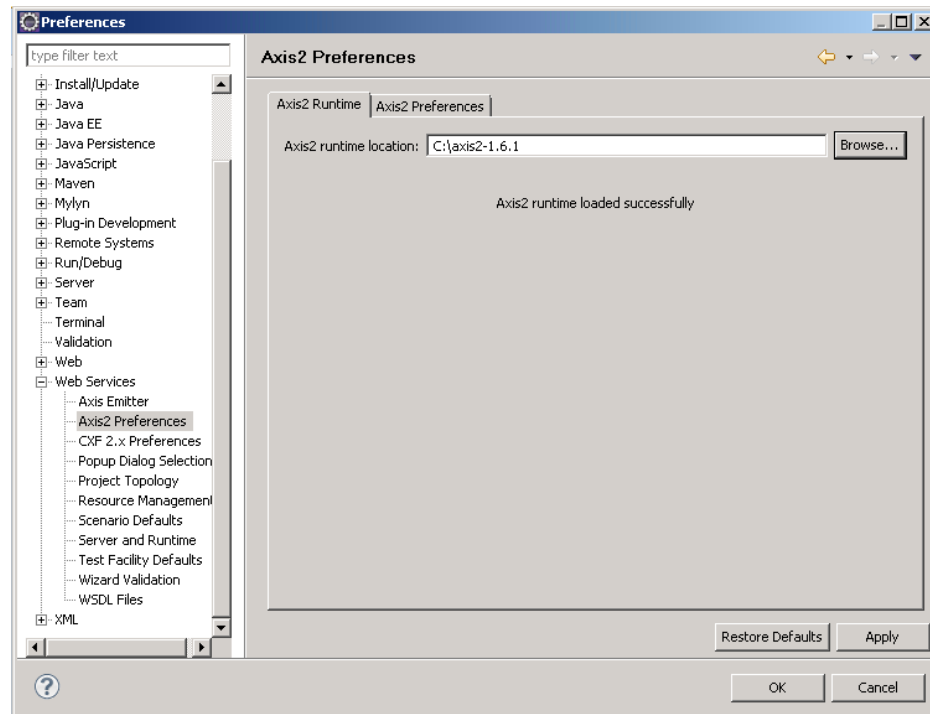
2.3.1 Specifying the Axis2 Runtime Environment

The Axis2 Runtime will need to be specified to develop and test an outbound web service.

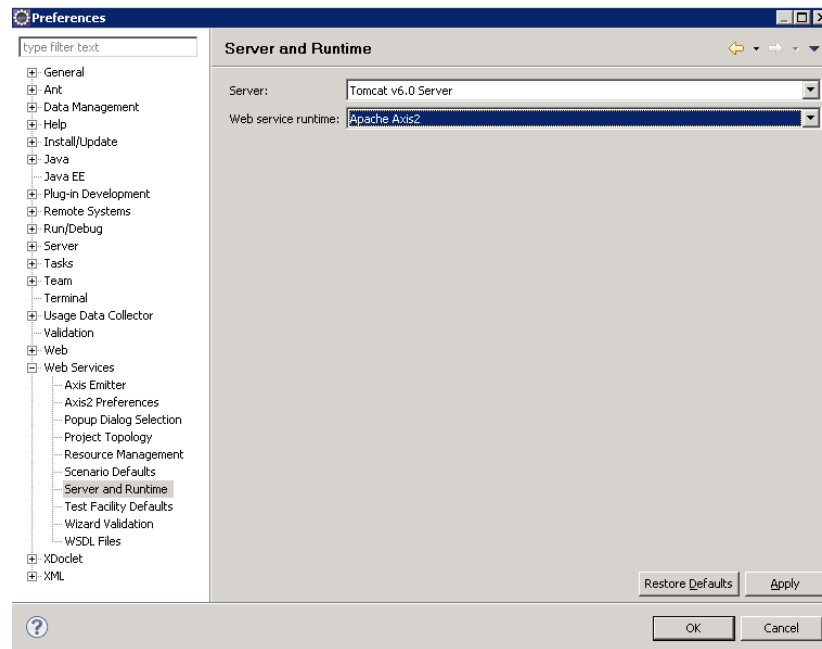


1. Select **Preferences** from the **Window** menu. In the **Preferences** dialog, expand the Web Services tree on the left and select the **Axis2 Preferences** option. You may download the axis2 binary from the internet

at <http://axis.apache.org/axis2/java/core/download.cgi>. For this course, use the **axis2-1.6.1** Zip file provided at **C:\Course Resources\WebService Resources\axis2-1.6.1.zip**, Extract it into **C: Drive**.



2. Select the **Server and Runtime** option from the Web Service tree on the left. Change the **Web service runtime** from Apache Axis to **Apache Axis2**.



3. Close out of the **Preferences** window by clicking **OK**.

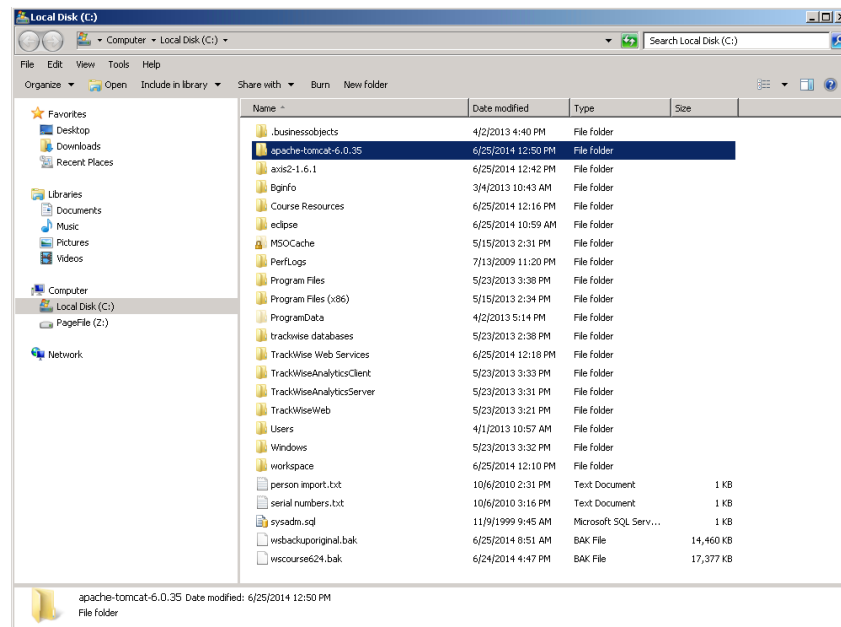
2.3.2 Installing a Tomcat Server in Eclipse

We will install a second Tomcat server in Eclipse to host the web services being developed. The benefit of hosting a server through Eclipse is that Eclipse automatically deploys the web service to the server and that you may restart the web service server without restarting your TrackWise server.

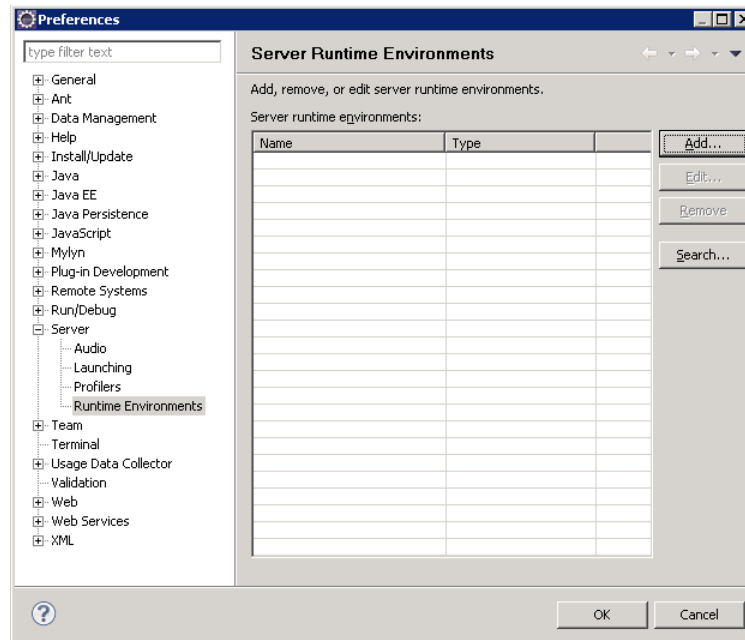


To install a server:

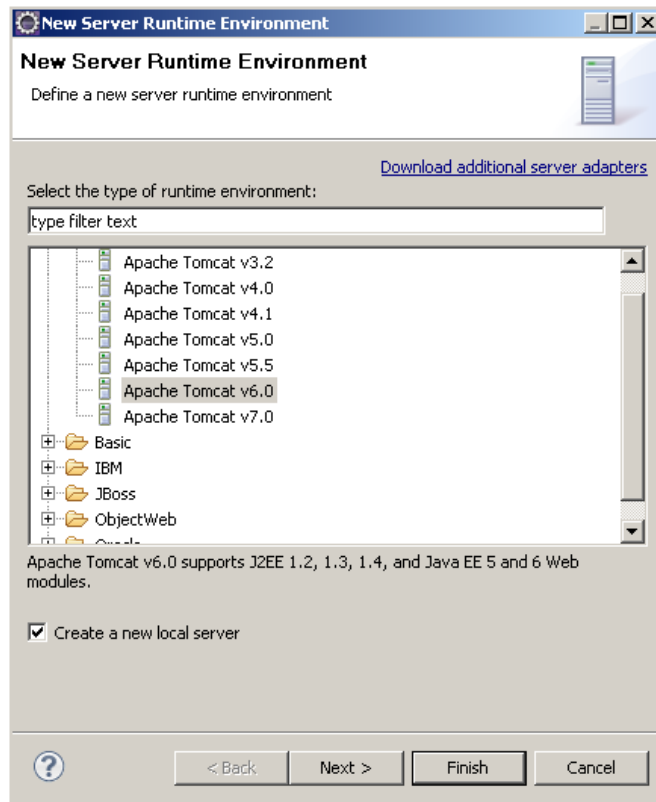
1. Unzip the apache tomcat installation **apache-tomcat-6.0.35.zip** located in **the C:\Course Resources\WebService Resources** folder into the **C:** directory. These files will be referenced by eclipse for the Tomcat server.



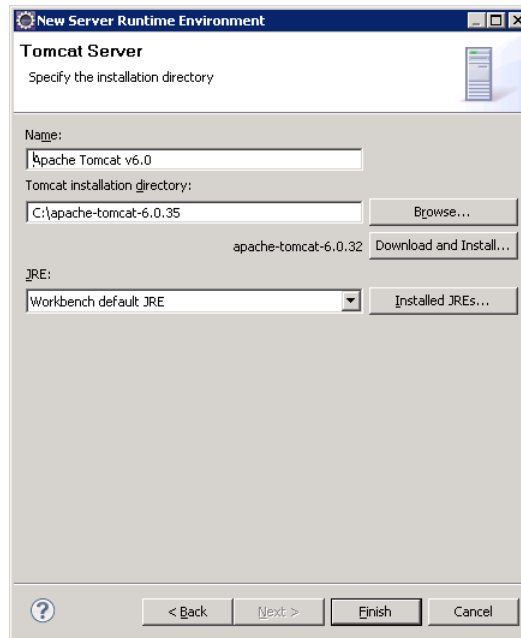
2. Select **Preferences** from the window menu. Expand the server tree on the left hand side and select **Runtime Environments**. Click **Add**.



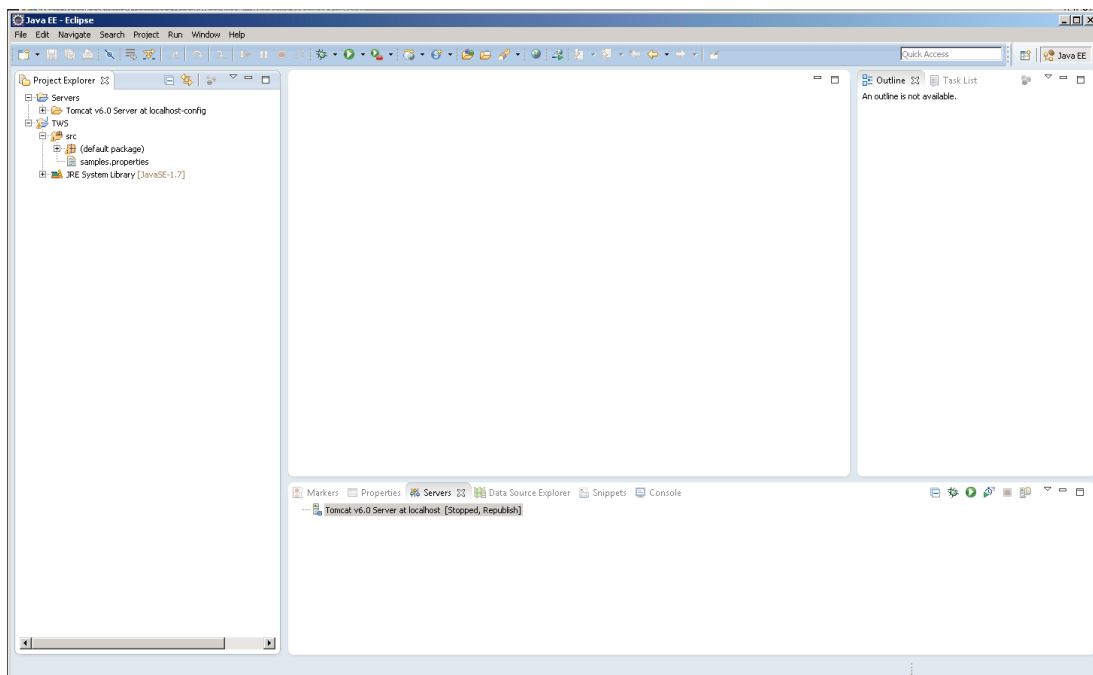
3. Open the Apache folder and select **Apache Tomcat v6.0** , Select the “**Create a new Local Server**” Option and click **Next**.



4. Locate the Tomcat directory that you previously unzipped the Tomcat server files to. In this course, the directory was **C:\apache-tomcat-6.0.35**. Click **Finish**. Click **Finish** then **OK** to close out the **Preferences** dialog.

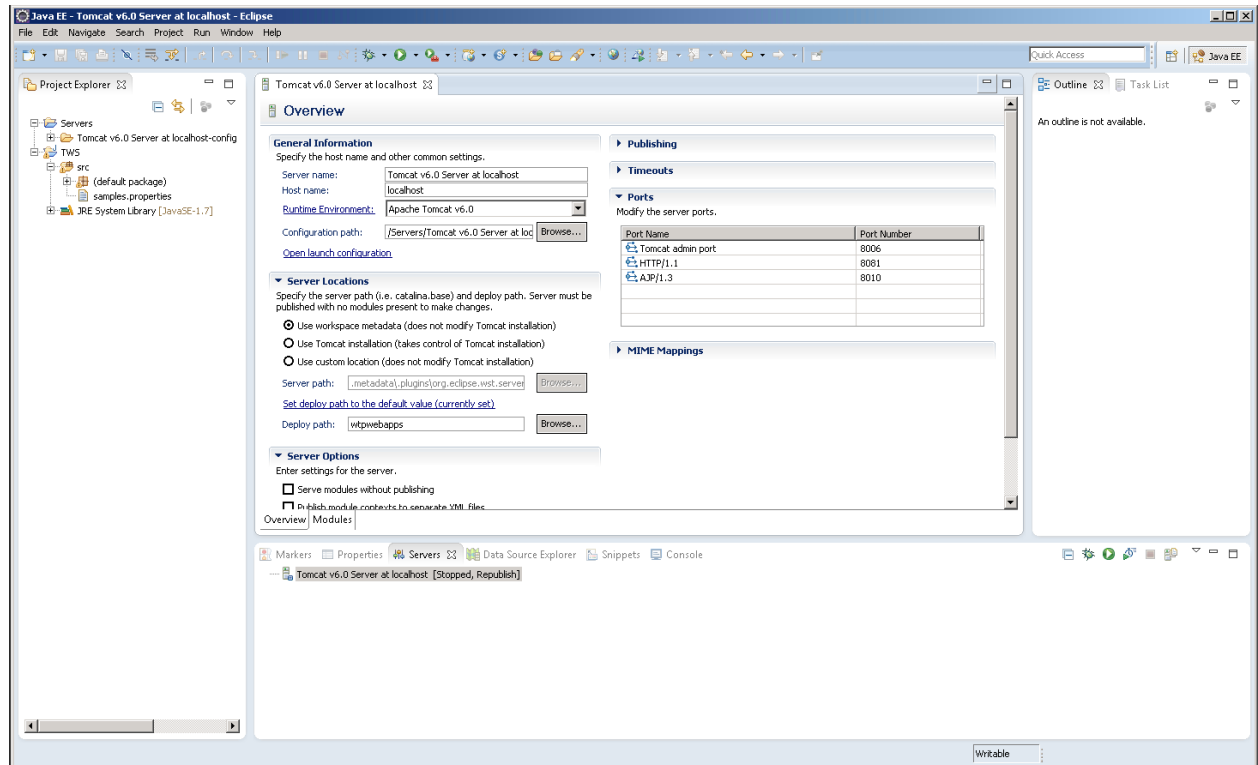


5. Go to **Window** from the menu, select **Show View - Servers**.
6. Click on the **new server wizard** link in the server window that appears.



7. Expand the Apache folder and select **Tomcat v6.0 Server**. Click **Next**. Then **Finish**.

8. Double-click the Server to open the server configuration settings. Since a Tomcat server is already installed on the machine edit the ports of the Eclipse server so that they do not interfere with the local Tomcat server. Change the **Tomcat admin port** to **8006**. The **HTTP/1.1** port to **8081**, and the **AJP/1.3** port to **8010**.



9. Save the server configuration. Click the save button on the tool bar or go to the menu and select **File -> Save**.

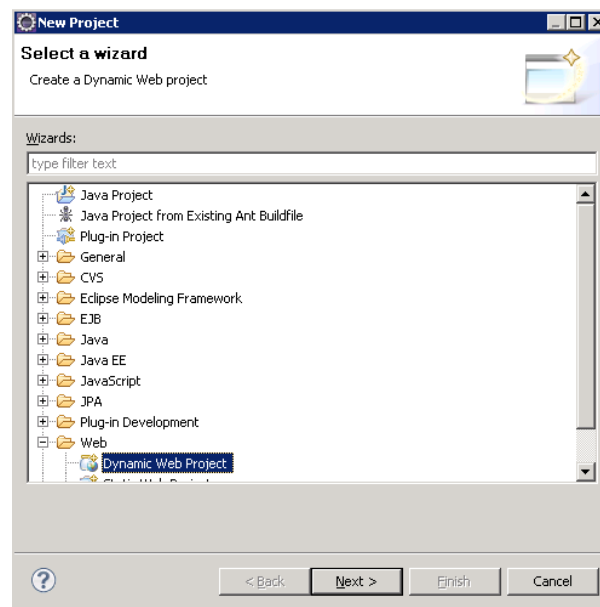
2.3.3 Creating an OWS project in Eclipse

Creation of an Outbound Web Service project in Eclipse is slightly different than the previous exercise to create and IWS.

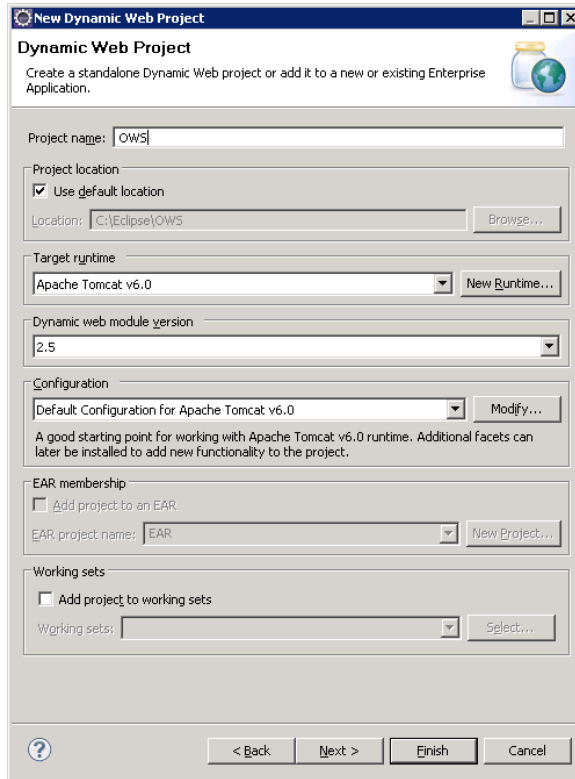


Workbook Exercises

1. Click the **New** button in the top left corner to bring up the **New Project** dialog. Scroll down in the window and expand the **Web** folder. Select **Dynamic Web Project**. Click **Next**.



2. Specify the **Project name** as **OWS**. Ensure that the **Target runtime** that is selected is **Apache Tomcat v6.0**. Click **Finish**.



New Dynamic Web Project

Dynamic Web Project
Create a standalone Dynamic Web project or add it to a new or existing Enterprise Application.

Project name:

Project location
☒ Use default location
Location:

Target runtime

Dynamic web module version

Configuration

A good starting point for working with Apache Tomcat v6.0 runtime. Additional facets can later be installed to add new functionality to the project.

EAR membership
☐ Add project to an EAR
EAR project name:

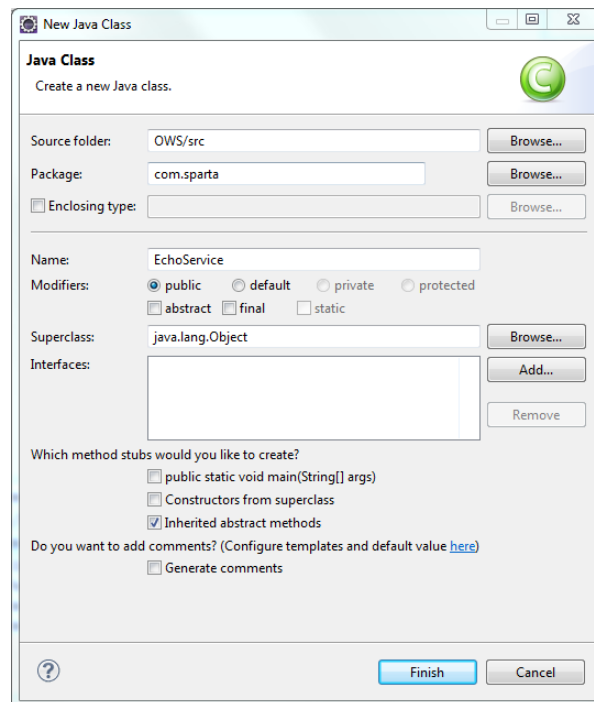
Working sets
☐ Add project to working sets
Working sets:

2.3.4 Creating a simple OWS

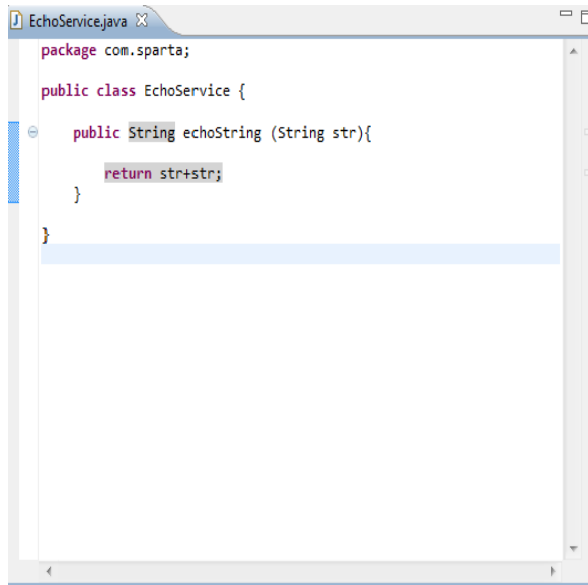
In this sub section we will create a simple OWS which takes a String as the input and “echo’s” the String as the output.



1. Expand the **Java Resources** folder and select **New-Class**. Type **com.sparta** for the package, name your new class **EchoService** and click **Finish**.



2. Complete the class by creating a simple public method called **echoString** that will take a string as a parameter and return the string concatenated with itself. Save the class file when complete.



```
package com.sparta;

public class EchoService {

    public String echoString (String str){
        return str+str;
    }

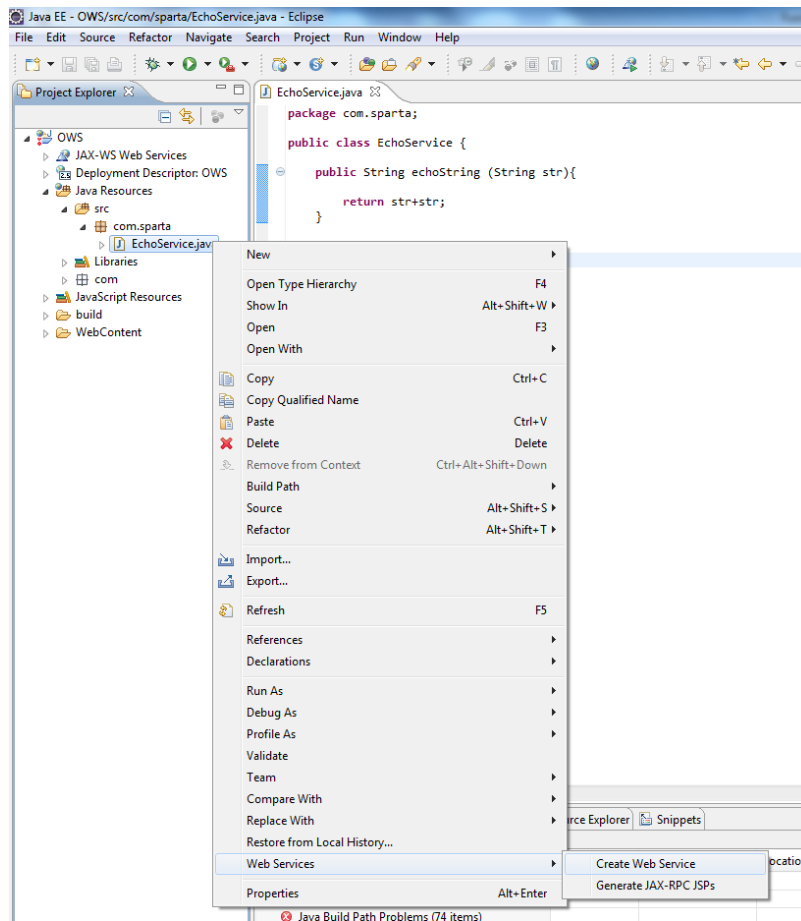
}
```

2.3.5 Deploy the Web Service in Eclipse

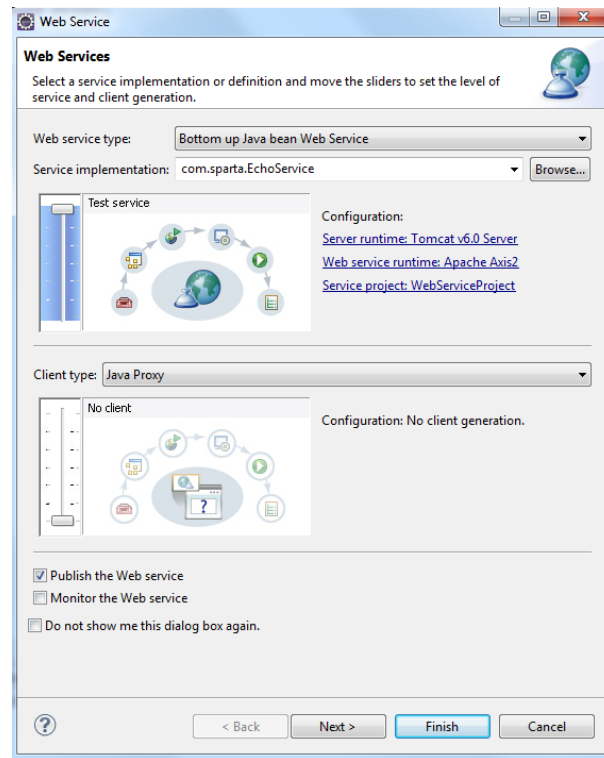
In order to use the EchoService web service in TrackWise, it will have to be wrapped as a service and deployed.



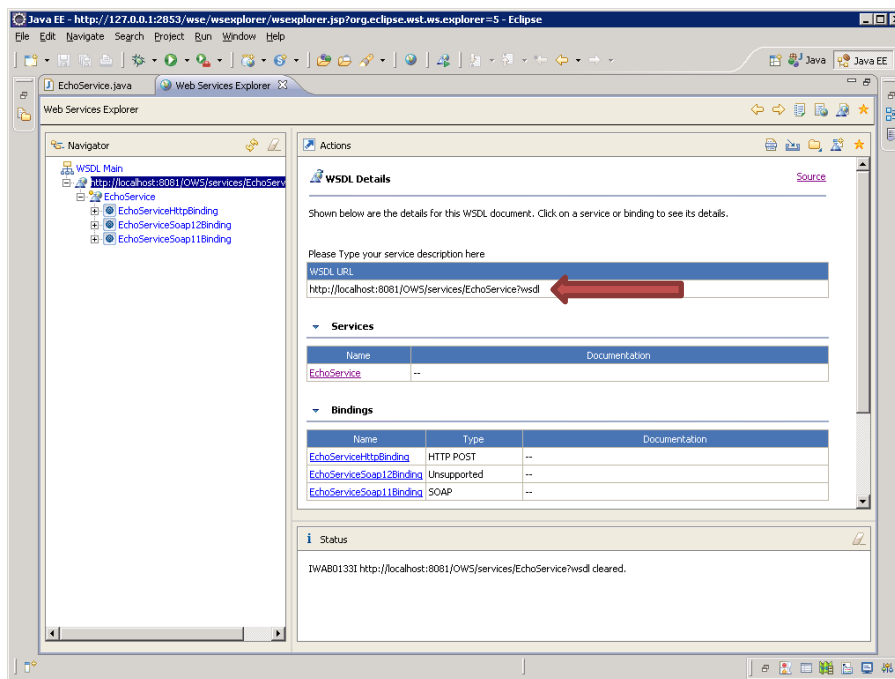
1. Right click the **EchoService.java** file. Select **Web Services -Create Web service**.



2. Move the slider to **Test service** and click **Finish**.



3. Click the URL in the tree to the left in the navigator pane that appears in the **Web Services Explorer** window. Copy the **WSDL URL** from the **Actions** pane.

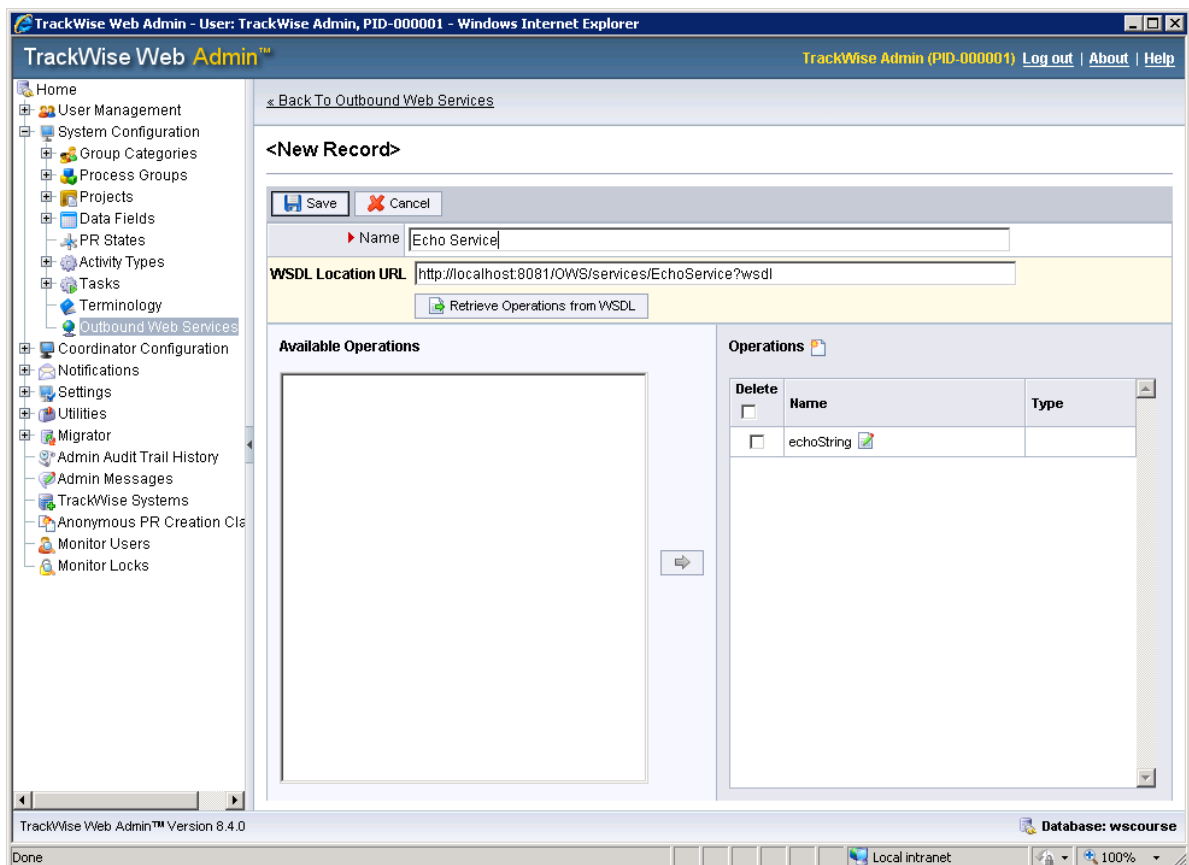


2.3.6 Configure OWS in Web Admin

TrackWise must be configured in order to take advantage of the OWS we just built.



1. Log into TrackWise WebAdmin.
2. Under the **System Configuration** tree, select the **Outbound Web Services** item. Click **New**. Specify a name for the OWS configuration, paste the WSDL location into the appropriate box and click **Retrieve Operations from WSDL**. Move the web service from the **Available Operations** window to the **Operations** window. Click **Edit** next to the web service name.



TrackWise Web Admin - User: TrackWise Admin, PID-000001 - Windows Internet Explorer

TrackWise Admin (PID-000001) [Log out](#) | [About](#) | [Help](#)

Home

- User Management
- System Configuration
 - Group Categories
 - Process Groups
 - Projects
 - Data Fields
 - PR States
 - Activity Types
 - Tasks
 - Terminology
 - Outbound Web Services
- Coordinator Configuration
- Notifications
- Settings
- Utilities
- Migrator
- Admin Audit Trail History
- Admin Messages
- TrackWise Systems
- Anonymous PR Creation Cla
- Monitor Users
- Monitor Locks

Back To Outbound Web Services

<New Record>

Save Cancel

Name Echo Service

WSDL Location URL http://localhost:8081/OWS/services/EchoService?wsdl

Retrieve Operations from WSDL

Available Operations

Operations

Delete	Name	Type
☐	echoString	

TrackWise Web Admin™ Version 8.4.0

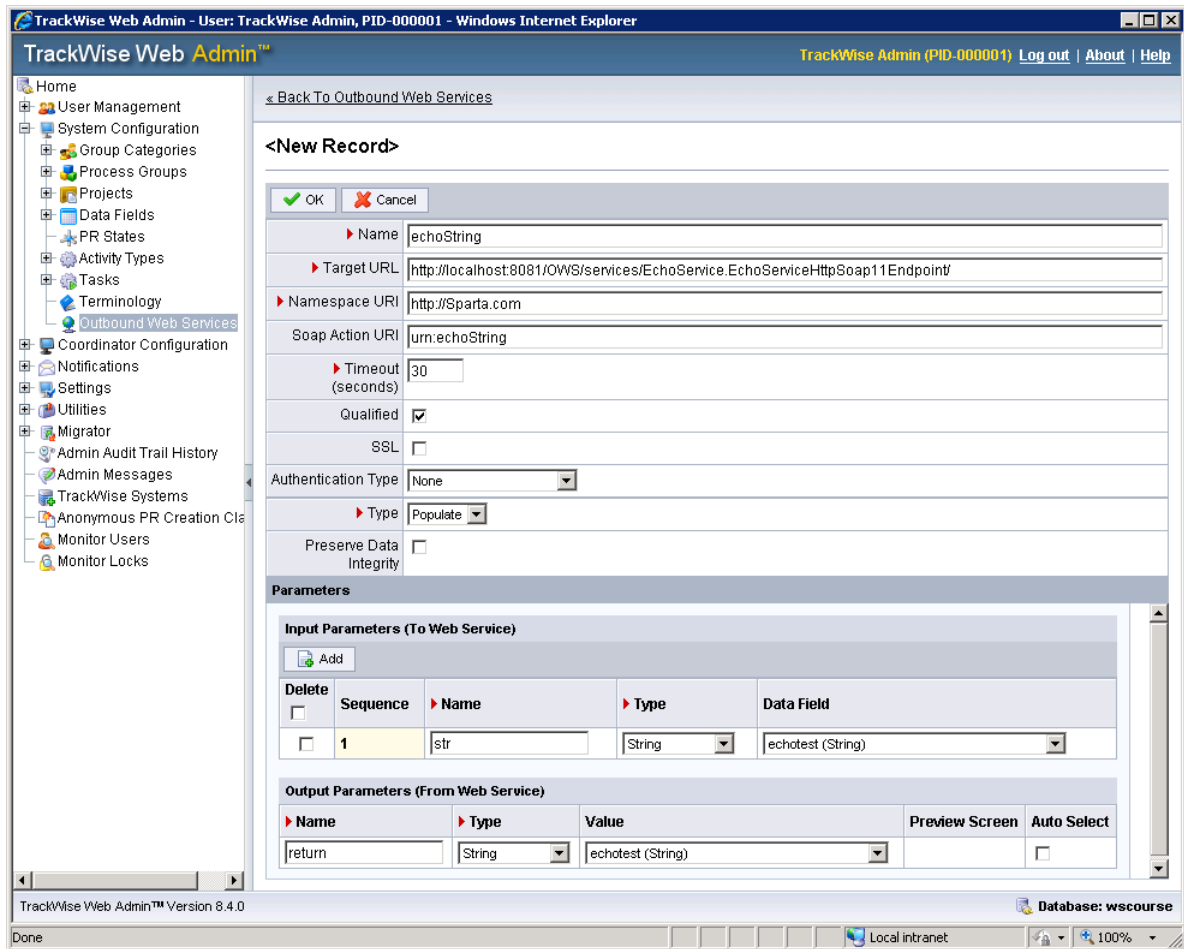
Database: wscourse

Done

Local intranet

100%

3. Select **Populate** as the type. Select the **Input** and **Output** data fields. Use the **echotest** field as both the input and the output. Check **Auto Select** box. Click **OK** and **Save** the outbound web service.



TrackWise Web Admin - User: TrackWise Admin, PID-000001 - Windows Internet Explorer

TrackWise Admin (PID-000001) [Log out](#) | [About](#) | [Help](#)

Home

- User Management
- System Configuration
 - Group Categories
 - Process Groups
 - Projects
 - Data Fields
 - PR States
 - Activity Types
 - Tasks
 - Terminology
 - Outbound Web Services
- Coordinator Configuration
- Notifications
- Settings
- Utilities
- Migrator
- Admin Audit Trail History
- Admin Messages
- TrackWise Systems
- Anonymous PR Creation Cla
- Monitor Users
- Monitor Locks

Back To Outbound Web Services

<New Record>

OK Cancel

Name: echoString

Target URL: http://localhost:8081/OWS/services/EchoService.EchoServiceHttpSoap11Endpoint?

Namespace URI: http://Sparta.com

Soap Action URI: urn:echoString

Timeout (seconds): 30

Qualified: ☒

SSL: ☐

Authentication Type: None

Type: Populate

Preserve Data Integrity: ☐

Parameters

Input Parameters (To Web Service)

Add

Delete	Sequence	Name	Type	Data Field
☐	1	str	String	echotest (String)

Output Parameters (From Web Service)

Name	Type	Value	Preview Screen	Auto Select
return	String	echotest (String)		☐

TrackWise Web Admin™ Version 8.4.0

Database: wscourse

Done

Local intranet

100%

- Assign the outbound web service operation to the data field. Select the **Trigger Outbound Web Services** box, select the **echoString** operation from the drop down and enter **Echo** for the **Populate Button Text**. Restart TrackWise Web for the changes to take place.

TrackWise Web Admin - User: TrackWise Admin, PID-000001 - Windows Internet Explorer

TrackWise Web Admin™ TrackWise Admin (PID-000001) Log out | About | Help

Home | User Management | System Configuration | Group Categories | Process Groups | Projects | **Data Fields** | PR States | Activity Types | Tasks | Terminology | Outbound Web Services | Coordinator Configuration | Notifications | Settings | Utilities | Migrator | Admin Audit Trail History | Admin Messages | TrackWise Systems | Anonymous PR Creation Cts | Monitor Users | Monitor Locks

Back To Data Fields Process Group: Complaints

Delete Report

Data Field Details: echotest Associated With Process Group

Details

Save Save+ Cancel

System Settings

Field Name - Customized	echotest	Highlight Change	☐
Field Type	String	Meaning Comment Req'd	☐
Field Category	PR (User Defined)		
Is Visible	☒	Seq. No	55
		Is Virtual	☐
Copy To Child	☐	Audit Trail Activity Type Triggered	<No Options Found>
Max Length		Trigger Outbound Web Services	☒

Outbound Web Services Configuration

Search Operation	<No Options Found>		
Populate Operation	echoString	Populate Button Text	Echo

TrackWise Web Admin™ Version 8.4.0 Database: wscourse

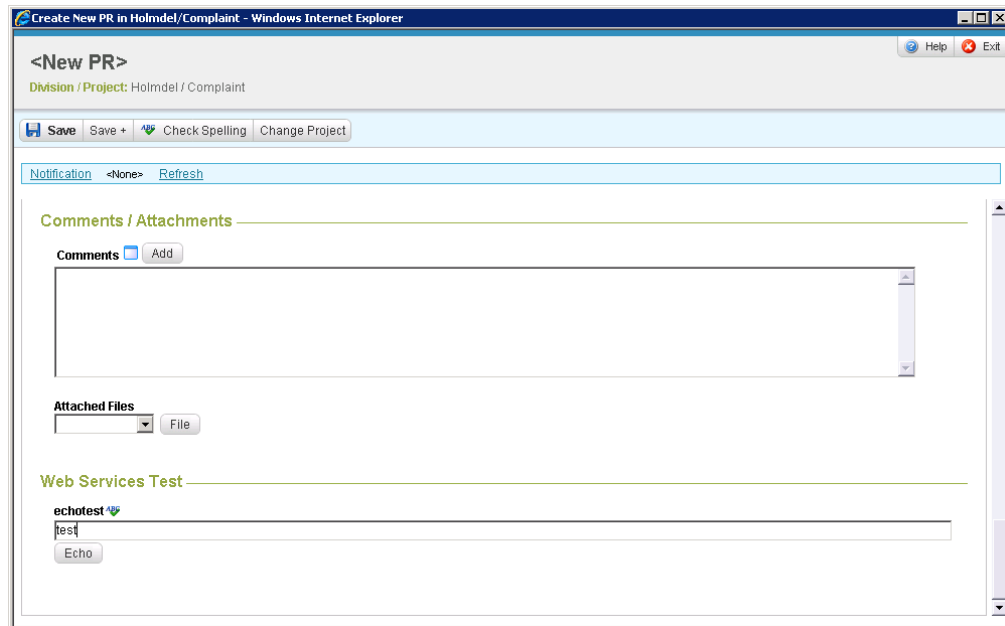
Done Local intranet 100%

2.3.7 Test OWS in TrackWise

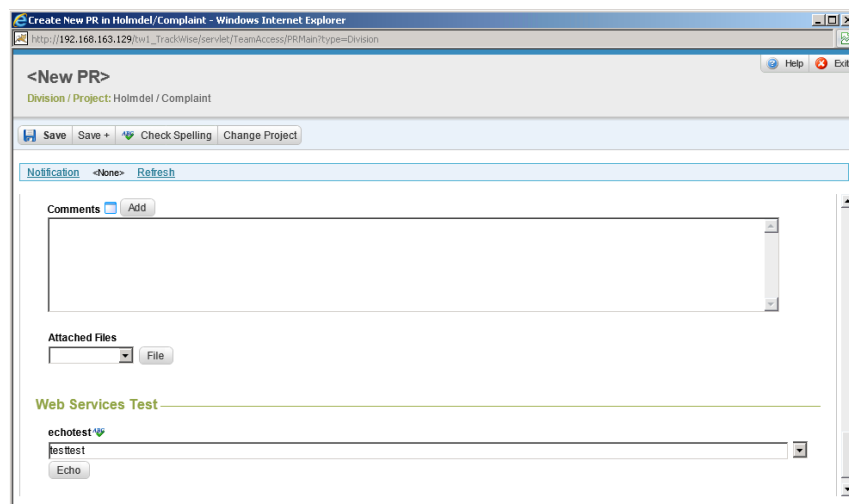
The OWS should be tested in TrackWise to verify functionality. In order to test, the Tomcat instance that TrackWise is running on top of must first be restarted.



1. Place a string into the **Web Service** field and click **Echo**.



2. See that the expected string was returned by the web service.



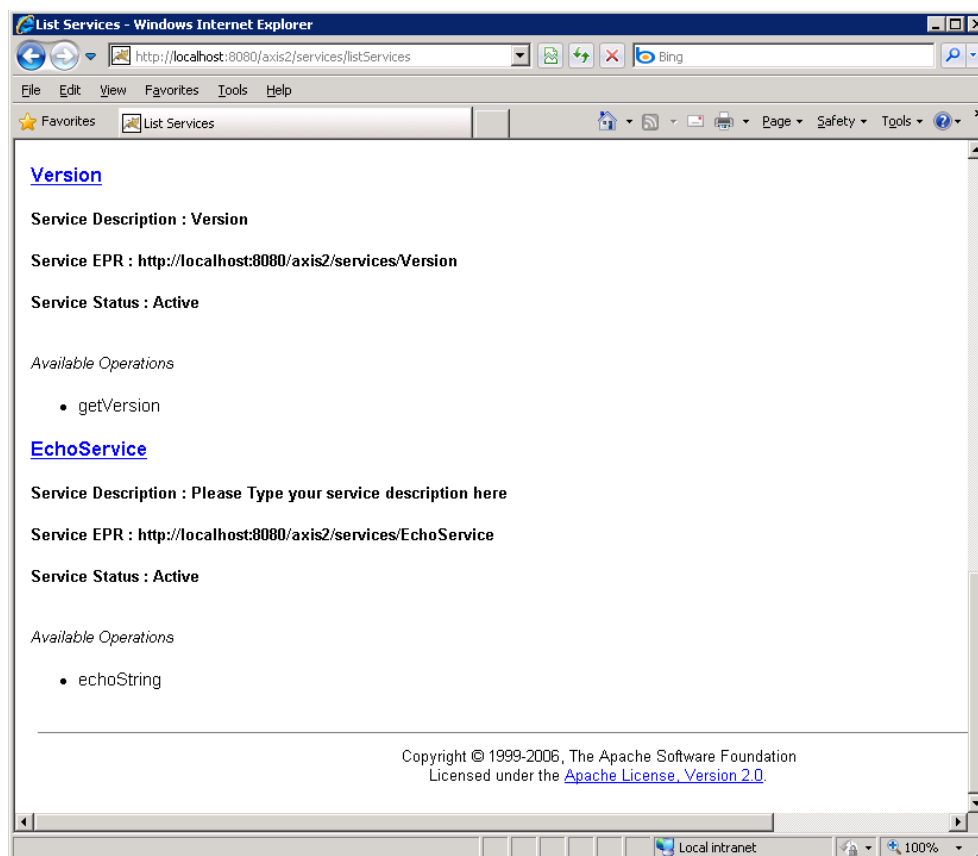
2.3.8 Transferring an OWS from Eclipse to another Tomcat Installation

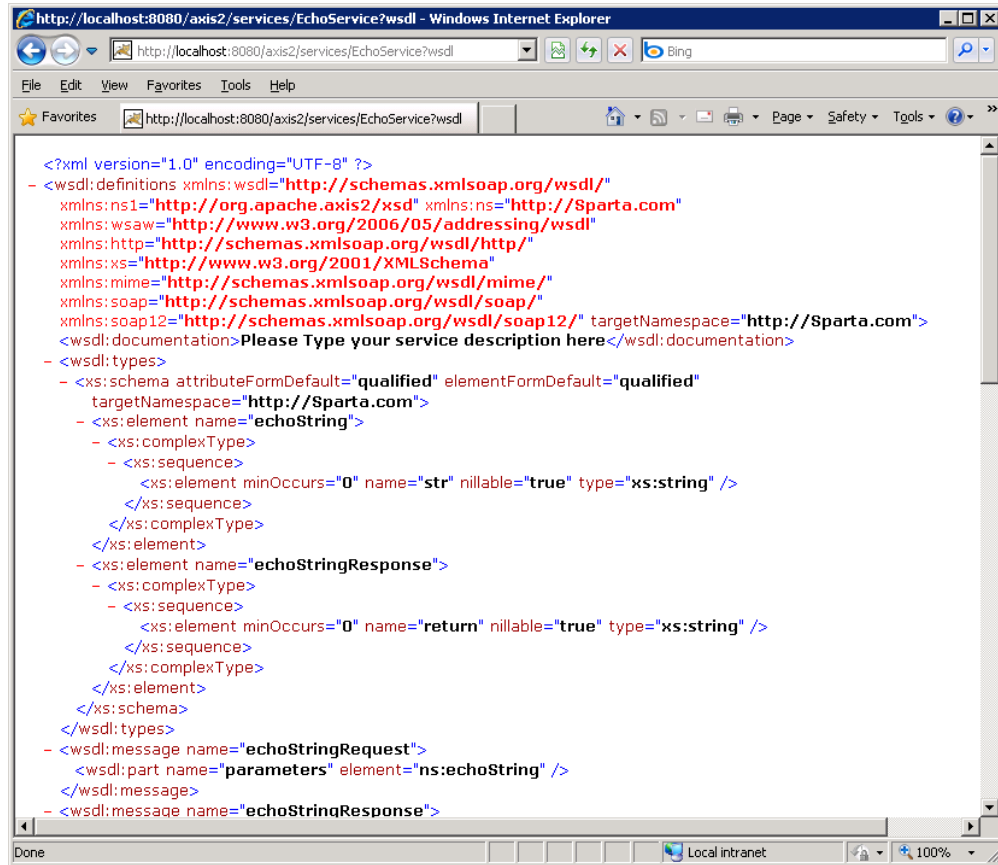
After you have completed development of the web service you will want to deploy the web service on another Tomcat server outside of the development environment and point the TrackWise installation to the newly deployed service.



Workbook Exercises

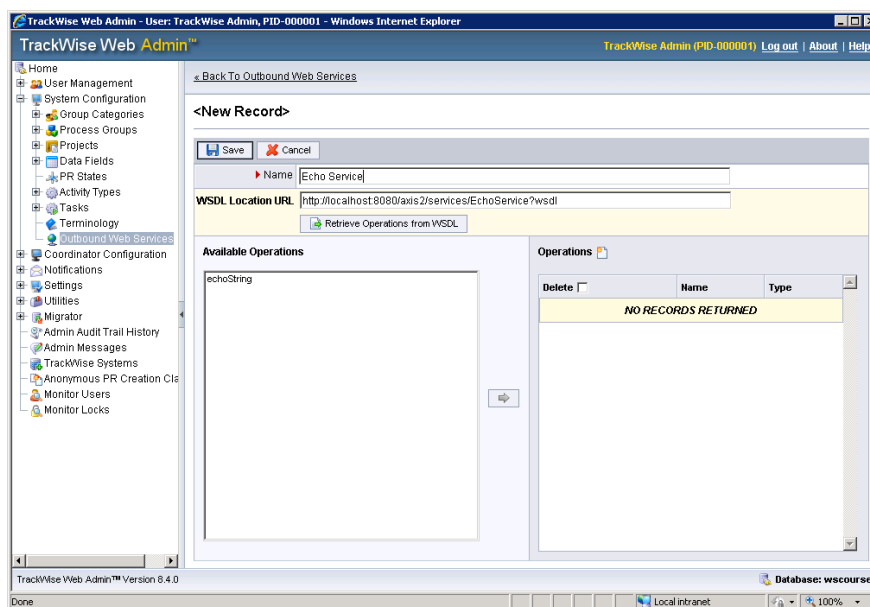
1. The web service **EchoService Folder** resides under **<Eclipse Workspace>\<Eclipse Project>\Web Content\WEB-INF\services** . Copy this folder to the new **<local Tomcat>\<webapps>\<axis2>\WEB-INF\services** directory.
2. After copying the service folder the service should be deployed automatically.
3. Confirm that the service is available on the local Tomcat instance by going to <http://localhost/axis2/services/listServices>. Click the **EchoService** link to confirm that the .wsdl is displayed.





```
<?xml version="1.0" encoding="UTF-8" ?>
- <wsdl:definitions xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/"
  xmlns:ns1="http://org.apache.axis2/xsd" xmlns:ns="http://Sparta.com"
  xmlns:wsaw="http://www.w3.org/2006/05/addressing/wsdl"
  xmlns:http="http://schemas.xmlsoap.org/wsdl/http/"
  xmlns:xs="http://www.w3.org/2001/XMLSchema"
  xmlns:mime="http://schemas.xmlsoap.org/wsdl/mime/"
  xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
  xmlns:soap12="http://schemas.xmlsoap.org/wsdl/soap12/" targetNamespace="http://Sparta.com">
  <wsdl:documentation>Please Type your service description here</wsdl:documentation>
  - <wsdl:types>
  - <xs:schema attributeFormDefault="qualified" elementFormDefault="qualified"
    targetNamespace="http://Sparta.com">
    - <xs:element name="echoString">
      - <xs:complexType>
      - <xs:sequence>
      - <xs:element minOccurs="0" name="str" nillable="true" type="xs:string" />
      </xs:sequence>
      </xs:complexType>
    </xs:element>
    - <xs:element name="echoStringResponse">
      - <xs:complexType>
      - <xs:sequence>
      - <xs:element minOccurs="0" name="return" nillable="true" type="xs:string" />
      </xs:sequence>
      </xs:complexType>
    </xs:element>
    </xs:schema>
  </wsdl:types>
  - <wsdl:message name="echoStringRequest">
    <wsdl:part name="parameters" element="ns:echoString" />
  </wsdl:message>
  - <wsdl:message name="echoStringResponse">
```

- Log into WebAdmin and change the WSDL location of the configured OWS instance in Web Admin to point to the local Tomcat server.



5. Update the **Target URL** to point to the correct server and service name. The Target URL can be found towards the bottom of the .wsdl file. Save the new OWS configuration.

Back To Outbound Web Services

Delete

Echo Service

Operation: echoString

OK
Cancel

Name: echoString

Target URL: http://192.168.30.2:8080/EchoService/services/EchoServiceEchoServiceHttpSoap11Endpoint/

Namespace URI: http://Sparta.com

Soap Action URI: urn:echoString

Timeout (seconds): 30

Qualified: ☒

SSL: ☐

Authentication Type: None

Type: Populate

Preserve Data Integrity: ☐

Parameters

Input Parameters (To Web Service)

Add

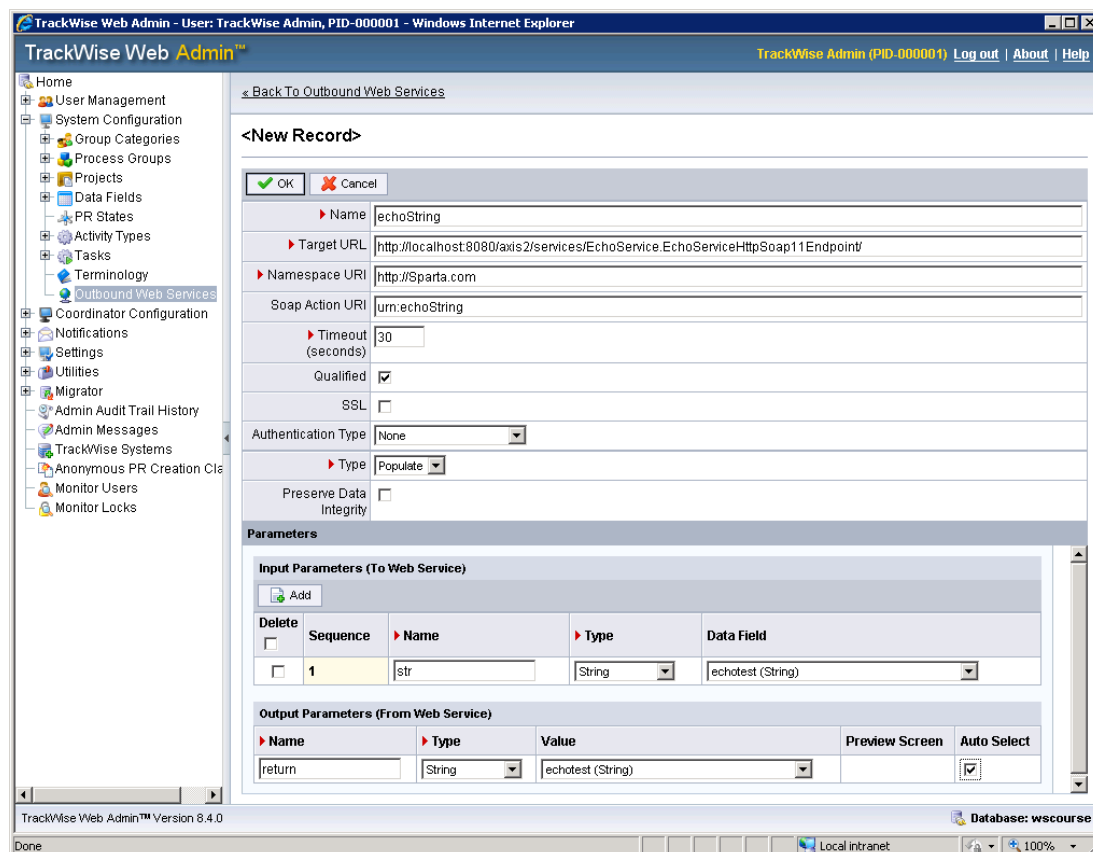
Delete	Sequence	Name	Type	Data Field
☐	1	str	String	OWS String (String)

Output Parameters (From Web Service)

Name	Type	Value	Preview Screen	Auto Select
return	String	OWS String (String)		☒

Database: TWSQL

6. Restart the Tomcat instance that TrackWise is running on. Log into TrackWise and confirm that the OWS call is successful with the new configuration.



TrackWise Web Admin - User: TrackWise Admin, PID-000001 - Windows Internet Explorer

TrackWise Admin (PID-000001) [Log out](#) | [About](#) | [Help](#)

Home

- User Management
- System Configuration
 - Group Categories
 - Process Groups
 - Projects
 - Data Fields
 - PR States
 - Activity Types
 - Tasks
 - Terminology
 - Outbound Web Services
- Coordinator Configuration
- Notifications
- Settings
- Utilities
- Migrator
- Admin Audit Trail History
- Admin Messages
- TrackWise Systems
- Anonymous PR Creation Cls
- Monitor Users
- Monitor Locks

Back To Outbound Web Services

<New Record>

OK Cancel

Name: echoString

Target URL: http://localhost:8080/axis2/services/EchoService.EchoServiceHttpSoap11Endpoint/

Namespace URI: http://Sparta.com

Soap Action URI: urn:echoString

Timeout (seconds): 30

Qualified: ☒

SSL: ☐

Authentication Type: None

Type: Populate

Preserve Data Integrity: ☐

Parameters

Input Parameters (To Web Service)

Add

Delete	Sequence	Name	Type	Data Field
☐	1	str	String	echotest (String)

Output Parameters (From Web Service)

Name	Type	Value	Preview Screen	Auto Select
return	String	echotest (String)		☒

TrackWise Web Admin™ Version 8.4.0

Database: wscourse

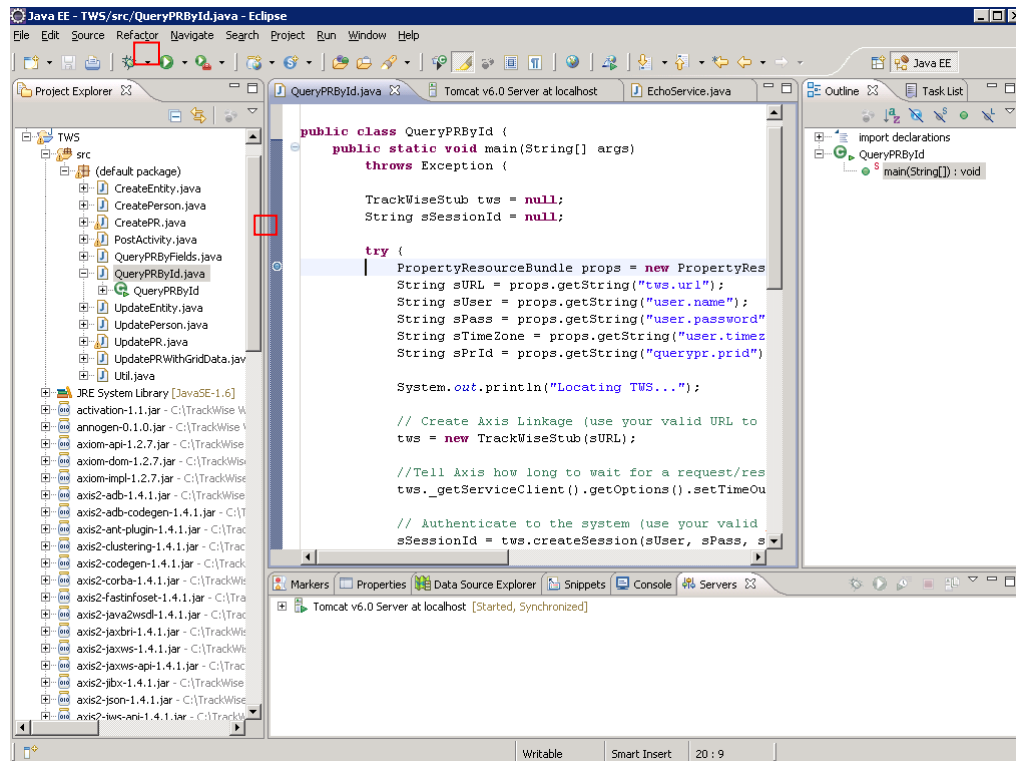
Done

Local intranet

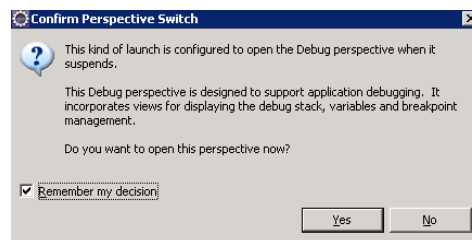
100%

2.4 Debugging in Eclipse

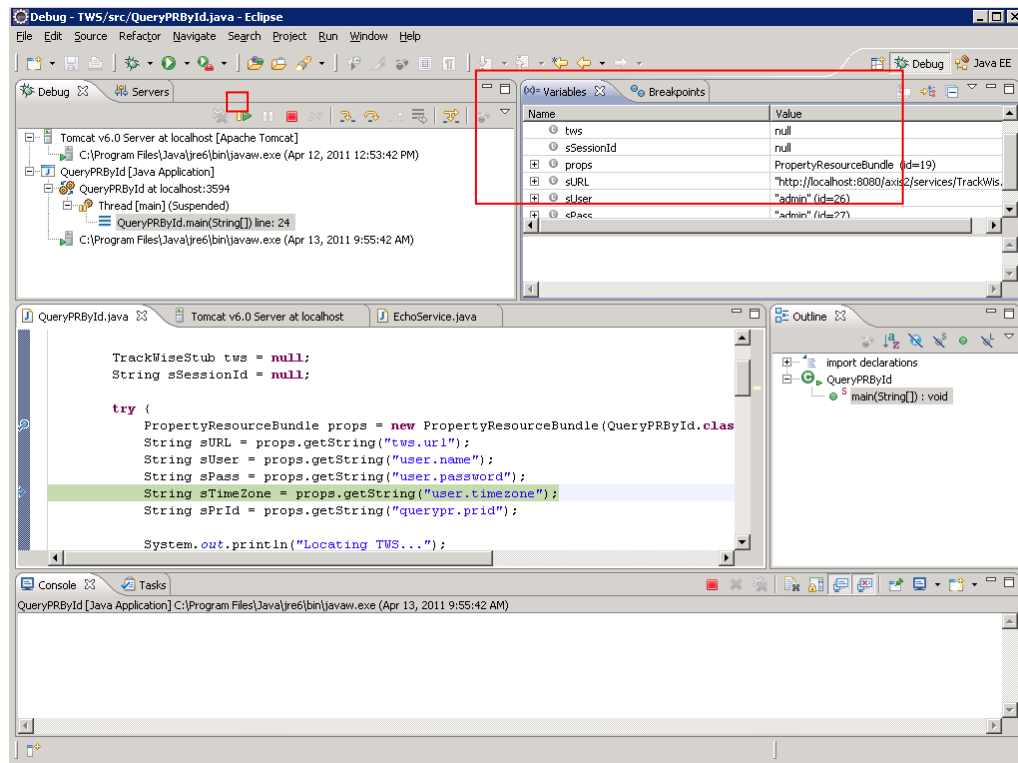
1. To debug in Eclipse you first have to insert a line break where you'd like to begin debugging. To do this, double click the gray area next to the line that you'd like to place the line break. To begin debugging, click the **Bug** icon.



2. If this is the first time that you are debugging then you'll be prompted to view the Debug perspective. Select **Remember my decision** and click **Yes**.



3. From the Debug perspective, you may step through each line of your code. **F6** is used to step to the next line of the source code. **F5** is used to step into the method of the currently highlighted line. If the line of code was assigning a value to a variable then the variable's value will be updated in the **Variables** window. If you would like to run the code to completion or jump to the next line break then use the **Resume** button.



- OWS can also be debugged through Eclipse. To run the server in debug mode, specify the line break in the OWS class and trigger the OWS call from TrackWise. Debugging will then begin in Eclipse.

3. TW WS Best Practices & Conventions

3.1 Introduction

The term “Best Practices” as it applies to coding is very broad and can mean any of several different things: best practice coding conventions, general coding best practices, generally accepted design patterns, etc. Since any one of these topics could be expanded to take up entire textbooks on their own, we focus this chapter on best practices as they pertain to TrackWise Web Services specifically. For the items listed above, we provide some references on where to start in the table below:

Reference
Java Enterprise Best Practices by O’Reilly Media
Quickstart Apache Axis2 by Packt Publishing

In this chapter, you will find Sparta Systems’ suggestions for how build your TrackWise web services to be readable, maintainable, and reusable.

3.2 Code Convention

While there is no one right way to develop code, following certain conventions can help improve the lifelong quality of the software that is developed. Conventions in the manner in which code and its associated documentation will lead to software that is easier to understand, develop, and maintain. Some overall guidelines are presented below; however teams should work towards adopting an agreed-upon methodology that works for their situation.

3.2.1 Naming Conventions

Naming conventions can help properly communicate the intention of the various elements and their relations. Always choose a naming convention that indicates the meaning of the element while avoiding confusion with other similarly named elements. Hungarian notation is not preferred (partially due to strict type definitions within the Java domain).

Packages – meaningful named group by logical domain, preceded by reversed internet domain namespace. For example: `com.sparta.samplepackage`

Classes - named with a full meaningful description, first letters of each word capitalized. For example: `MySampleClassName`

Fields – such as variables, named with full meaningful description, first letter of non-initial word capitalized. For example: `myString`

Constants – using meaningful description, fully capitalized. For example: `CONSTANT`

Accessors/Mutators – getter/setter methods which use `get/set` followed by variable name being set.

For example: `getWeather()`

3.2.2 Syntax Conventions

Whitespace – Generous use of whitespace can help arrange code in a logical order, which will help others in understanding the flow of control in the program. At least one line break should group logical sections such as method definitions, loops, etc. Whitespace between variable assignments, keywords, and operators can also be useful.

Indentation – The use of indentation (1 tab or 4-5 spaces) can help clarify the scope of code. Logical hierarchies of code, such as methods or loops, should be indented to help indicated which elements are in the same scope as others.

Braces – While the placement of braces (`{` and `}`) changes between languages and developers, Oracle's suggestion for Java is to place an opening brace (`{`) on the line opening the code and a closing brace on a line of its own (indented to match the opening brace).

Documentation – It is highly suggested that code be commented in a concise form so that the purpose of each block of code is clear to another developer with no background on the intention of the code. Javadoc style is suggested, depending on the scale and purpose of the code. Remember the saying *“if the program isn't worth documenting, it's not worth running”*.

3.2.3 Examples

```
/**
 * ...method doSomething documentation comment...
 * ...explain what doSomething will performe...
 */
public void doSomething() {
    // ...implementation goes here...
}
```

3.2.4 Additional References

- Oracle's suggested conventions for Java:
<http://java.sun.com/docs/codeconv/CodeConventions.pdf>
- Oracle's Javadoc homepage:
<http://www.oracle.com/technetwork/java/javase/documentation/index-jsp-135444.html>
- Scott Ambler's "Writing Robust Java Code"
<http://www.ambysoft.com/downloads/javaCodingStandards.pdf>

3.3 Logging

During both service development and execution, it is often critical to be able to reflect on the status and flow of control of a program. In order to communicate this information, data is often outputted either via system print statements or writing to a text file.

3.3.1 Java Logger Framework

The Java language has provided a robust specification to facilitate writing to log files, the Java Logger API. One of the benefits of using the Logger is that writing to the logs can be done without regard for the specifics of the actual output of the log. The developer sends logging data to the Logger object and continues with coding, the Logger configuration will dictate the details of the generated log. A popular implementation of this specification is the Apache Log4J tool, which will be used throughout this course.

3.3.2 Logging Fundamentals

- **Logger Object** – The `Logger` object is the logging object that all logging data is written to from within the program.
- **Appender** – The logger, once initialized, will use an `Appender` to dictate the details such as type, location, and size of the log files generated.
- **Formatter** – The `Formatter` will independently format the log data with elements such as timestamps, calling code, and logging level.
- **Levels** – Loggers can log data to one of multiple “levels” of logging which use a hierarchy to indicate which data will be output. The 5 available levels (from highest to lowest respective level) are: `FATAL`, `ERROR`, `WARN`, `INFO`, and `DEBUG`.

3.3.3 Best Practices

It is strongly suggested that an implementation of the Logging API be used rather than a manual logging or system prints. In order to use Logging effectively, consideration should be used in how logger objects are created and the data written to each level.

During day-to-day program execution, the `INFO` level is typically the suggested logging level to be used. Log statements of interest to daily usage (such as program initialization or updates to records) are typically to be written to the `INFO` level.

The `DEBUG` logging level should be used to track the flow of program control, status of variables during execution. With regard to webservice development, the payload and response of major operations should typically be sent to a debug level logger upon receipt and sending of messages.

For more information, see [logging appendix](#).

3.4 Connection / Session Handling

Inbound TrackWise Web Services maintains connections with sessions that are tied to specific TrackWise Login Accounts. In general, the following concepts have been observed to help improve performance of the Web Services.

3.4.1 Dedicated Login Accounts

The use of dedicated Login Accounts for the TrackWise WebService user is preferred rather than using accounts tied to other individuals, and at times even the coordinator. It is suggested that a login account be created for each logical domain of integration (i.e. if there is an ERP integration for Investigations and a separate integration for Complaint systems, an account be dedicated for each). In addition to preventing conflicts on the account (simultaneous login attempts), distinct accounts improve performance and accountability of the connections.

3.4.2 Session Management

While SOAP web services are stateless, the TrackWise session is persisted between calls to ensure proper connection to the service. It is suggested that sessions be created and terminated foreach iteration of work so that idle connections do not persist. The use of `createSession()` should be used to initially create the session and only utilize the `reacquireSession()` operation if there is an unexpected failure on the `createSession()` attempt.

3.4.3 Concurrency

TrackWise Web Services is capable of handling multiple sessions in parallel from the service interface, albeit with some limitations. Throughput of the service tends to plateau at roughly 2 parallel sessions, therefor if more simultaneous sessions are needed, additional servers can be used to balance the load. In addition to maintaining parallel connections, the use of specific login accounts and sessions needs to be synchronized on the client side to ensure that a client does not interfere with another client's operations, see [Record Locking](#).

3.5 TrackWise Configuration

There are certain TrackWise configuration elements that may help or hinder the behavior of TrackWise Web Service integration.

3.5.1 Supported Data Fields

While currently not all DataField types are available via the TrackWise Web Service interface, the product is always being expanded. Refer to the documentation on the TrackWise CD for the version being used to ensure intended data can be interacted with via web services.

3.5.2 Group Categories

It is strongly recommended that a dedicated TrackWise Group Category be created for the TrackWise Web Service client to utilize during connections. The messages sent to and from the web services interface are composed of the Data Fields which the client login account has permissions to view/edit. By limiting the Group Category permissions of this account to only fields that will be needed performance can be improved, especially if network latency is seen.

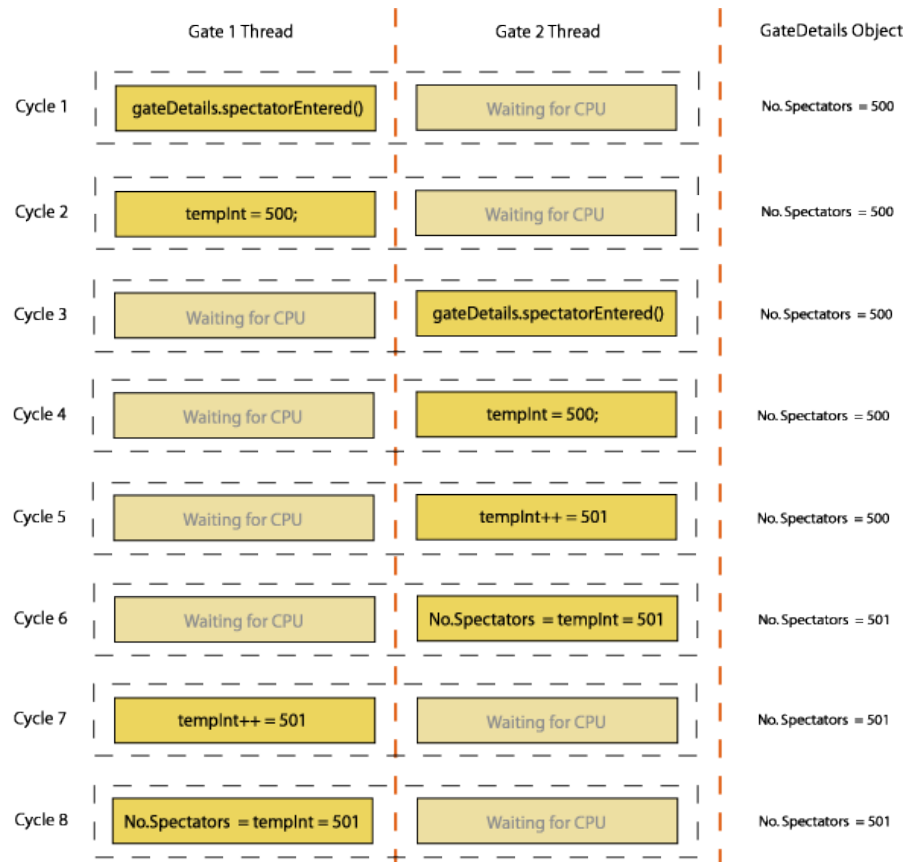
3.5.3 Workflow Design

Efficient workflow design can improve not only the true performance of the TrackWise Web Service product, but the perceived reliability and performance of the product. TrackWise Web Services is permitted to update records in Locked States, allowing for configuration of specific 'holding' states where Web Services actions are pending. For example, a "Pending TWS Update" status may be used for certain milestones within the system where Web Services should take action. Using such a configuration, records needing an update can be readily identified by the service and redundancy can be configured into the system using tools such as TrackWise Coordinator to potentially notify admins if the record has not progressed for some reason. This can be especially useful if an Outbound Web Service call is also triggering an Inbound Web Service operation and there is a problem with the handoff of communication.

3.5.4 Record Locking

Prior to TrackWise 8.3.2, the Web Service interface was not able to obtain exclusive locks on records during operations, which meant workflow design was much more critical as was efficient design approaches. If using an earlier version of the product, it is suggested that records be updated in Locked States to prevent users from modifying data while the service is was also modifying data. In all cases, a record should be queried, processed, and updated as quickly as possible to ensure minimal exposure to race conditions.

Example of concurrency issue ("Race condition"):



3.6 Properties Files, Parameter Intake

Software in validated computer systems should typically not be updated between the various environments involved (e.g. Development, Validation, and Production). To help ensure consistent software versioning it is suggested that elements subject to change, such as the TrackWise login account information, endpoint reference, etc. be provided as parameters to the software rather than hard-coded into the program. Popular choices for handling such configuration are the Java `PropertyResourceBundle` tools or the Apache Commons Configuration package. In this course, built-in Java `PropertyResourceBundle` is used to input parameters via a `.properties` file.

A `.properties` file provides a Key-Value pair method of taking input, the software will read the file and query for the values associated to keys in the file. An example of reading a properties file is shown below, which reads a String “value” for the key named “key”.

```
PropertyResourceBundle props = new  
    PropertyResourceBundle(Template.class.getResourceAsStream(tws-  
        config.properties));
```

```
String value = props.getString("key");
```

See [course properties file example](#).

3.7 Usage (when to use a web service vs. TWC or TW Utilities)

While TrackWise WebServices allow for greatly expanded capabilities of the TrackWise suite, the situations in which web services are the best option can vary. In addition to evaluating whether TrackWise WebServices are capable of a task, it should be verified there is no method of accomplishing the same goal using pre-existing functionality within TrackWise. Validation and maintenance activities for built-in TrackWise components are typically much lower effort than those associated with the use of customized web services. If a goal can be achieved through effective use of standard TrackWise tools, such an approach is often the recommended one. Several common considerations for web services are considered below.

3.7.1 Web Services vs. Coordinator

One area that TWS can be leveraged is in fulfilling automated tasks that the TrackWise Coordinator cannot accomplish. The decision to utilize web services should include not only capabilities of the TWC, but also take a risk-based approach toward using an additional service. TWC is a powerful tool that is built into TrackWise and configured from within the software.

3.7.2 External Fields

Outbound WebServices can be leveraged to retrieve data from other systems to populate fields from within TrackWise in real-time, however this behavior is also available through an External Fields functionality. External Fields can be configured to connect to another database, perform a lookup, and return data directly to TrackWise. While Outbound WebService calls can be designed to handle

circumstances beyond the limitations of External Fields, the use of such should be evaluated before deciding to proceed with OWS lookup calls.

3.7.3 Integration Manager

TrackWise provides a powerful set of tools referred to as Integration Manager, which consists of PR Import, Selection Import, Entity Import, and Person Import. These tools are designed to create and/or update their respective record-types using CSV files that can be read via a background service.

4. Inbound Web Services (IWS)

4.1 Introduction to IWS

As you learned earlier, TrackWise IWS provides an interface that exposes a TrackWise API to third party applications.

A common need for inbound TrackWise Web Services in a real world setting is to allow a third party system or application to query, create or modify records in the TrackWise database. Some examples of this include:

- A call center feeding complaint intake information into TrackWise for processing via a complaints workflow
- Users entering record information into a custom web form from a mobile device, which creates or updates a PR in TrackWise
- An external system is used to manage a laboratory investigation and feeds results back to the main TrackWise record once complete

This chapter will focus on leveraging IWS functionality to integrate a third party call center into a TrackWise Complaints system. The program that we will work on will take output from the call center system (represented by a flat file) and create, update, or perform activities against Complaint records in the TrackWise system. Throughout the chapter, you will be making this web service functional – starting with simple examples and working up to more complicated interactions.

The web service in question here will first locate a record to work on (or identify the request for a new record) and then take action on that record, create a new record, or post an activity. All of these items require different functionality, which will be explained through examples.

4.2 Connection Logic

The foundation of any web-based program is the method of communication. In order to communicate between applications, there first needs to be a connection in place and security around that connection to prevent unauthorized use.

TrackWise IWS controls access to the TrackWise database through permissions using the same mechanism as the main TrackWise application does – via TrackWise login accounts. In order to connect to TrackWise, edit fields, perform activities, or take any action, the IWS client must log into the TrackWise database with a user account that has the appropriate permissions. In this section we will take a look at the connection logic that will need to be in place to establish a connection to the TrackWise database and maintain a session throughout the lifecycle and workflow of the program.

4.2.1 Setting up the TrackWise login account

In order to follow the configuration rules (Workflow, Field Permissions, Audit Trail accountability, etc...) that have been designated in the system, IWS interacts with the TrackWise WebService API via a Login Account. The login account associated with the IWS operations needs to meet certain criteria, including having Allow Access to “TrackWise – Main”, Local Authentication Source, and Project Membership (Group Category) designation appropriate for the expected tasks being performed. As a user may not be logged into TrackWise and the IWS interface concurrently, it is strongly recommended that a dedicated Login Account is created for use with IWS.

Note: In this course, the login account has been set up for you – when setting up a web services login account on your own, follow the instructions below

1. Within the WebAdmin Application, navigate to **User Management -Login Accounts**.
2. Create a new **Login Account**, named **tw**s for a new person named **TrackWise WebServices**.
3. Set **Authentication Source** to **Local** and provide a password to be used (e.g. “tw”).
4. Enable TrackWise access on the **Application** tab.
5. Provide **Full Permission Group Category** membership to both the **Product Issue** and **Complaint** projects.

4.2.2 Developing the Connection Code

The TrackWise IWS Client API provides a structure for connecting to the TrackWise service, called the `TrackWiseStub`, which provides an abstraction for the operations available to the services.

In this exercise we will login to TrackWise via the IWS and create a Session.



1. Create a `TrackWiseStub` object with the TrackWise EndPoint Reference (as seen on the Axis2 description page) as the constructor:

```
TrackWiseStub tws = new TrackWiseStub(twsUrl);
```

The following options are settings which are used to control the specific behavior of the connection:

```
// Set the duration the client should wait for a response before a timeout.  
tws._getServiceClient().getOptions().setTimeoutInMilliseconds(twsTimeout);
```

```
// Create the multithreaded connection manager and set the maximum number of
connections
MultiThreadedHttpConnectionManager conmgr = new
    MultiThreadedHttpConnectionManager();
conmgr.getParams().setDefaultMaxConnectionsPerHost(TWS_TIMEOUT);
HttpClient client = new HttpClient(conmgr);
```

```
// Set other options used to facilitate faster connection creation and reclamation
twc._getServiceClient().getServiceContext().getConfigurationContext().setProperty(
    HTTPConstants.CACHED_HTTP_CLIENT, client);
twc._getServiceClient().getServiceContext().getConfigurationContext().setProperty(
    HTTPConstants.REUSE_HTTP_CLIENT, "true");
```

4.2.3 Managing the Session

Having informed the client the location of interface, it is now possible to connect to the interface by creating a new session. The session identifier is used by the interface to maintain the connection as a programmatic analog to a user logging into TrackWise. A successful `createSession()` call will return the session identifier, which will be required for all subsequent operations made by the client. This call is implemented as follows:

```
String sessionId = twc.createSession(username, password, timezone);
```

Once work has completed, the session may be terminated through the use of the `twc.endSession(sessionId)` API call, which will attempt to end the session provided.

Note: An additional operation `reacquireSession(twcUser, twcPass, sessionId, twcTime)`; is available to recapture a session that was not properly terminated and an “already logged in” error occurs.

4.2.4 Example



On Your Own Exercises

Now that we have reviewed the `TrackWiseStub` and session management, it is time to put this information to use. We will use the code snippets above to write a Java application that will connect to the service interface. Please Import to Eclipse CodeBase Project from C:\Course Resources.

The codebase Project provides a template class that we can base our program on using **Copy -> Paste** and assigning the program a new name (Example_4_2.java).

We will also utilize the **twc-course.properties** file to read the parameters needed to input into the service. Refer to the solved codebase examples for additional guidance if needed.

General Program Structure

1. Initialize Program, including the `Logger` and `PropertyResourceBundle`.
2. Read needed parameters from the properties file: `tw.s.url`, `tw.s.user`, `tw.s.pass`, `tw.s.timezone`.
3. Create a `TrackWiseStub` object.
4. Set the connection parameters as described above.
5. Create a TWS session.
6. Print the TWS session ID
7. End the TWS Session.

Note that the End session code typically should be contained in the finally portion of a try-catch block – we will want to disconnect even in the event of an error during business-logic code.

4.3 IWS Data Types

The Inbound API has objects which provide a representation for many of the concepts relevant to daily use of the TrackWise system, such as PR (records), Persons, Entities, and Login Accounts. All of these Data Types are simply a set of properties, (Data) Fields, which are used to describe the data relevant to the specific Field Type they represent. These Data Types can be described as `ComplexTypes` and `SimpleTypes`. In this context, the `ComplexType` can be perceived as an aggregation of other `SimpleTypes` as well as other `ComplexTypes`. `SimpleTypes` more generally are representative of a Name/Value property – the various `SimpleTypes` mostly differ in the type of data they contain and the signature of the corresponding `getValue()` method. A comprehensive review of the Data Types and Fields are included in the appendix and documentation, with a summary of commonly used types in the following sections.

4.3.1 Fields

All specific fields are extensions of a common `Field` object and therefore share some traits and behaviors that are not specific to the type of field that is being used, as in the case of `Field` field:

- `field.getName()` – Returns the name of the current field (e.g. “Short Desc.”);
- `field.getMetaData()` – Provides access to many of the attributes of the field, including the `FieldClass` (type of `DataField`), whether the field is required, editable, copytochild, etc.

Each field’s specific type will drive the behavior of certain metadata attributes as well as the `getValue()` operation. Once the field’s type is known, it can be cast to the appropriate type for further interaction.

4.3.2 String Fields

The `StringField` object is used to interact with String data-fields in the system and is initialized as follows:

```
StringField stringField = (StringField) field;
```

`StringField` objects leverage the following methods to retrieve or set the field’s value:

- `stringField.getValue()` – Returns a `String` with the field’s value.

- **`stringField.setValue(String value)`** – Sets the value of the field.

4.3.3 Number Fields

The `NumberField` object is used to interact with Number datafields in the system and is initialized as follows:

```
NumberField numberField = (NumberField) field;
```

`NumberField` objects leverage the following methods to retrieve or set the field's value:

- **`numberField.getValue()`** – Returns an `Integer` with the field's value.
- **`numberField.setValue(Integer value)`** – Sets the value of the field.

4.3.4 Selection Fields

The `SelectionField` contains a `Selection` object which is used to interact with the field and is initialized as follows:

```
SelectionField selectionField = (SelectionField) field;
```

`SelectionField` objects leverage the following methods to retrieve or set a field's value:

- **`selectionField.getSelection()`** – Returns a `Selection` Object.
- **`selectionField.setSelection(Selection value)`** – Sets the value of the field.

The `Selection` object is an object with 3 available properties: `Id`, `Local Id`, and `Value`. The `Value` property is most commonly used when interacting with iWS and be used to read and write to a selection. There is a `querySelection()` operation available that allows for the identification and verification of available selection values in the system which can be used to ensure a value exists prior to writing.

```
Selection selection = selectionField.getSelection();
```

- **`Selection.getId()`** – Returns the ID of the selection value.
- **`Selection.getLocalId()`** – Returns the "Local ID" (External Id for external fields) of the value;
- **`Selection.getValue()`** – Returns a `String` of the value;

4.3.5 Date Fields

The `DateField` contains a `String` value (in the format YYYY-MM-DD) that represents the Date field in the system and is initialized as follows:

```
DateField dateField = (DateField) field;
```

`DateField` objects leverage the following methods to retrieve or set a field's value:

- **`dateField.getValue()`** – Returns the YYYY-MM-DD `String` representation of the Date value;
- **`dateField.setValue(String value)`** – Sets the value of the field from a `String` in the form YYYY-MM-DD.

[Code example appendix for converting date Format.](#)

4.3.6 DateTime Fields

The `DateTime` field contains a `Calendar` object representing the field's value and is initialized as follows:

```
DateTimeField dateTimeField = (DateTimeField) field;
```

`DateTimeField` objects leverage the following methods to retrieve or set a field's value:

- `dateTimeField.getValue()` – Returns a `Calendar` object representing the fields value.
- `dateTimeField.setValue(Calendar value)` – Takes a `Calendar` object with the fields value.

4.3.7 Person Fields

The `PersonField` is a complex type that uses a `Person` object to represent the field's value and is initialized as follows:

```
PersonField personField = (PersonField) field;
```

`PersonField` objects leverage the following methods to retrieve or set a field's value:

- `personField.getPerson()` – Returns a `Person` object with the fields value.
- `personField.setPerson(Person value)` – Sets the field with the value of the `Person` object;

The `Person` object is composed of typical `DataFields` representing the person information (i.e. contains a `StringField` "First Name" with the person's name.

4.3.8 Complex Types

The previous case of the `Person` object is an example of a Complex Type – that is, a Data Type that composed of other Data Types (Fields). `PR`, `Person`, `Entity`, `Address`, `Activity`, and `Login` are available Complex Types in the TrackWise IWS API (The full definitions for the various types can be found in the documentation). For the complex types, the set of underlying Fields can be accessed using the `.getFields()` method which returns an array of Fields. Alternatively, the supplied client `Util` class contains a helper method which puts the fields into a `Map` object, identified by each field's name.

```
Field[] fields = PR.getFields();
```

or

```
Map<String, Field> fieldMap = Util.toFieldMap(PR);
```

4.3.9 Example



In this example, a PR will be fetched from the TrackWise API and the various Data Fields will be explored. (The `Util` method `getPrById()` will be used to query the record).

1. To begin, create or identify an existing Complaint PR in the course TrackWise system.
2. Ensure the following fields have been populated (fill in if needed):
 - a. Date Opened
 - b. Short Description
 - c. Assigned To
 - d. Description
 - e. Awareness Date
3. Starting with the Create Session example, we will get a `PR` object and output the Data Field information using the `FieldMap` technique for access the underlying fields and printing their values to the log.
4. The structure of the program should follow:
 - a. Locate the TrackWise Endpoint (connection)
 - b. Create the TrackWise session.
 - c. Obtain `PR` object using `Util.getPrById()`;
 - d. Obtain `FieldMap` using `Util.toFieldMap()`;
 - e. Get each field above from the `FieldMap`.
 - f. Print the Name and Value of each field.
 - g. End the TrackWise session.

4.4 Querying Records

Once a proper connection is established, an IWS client must be able to find the records that are to be acted upon – whether that action be editing, pulling back information, or performing an activity. IWS provides the same search functionality through its API that is available to users in the TrackWise system.

This section will go through the process of constructing and executing queries against the TrackWise database, and expand the program that you have been working on include this functionality.

4.4.1 Query Fields

In order to query for records, the API includes an object referred to as `QueryFields`. Each `QueryField` instance is used to define a specific `DataField` to be searched for, a value criteria, and an operator that indicates the relationship between the criteria and the returned value (equal to, less than, etc...).

A set of one or more `QueryField` objects is assembled into an array and passed to the appropriate web service query operation. As an example, the set of `QueryField` objects used in the `Util.getPrById()` operation is:

```
QueryField idQueryField = new QueryField();
```

```
idQueryField.setOperator(QueryOperator.EQUAL);  
NumberField prId = new NumberField();  
prId.setName(NameDefault.PR.ID); //DataField Name  
prId.setValue(id);  
idQueryField.setField(prId);  
  
PR[] prs = tw.queryPR(sSessionId, new QueryField[]{idQueryField});
```

The `queryPR()` operation will return an array of PRs – in the above case only the `prs[0]` object will exist.

Note the use of the `QueryOperator`, which is a constant available in the API to indicate the search criteria to be used by the service.

4.4.2 Example



For this exercise, we will query for a PR by the Data Field “Short Description”.

1. Create a new record in TrackWise and assign the **Short Description** field a known value (“**Test Complaint Record**”);
2. Set the **tw-course.properties** field key `querypr.shortdesc=` with the value from previous step.
3. Create a new program based on our create session example.
4. The program structure will:
 - a. Connect to the TrackWise Web Service
 - b. Create a `Field` for Short Description, with the value from the properties file.
 - c. Create a `QueryField` object.
 - d. Add the Short Description field to the `QueryField` object.
 - e. Set the `QueryField` object’s operator.
 - f. Run the `QueryPR` operation with an array containing the created `QueryField`.
 - g. Print the result from the `QueryPR` operation (a PR Array).

4.5 Updating Records

Once a record is found, the IWS client can update the fields on that record with new information. In the example we are using, the use case is that new information came in about a complaint and must be updated accordingly. It is the job of IWS to push the new complaint information into the TrackWise system and update the appropriate record.

There are a variety of field types available in the TrackWise system that are all handled slightly differently in web services. This must be accounted for when writing the IWS client and handling the different data types that comprise the TrackWise record. The previous section on working with various Data Types provided a summary of interacting with these fields. Detailed definitions of this information are available in the course appendix.

Throughout this section, you will learn how to update information into fields on a TrackWise record with web services and expand your program to incorporate this logic. While the exercises work with PR Data Types, the manner of updating other types such as Persons, Entities, and Login Accounts is similar.

4.5.1 Record Locking

In order to prevent the underlying record data from changing while the client code is processing a record (see ["Race Condition example"](#)), a locking mechanism has been included. A PR may be locked using the `editPR()` operation, with the lock being cleared by the `updatePRRequireLock()` or the `cancelEditPR()` operation.

Alternatively, the legacy `updatePR()` operation is available, however while this operation will respect other user's locks there is no lock acquired on the `queryPR()` operation. Changes to the record after the `queryPR()` operation is called may be reverted by a subsequent `updatePR()` call.

4.5.2 Updating Records

Once a record has been obtained and its fields processed, updates to the record may be desired. The API provides operations to update the record and set data field values. It should be noted that the API will respect the Login Account's Group Category Permissions, Lock Field Classes, and Required Field Classes during an update call – if any of these rules are violated a `ValidationException` may be thrown.

First a lock on the intended PR should be obtained using the `editPR()` method:

```
PR pr = editPR(String sessionId, Integer prId);
```

If the lock is obtained successfully, the PR object will be returned from the service. Then the object's fields may be accessed and updated values set appropriately. Once the record update is ready to be posted, the `updatePRRequireLock(String sessionId, PR pr)` operation can be used.

4.5.3 Example



On Your Own Exercises

In this example, we will build off the previous example in which we had been querying a PR in order to update the record that has been returned.

1. Starting with the code from the previous example, we will set:
 - a. Date of Birth
 - b. Height
 - c. Height Units
 - d. Weight
 - e. Weight Units
 - f. Customers Words
2. The structure of the program should follow:
 - a. Locate the TrackWise Endpoint (connection)
 - b. Create the TrackWise session.
 - c. Obtain `PR` object using `editPR()` ;
 - d. Obtain `FieldMap` using `Util.toFieldMap()` ;
 - e. Get each field above from the `FieldMap`.
 - f. Assign a new value to each of the fields above.
 - g. Update the `PR` using `updatePRRequireLock()` ;
 - h. End the TrackWise session.

4.6 New Record Creation

The TrackWise Inbound WebService API is also capable of creating new records, for example in the case of a Complaint integration service in which data does not already exist in TrackWise. The approach to creating new records is slightly different than updating records, as the details of which fields are available to work with depend on the type of record being created.

4.6.1 Obtaining an Empty PR

In the previous examples, we worked with `PR` objects that already existed and were retrieved via the `queryPR()` operation. For New records, we will need to obtain an “empty” object, which is an object that does not yet exist in the TrackWise system, from the web service. The API defines the following method:

```
PR pr = tws.getEmptyPR(String sessionId, Selection divison, Selection project);
```

This `getEmptyPR` will verify the service accounts project membership and field permissions and return a `PR` object, but without the fields populated – similar to the New PR screen used in TrackWise.

Once the empty `PR` object is obtained, most data fields can be accessed similarly to previous exercises – via `toFieldMap` utility functions, however `GridFields` are unique and function differently.

4.6.2 Empty GridRows

Similar to PRs that don't yet exist, a `GridField` that does not yet have data populated does not have certain attributes such as the column definitions defined. In order to populate a currently blank `GridField`, `GridRow` objects must be obtained from the service API via the `getEmptyGridRow` operation.

```
GridRow gridRow = tws.getEmptyGridRow(String sessionId, Selection division,  
    Selection project, String gridFieldName);
```

The `gridRow` that is returned will contain an array of `DataFields` – the Fields that are contained in the grid and are ready to populate. At this time, the newly populated `gridRow` objects may be added to the `GridField` as part of the PR object.

4.6.3 Create PR

Having obtained an empty PR object and populated the data fields, you may now create the record via the `tws.createPR()` operation.

```
Int prId = tws.createPR(String sessionId, PR pr);
```

The `createPR` operation, if successful, will return the PR Id of the newly created record. In addition to creating standalone records, by setting the Parent ID on the record, children records may be created using the same `createPR` operation.

4



Using the operations discussed above, we will create a new TrackWise PR record with the following program structure. Reference the `createPr` set of keys in the `twc-course.properties` file for the parameters.

1. Connect to TrackWise Service.
2. Obtain an Empty PR via the `getEmptyPR()` operation. (Holmdel/Complaint).
3. Set the following **Data Fields**:
 - a. **Parent ID** (If desire to create a child).
 - b. **Assigned To**
 - c. **Initial Reporter**
 - d. **Initial Reporter Company**
 - e. **Event Date**
 - f. **Awareness Date**
 - g. **Lot Number**
 - h. **Materials Affected** (Use `CourseUtil.fetchMaterialAffected(lotnumber)`).
4. Create PR.
5. Print PR ID created

6. Disconnect from the Web Service.
7. Review the newly created record in the TrackWise system.

4.7 Post an Activity

In some cases, an activity may need to be performed in order to progress a record forward or backward in the workflow as a result of an external system. In this section, you will learn through the course of examples how to post and schedule an activity in TrackWise.

4.7.1 Activities

The TrackWise Inbound Web Service API has the ability to work with activities, including querying, performing, and scheduling of activities. The client API has an `Activity` object which is a `ComplexType` object (composed of other types) as well as a `twsw.postActivity()` operation to post the activity.

The full details of the `Activity` object can be seen in the documentation/wSDL, however some of the fields available (and potentially required) include:

- PR ID (`NumberField`)
- Activity Type (`SelectionField`)
- DatePerformed (`DateTimeField`)
- Person Responsible (`PersonField`)
- Activity Summary (`StringField`)

4.7.2 Performing vs. Scheduling an Activity

One of the required fields to include in an `Activity` object is Date Performed and/or Date Scheduled – at least one of these values must be present. Consideration also must be given for the additional validation rules – Date Performed must be after the record has entered into the state, but not in the future. During automated activities across various servers, small differences in system clocks can result in validation issues. Consideration for handling these circumstances must be planned when developing production-hardened code.

4.7.3 Example



In this exercise, we will create an `Activity` object and perform an activity against a record, using the general program structure below.

1. Create or identify a Complaint PR in the state of Opened.

2. Connect to TrackWise WebService.
3. Obtain an **Activity** object via the `twS.getEmptyActivity()` call.
4. Populate the following attribute fields, using the **twS-course.properties** file for the values:
 - a. **PR ID**
 - b. **Activity Type** (Name)
 - c. **Date Performed** (Create a Calendar instance for “Now”).
 - d. **Person Responsible** (The Web Service Person, obtain via `queryPerson()`).
 - e. **Activity Summary**
5. Add the attributes as a `Field[]` to the `Activity` object.
6. Post the activity via the `twS.postActivity()` operation.
7. Disconnect from the WebService.
8. Verify in TrackWise the activity has been posted to the record.

Refer to the solved codebase examples for additional guidance.

4.8 IWS Limitations

- Web Service must use Valid Login to connect IWS, in case of Web Service that runs parallel, each Service Instance will need different Login.
- Record that is currently open by a User can't be updated via IWS.
- Which fields can be retrieved or changed depends on the permission of the login used.
- Certain system fields can't be changed by IWS

4.9 Security And User Permissions In IWS

- Web Service must use a Valid Login (User, Password).
- The Web Service Actions on TW are under the restrictions of the Login account permissions it using.
- Using Apache Rampart - Axis2 Security Module, Supports :
 - Encrypted headers.
 - Encrypted SHA1 references.
 - Policy
- Running Web Services over HTTPS

5. Outbound Web Services (OWS)

5.1 Introduction to OWS

TrackWise OWS provides the ability for the TrackWise application to reach out to a third party program for some action rather than having a third party program initiate the action, as in IWS.

Some common use cases for OWS include:

- Looking up master data from another system
- Flagging a product or lot in another system based on some event in TrackWise

A common scenario encountered in a TrackWise complaints implementation is that multiple pieces of product information must be captured to support the complaint investigation. Although each data element must be captured as a separate field in TrackWise (lot number, name, expiration date, etc.), there is a relationship between these pieces of data which we need to maintain in the TrackWise record for data integrity – only certain lot numbers can be associated with certain products. Normally, this product information is stored in a 3rd party system external to TrackWise, which gives us several options for how to proceed using out of the box functionality that both fall short of an ‘ideal system’.

There are two facets to the problem – obtaining the information for each field, and maintaining the relationship between fields to ensure data integrity. To obtain the information, TrackWise offers two options – importing the data into each field via selection import, or leveraging external fields to reach into the other system to reference the values. Each of these options has downsides, specifically:

- Batched importing duplicates product master data into TrackWise and is not real time
- External fields require that the external database follow a very specific schema in order to function properly

In order to maintain information integrity, we can set up a dependent field relation in TrackWise that will maintain the relationships between the fields in question. However, this is limiting in that the user must start entering information with the field at the top of the dependency. For example – if we put the lot number at the top of the relation and the user does not know the lot number, they cannot enter any product information.

OWS presents a novel solution and ideal approach to this problem by providing search and populate operations. These provide the ability for a user to search an external database in real time and bring back matching values for product information in both the field searched and other fields on the form, while not being bound to the traditional dependent relation in TrackWise. Additionally, searches can be ‘daisy-chained’ together to allow filtering of the result set based on known field relationships. We will examine how to set up and code for these relationships and gain an understanding how powerful this OWS functionality can be.

This chapter will focus on setting both the search and populate OWS operations to pull back master data from another system into the TrackWise form on demand. Chapter 6 will cover the remaining OWS operation – transfer – which allows data from TrackWise to be passed one-way to another program for action to be taken.

Note that for the OWS example this course provides a pre-written WSDL file (see the Appendix), however understanding WSDL structure is a fundamental component of working with WebServices.

5.2 Building the Interface Contract

Unlike IWS, which provides us a WSDL and defined operations, when coding an OWS we must first design and code a WSDL to describe the web services and operations that we will be creating. This WSDL will function as the interface contract between TrackWise and the OWS interface. For this section and the remainder of the chapter, whenever adding or changing OWS functions we will reevaluate the WSDL to ensure that we are appropriately describing the functionality of the service.

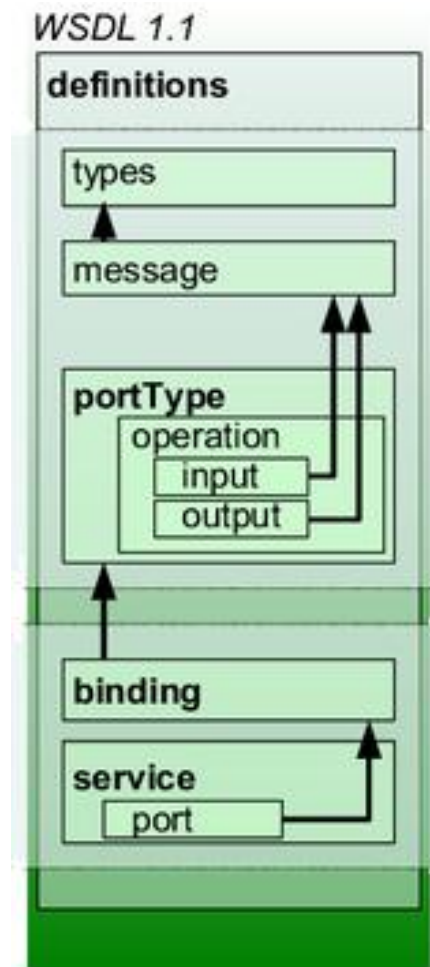
After building the WSDL file it is suggested that the file be configured in WebAdmin to verify that the file can be parsed by TrackWise before moving into code development.

5.2.1 WSDL Structure

The WSDL file is an XML file that describes the web service to the service client (TrackWise, in this case) which operations are available, what data the service expects to receive, and what data the service will return. The WSDL file follows a standard formation, typically built from the bottom-up.

As introduced in Chapter 1, the WSDL file describes the web service including the operations available and the structure of the messages sent and received by the operations. We will look at the main components that compose a WSDL file and review an example web service.

Component	Description
Types	The types section defines the data types that will make up the messages sent back and forth. These types follow typical XML Schema conventions.
Message	The messages define the data type elements used to compose the input and output of the operations.
Port Type	Describes the operations available, which messages the operation takes as input and which messages the operation gives as output.
Binding	Indicates the protocol and format information for the SOAP types that are available.
Service	Defines the services that are available in this instance and the port types / bindings that should be used.



5.2.2 Example

An example WSDL File

```

<?xml version="1.0" encoding="UTF-8"?>
<wsdl:definitions name="Example" targetNamespace="http://example.sparta.com"
  xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/"
  xmlns:tns="http://example.sparta.com"
  xmlns:xs="http://www.w3.org/2001/XMLSchema"
  xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/">
  <wsdl:documentation>A TrackWise Outbound Web Service (OWS) to Populate Example as
    a Field Value.</wsdl:documentation>
  <wsdl:types>
    <xs:schema attributeFormDefault="unqualified" elementFormDefault="unqualified"
      targetNamespace="http://example.sparta.com">
      <xs:element name="ExampleRequest">
        <xs:complexType>
          <xs:sequence>
            <xs:element minOccurs="0" name="str" nillable="true"
              type="xs:string"/>
          </xs:sequence>
        </xs:complexType>
      </xs:element>
      <xs:element name="ExampleResponse">

```

```
<xs:complexType>
  <xs:sequence>
    <xs:element minOccurs="0" name="retStr"
      nillable="false" type="xs:string"/>
  </xs:sequence>
</xs:complexType>
</xs:element>
</xs:schema>
</wsdl:types>
<wsdl:message name="ExampleRequest">
  <wsdl:part name="parameters" element="tns:ExampleRequest">
  </wsdl:part>
</wsdl:message>
<wsdl:message name="ExampleResponse">
  <wsdl:part name="parameters" element="tns:ExampleResponse">
  </wsdl:part>
</wsdl:message>
<wsdl:portType name="ExamplePortType">
  <wsdl:operation name="Example">
    <wsdl:input message="tns:ExampleRequest">
    </wsdl:input>
    <wsdl:output message="tns:ExampleResponse">
    </wsdl:output>
  </wsdl:operation>
</wsdl:portType>
<wsdl:binding name="ExampleBinding" type="tns:ExamplePortType">
  <soap:binding style="document"
    transport="http://schemas.xmlsoap.org/soap/http"/>
  <wsdl:operation name="Example">
    <soap:operation soapAction="http://example.sparta.com/Example"/>
    <wsdl:input>
      <soap:body use="literal"/>
    </wsdl:input>
    <wsdl:output>
      <soap:body use="literal"/>
    </wsdl:output>
  </wsdl:operation>
</wsdl:binding>
<wsdl:service name="ExampleService">
  <wsdl:port name="ExamplePort" binding="tns:ExampleBinding">
    <soap:address location="http://example.sparta.com/">
    </soap:address>
  </wsdl:port>
</wsdl:service>
</wsdl:definitions>
```

From the above WSDL file, we can identify the elements (working from bottom-up):

- **Service** – The service, `ExampleService` and its related Binding.
- **Binding** – The `ExampleBinding`, which indicates to use Document style SOAP protocol for the various operations.
- **PortTypes** – Defines the `ExamplePortType` with an operation named `Example`, specifying message types `ExampleRequest` and `ExampleResponse`.
- **Messages** – The `ExampleRequest` and `ExampleResponse` messages are described as each containing a similarly named element.
- **Types** – The elements `ExampleRequest` and `ExampleResponse` messages are each composed of a single string element.

5.3 Axis2 and AXIOM

5.3.1 WSDL2Java

Once a WSDL file has been generated and verified, it can be converted into a code-base which will be used at the starting point for the web service interface. The WSDL2Java framework provided by Axis2 can be used to generate server-side code (referred to as a skeleton) that will then require the business-specific logic to be filled in. During this section's exercise, a WSDL file will be created for the server-side code to be populated.

5.3.2 Axis2

During this course, we will use the Apache Axis2 framework as the engine for handling the web service communication. The Axis2 libraries are responsible for ensuring the requests are sent and received correctly, following SOAP protocols. The use of this framework allows for abstraction from the underlying details of XML messaging, while also providing access to the underlying details to control the nuances required for careful web service development.

5.3.3 AXIOM

Axis2 provides a number of frameworks to represent XML structures using Java code. Our examples will utilize the Axis Object Model (AXIOM), an implementation of the W3C Document Object Model (DOM). The AXIOM framework allows for programmatic manipulation of XML in a reliable fashion. We will use a subset of the AXIOM toolset, most notably the `OMElement` object, which is the AXIOM representation of XML elements.

5.3.4 OMElement

The `OMElement` object is the core item within the AXIOM framework that we will work with during these exercises. After a service skeleton is created via WSDL2Java, the service will both send and receive messages using this `OMElement` object.

The `OMFactory` model handles the construction of `OMElement` objects, taking as parameters the element's local name and intended namespace (in the form of an `OMNamespace` object which requires 2 parameters – the fully qualified namespace and an alias). With regard to TrackWise OWS, root-level messages (i.e. `FetchLotNumberResponse`) will be qualified with the sender's namespace, all other elements will use a null namespace.

```
OMFactory factory = OMAbstractFactory.getOMFactory();
OMNamespace wsNS = wsRequest.getNamespace();
OMNamespace nullNS = factory.createOMNamespace("", "");
OMElement wsResponse = factory.createOMElement("responseMessage", wsNS);
```

As additional sub-elements are created, they should be created using the `nullNS` and subsequently added as children to the response `OMElement`.

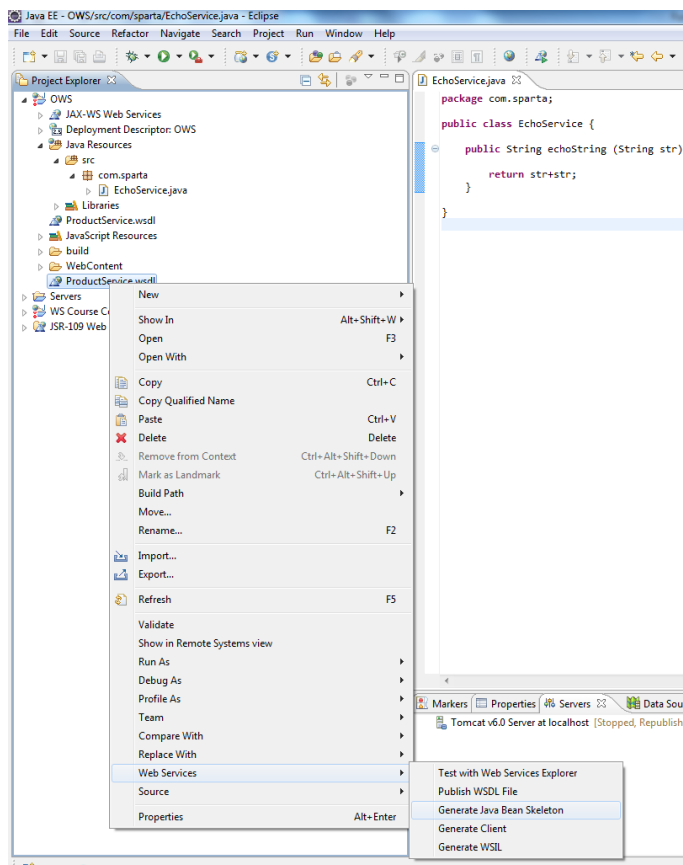
```
OMElement productNumberOM = factory.createOMElement("productNumber", nullNS);
productNumberOM.setText("1234");
wsResponse.addChild(productNumberOM);
```

5.3.5 Example

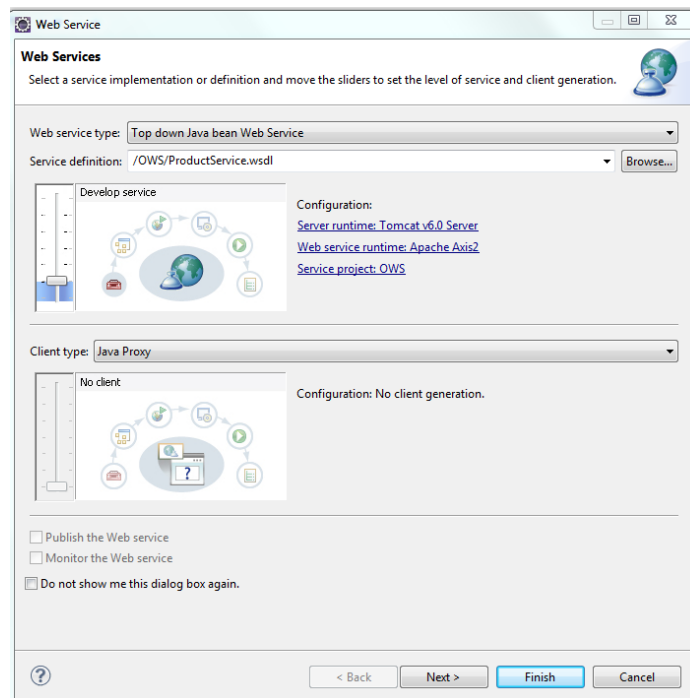


In this section, we will take a supplied WSDL file and use Eclipse WTP tool to create and deploy the skeleton for the web service course.

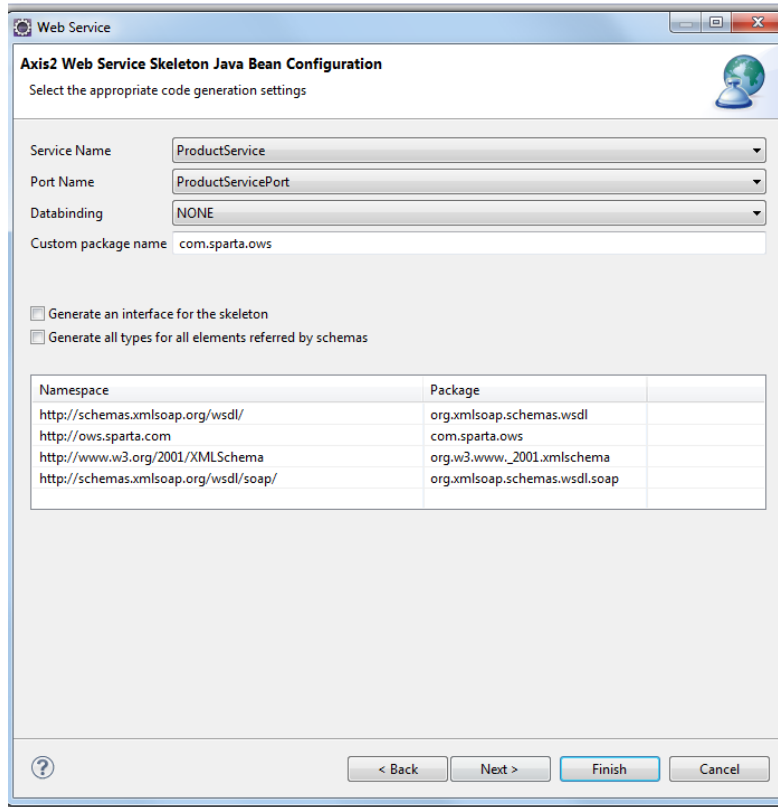
1. Locate the Util Folder in the course resources directory and copy it to the OWS Project under source.
2. Add to OWS Project Build Path All jars under C:\Trackwise Web Services\lib.
3. Add to OWS Project Build Path sqljdbc4.jar and tws.jar under C:\Course Resources\lib
4. Locate the **ProductService.wsdl** file in the course resources directory and copy it to the OWS Project created in Chapter 2.
5. Right Click on the **ProductService.wsdl** , select **Web Services - Generate Java Bean Skeleton Web Service**.



6. Ensure **the WSDL file** is selected. Click **Next**.



7. Set the options as Described below, ensuring **None** is selected the **Databinding** field.



Web Service

Axis2 Web Service Skeleton Java Bean Configuration

Select the appropriate code generation settings

Service Name: ProductService

Port Name: ProductServicePort

Databinding: NONE

Custom package name: com.sparta.ows

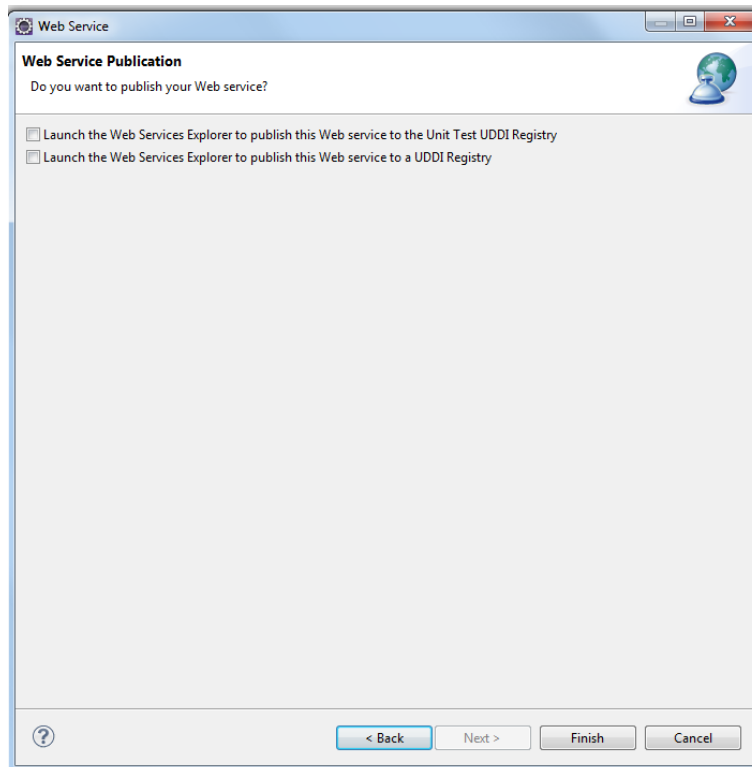
☐ Generate an interface for the skeleton

☐ Generate all types for all elements referred by schemas

Namespace	Package
http://schemas.xmlsoap.org/wsdl/	org.xmlsoap.schemas.wsdl
http://ows.sparta.com	com.sparta.ows
http://www.w3.org/2001/XMLSchema	org.w3.www_2001.xmlschema
http://schemas.xmlsoap.org/wsdl/soap/	org.xmlsoap.schemas.wsdl.soap

< Back Next > Finish Cancel

8. Click **Next**, then **Finish**.



Web Service

Web Service Publication

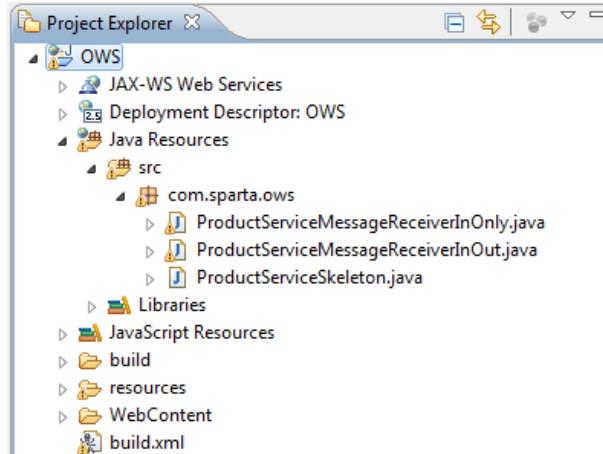
Do you want to publish your Web service?

☐ Launch the Web Services Explorer to publish this Web service to the Unit Test UDDI Registry

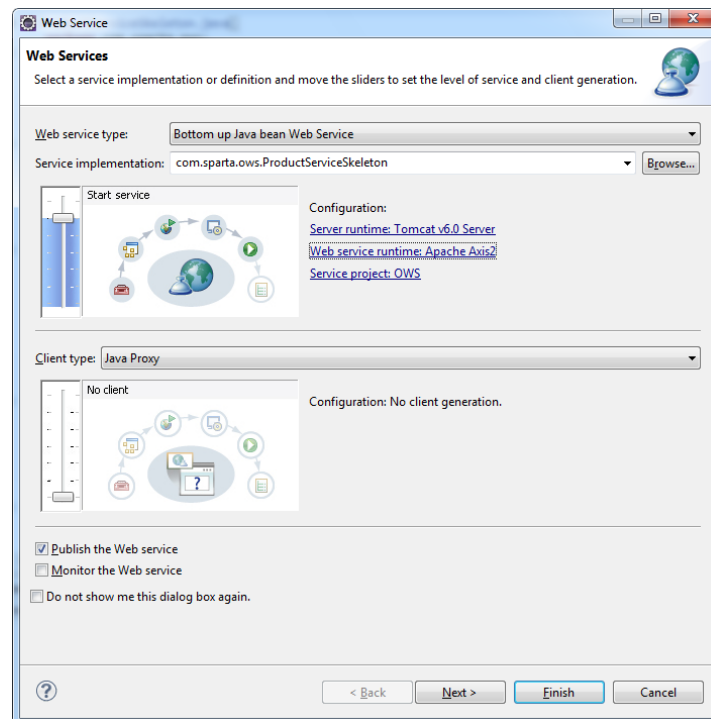
☐ Launch the Web Services Explorer to publish this Web service to a UDDI Registry

< Back Next > Finish Cancel

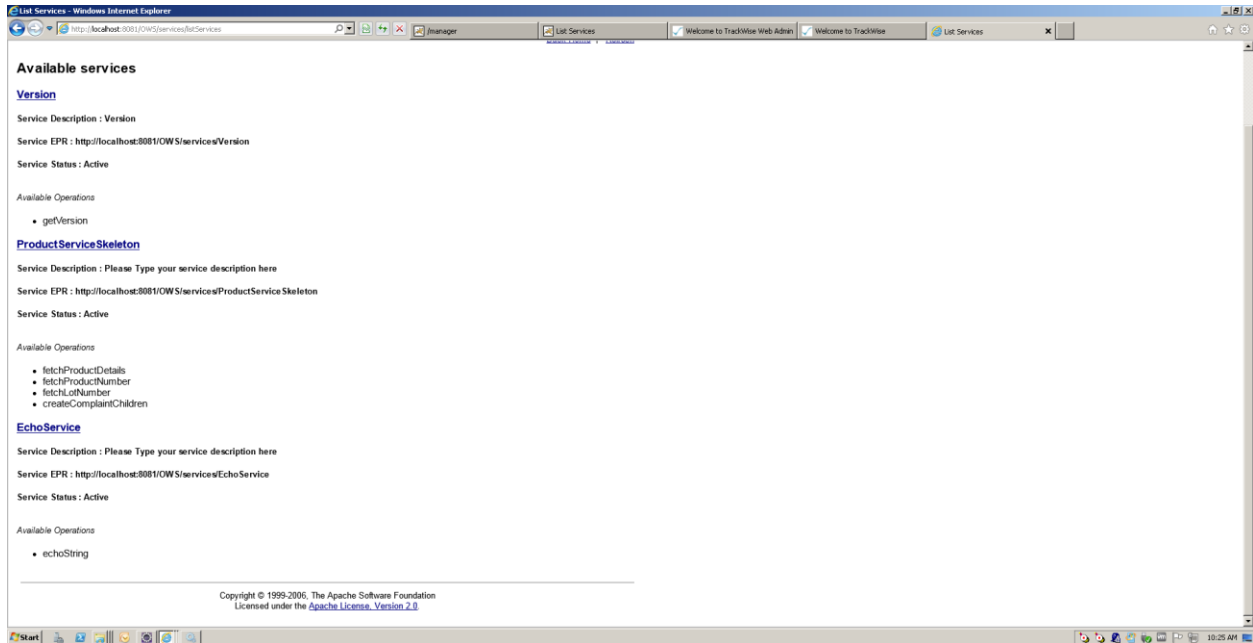
9. Select the **OWS Project Heading**. Right-click the **src** folder and select **Refresh**.
10. Confirm the **Java resources src** folder now has the ProductServiceSkeleton and message receivers created.



11. Right-click the `ProductServiceSkeleton` and select **Web Services- Create Web Service**. Ensuring the **runtime** is set to **Apache Axis2**, click **Finish**.



12. To Validate the Web Service is generated, Deployed & Running , Browse to <http://localhost:8081/OWS> and confirm the ProductServiceSkeleton is available.



13. To complete this exercise, let's take a moment to review the generated skeleton which will be filled in later. Open the **ProductServiceSkeleton.java** file and we will see the methods created for each operation defined in the WSDL file, with an **OMElement** object being provided as both the parameter and response of each method.

```
/**
 * ProductServiceSkeleton.java
 *
 * This file was auto-generated from WSDL
 * by the Apache Axis2 version: 1.6.2 Built on : Apr 17, 2012 (05:33:49 IST)
 */
package com.sparta.ows;

/**
 * ProductServiceSkeleton java skeleton for the axisService
 */
public class ProductServiceSkeleton {

    /**
     * Auto generated method signature
     *
     * @param fetchLotNumberRequest
     * @return fetchLotNumberResponse
     */

    public org.apache.axiom.om.OMElement
    fetchLotNumber(org.apache.axiom.om.OMElement fetchLotNumberRequest) {
        // TODO : fill this with the necessary business logic
        throw new java.lang.UnsupportedOperationException("Please implement
" + this.getClass().getName()
        + "#fetchLotNumber");
    }

}

/**
```

```

    * Auto generated method signature
    *
    * @param fetchProductNumberRequest
    * @return fetchProductNumberResponse
    */

    public org.apache.axiom.om.OMElement
    fetchProductNumber(org.apache.axiom.om.OMElement
    fetchProductNumberRequest) {
        // TODO : fill this with the necessary business logic
        throw new java.lang.UnsupportedOperationException("Please implement
    " + this.getClass().getName()
        + "#fetchProductNumber");
    }

    /**
    * Auto generated method signature
    *
    * @param fetchProductDetailsRequest
    * @return fetchProductDetailsResponse
    */

    public org.apache.axiom.om.OMElement
    fetchProductDetails(org.apache.axiom.om.OMElement
    fetchProductDetailsRequest) {
        // TODO : fill this with the necessary business logic
        throw new java.lang.UnsupportedOperationException("Please implement
    " + this.getClass().getName()
        + "#fetchProductDetails");
    }

    /**
    * Auto generated method signature
    *
    * @param createComplaintChildrenRequest
    * @return
    */

    public void createComplaintChildren(org.apache.axiom.om.OMElement
    createComplaintChildrenRequest) {
        // TODO : fill this with the necessary business logic
    }

}

```

5.4 Search

As in the IWS chapter, the first example that we will examine will be the simplest one. This section will focus on setting up a search to an external database from a single field on the form. Our example will be a search for a Product Number based on an input of a partial product number prefix – the system will search for products starting with that string and return a list of values.

First we will review the WSDL signature for the search call, then fill in the code, and finally configure the operation in TrackWise WebAdmin.

5.4.1 WSDL Definition

Our WSDL will reflect an operation `FetchProductNumber` that takes as input a string `productNumber` and returns a string `productNumber`.

Binding

```
<wsdl:operation name="FetchProductNumber">
  <soap:operation soapAction="http://ows.sparta.com/FetchProductNumber"/>
  <wsdl:input>
    <soap:body use="literal"/>
  </wsdl:input>
  <wsdl:output>
    <soap:body use="literal"/>
  </wsdl:output>
</wsdl:operation>
```

Port Type

```
<wsdl:operation name="FetchProductNumber">
  <wsdl:input message="tns:FetchProductNumberRequest">
  </wsdl:input>
  <wsdl:output message="tns:FetchProductNumberResponse">
  </wsdl:output>
</wsdl:operation>
```

Messages

```
<wsdl:message name="FetchProductNumberRequest">
  <wsdl:part name="parameters" element="tns:FetchProductNumberRequest">
  </wsdl:part>
</wsdl:message>
<wsdl:message name="FetchProductNumberResponse">
  <wsdl:part name="parameters" element="tns:FetchProductNumberResponse">
  </wsdl:part>
</wsdl:message>
```

Data Types

```
<xs:element name="FetchProductNumberRequest">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="productNumber" type="xs:string"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>
<xs:element name="FetchProductNumberResponse">
  <xs:complexType>
    <xs:sequence>
      <xs:element minOccurs="0" name="productNumber" type="xs:string"/>
      <xs:element minOccurs="0" name="detailedMessage" type="tns:DetailedMessage"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

Note the Response element contains 2 elements, the productNumber and also detailedMessage element. This element can be used to provide a message prompt back to the user, for example to inform them no values could be found or too many values were found.

The Detailed Message

```
<xs:complexType name="DetailedMessage">
  <xs:sequence>
    <xs:element name="message" nillable="false" type="xs:string"/>
    <xs:element name="isSuccess" nillable="true" type="xs:boolean"/>
  </xs:sequence>
</xs:complexType>
```

5.4.2 Code

Our `ProductServiceSkeleton` contains an operation `fetchProductNumber`, defined by the signature below.

```
public OMElement fetchProductNumber(OMElement fetchProductNumberRequest)
```

This method will perform the following logic:

- Parse the parameter to obtain the supplied partial Product Number.
- Perform a look-up of the available Product Numbers.
- Return the identified Product Number(s) back to the client.

Alternatively (in case of no products found), provide a Detailed Message to the client instead.

To parse the supplied product number, we need to get the child element in the message and read its value, using an Util Class method:

```
String productNumber = Util.getFirstChildWithLocalName  
    (fetchProductNumberRequest, "productNumber").getText
```

The supplied `CourseUtil.fetchProductNumber()` will perform the lookup of the product number from an external database, returning a list of identified product numbers. Once the response is identified, the service will return the value. As more than one value may be available to return, TrackWise is able to handle these scenarios by including each available option encapsulated by a `<value>` element.

Encapsulate each item in the returned product number list in a `<value>` tag.

```
String value; //As Returned from the lookup call.  
OMElement valueOM = factory.createOMElement("value", nullNS);  
valueOM.setText(value);
```

All ValueOM's , will be added as Childrens of productOM:

```
OMElement productOM = factory.createOMElement("productNumber", nullNS);  
productOM.addChild(valueOM);
```

productOM, will be added as Child of wsResponse that will be returned as response :

```
OMElement wsResponse = factory.createOMElement("fetchProductNumberResponse",  
    nullNS);  
wsResponse.addChild(multiValueOM);
```

See the Example created by the Instructor.

5.4.3 Configuration



Once the code has been completed, it is now time to configure and test the operation from within TrackWise. To configure an OWS Operation, the following steps need to be performed:

1. Add new WebService to the configuration.
2. Parse the WSDL for the new WebService.
3. Include operations.
4. Bind operation data types to TrackWise fields.
5. Trigger operations with specific configuration elements.
6. To add a New WebService to the configuration, in WebAdmin browse to the **System Configuration, Outbound WebServices** screen. Click **New**. Provide the URL of the `ProductServiceSkeleton` created earlier and select **Retrieve Operations from WSDL**.

<New Record>

Save Cancel

Name WS Course

WSDL Location URL

Retrieve Operations from WSDL

Available Operations

- createComplaintChildren
- fetchLotNumber
- fetchProductDetails
- fetchProductNumber

7. Select the `fetchProductNumber` operation and move to the Right-hand screen. Then select the **notepad** icon to configure the operation.
8. Select the **Type of Search**. The **Parameters** window displays, allowing the ability to map the `productNumber` input and output parameter to the **Product Number** data field under the request/response parameters complexes.

▶ Name	FetchProductNumber			
▶ Target URL	http://localhost:8081/WS_OWS/services/ProductService			
▶ Namespace URI	http://ows.sparta.com			
Soap Action URI	http://ows.sparta.com/FetchProductNumber			
▶ Timeout (seconds)	30			
Qualified	☐			
SSL	☐			
Authentication Type	None			
▶ Type	Search			
Preserve Data Integrity	☐			

Parameters

Input Parameters (To Web Service)

Delete	Sequence	▶ Name	▶ Type	Data Field
☐	1	productNumber	String	Product Number (Single Selection)

Output Parameters (From Web Service)

▶ Name	▶ Type	Value	Preview Screen	Auto Select
productNumber	String	Product Number (Single Selection)		☒
detailedMessage	Complex			

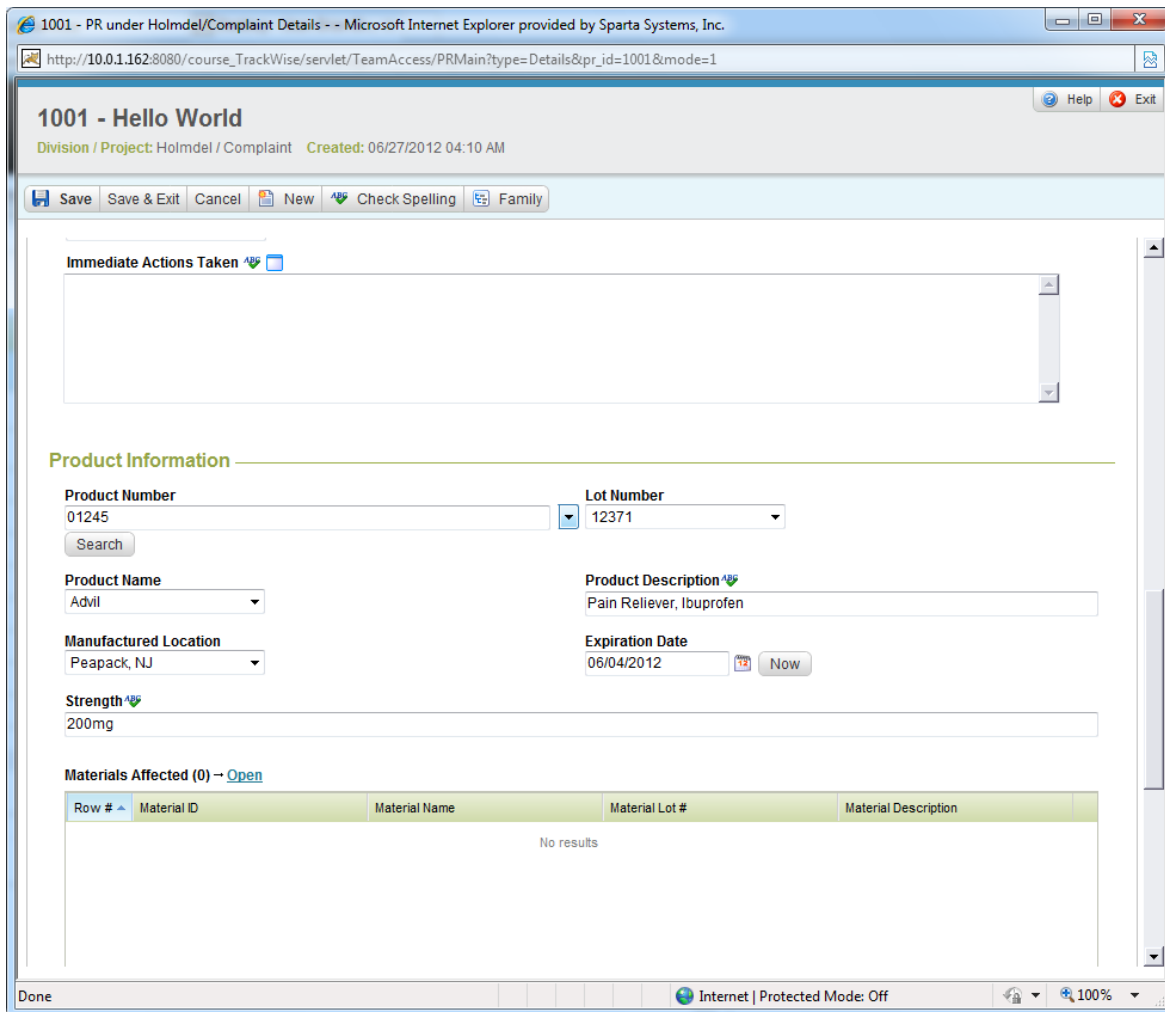
Database: wscourse

9. Select **OK** and then click **Save** to save the Outbound Web Service Configuration. Now it is time to associate the triggering configuration (in this case, the DataField itself) with the newly added WebService operation. Browse to the **DataFields- Product Number** screen. Check the **Trigger Outbound Webservices** box, then select the `fetchProductNumber` operation from the **Search Operation** dropdown. Click **Save**.

Data Field Details: Product Number

Details		Selection Values	Dependent Field
Save Save+ Cancel			
System Settings			
Field Name - Customized	Product Number		
Field Type	Single Selection		
Field Category	PR (User Defined)		
Is Visible	☒		
Selection Values Included In Configuration	No		
Is Virtual	☐		
Audit Trail Activity Type Triggered	<No Options Found>		
Is External	☐		
Trigger Outbound Web Services	☒		
Outbound Web Services Configuration			
Search Operation	FetchProductNumber		
Populate Operation	<No Options Found>		

- After saving the changes, it is time to test the service. First restart the Apache Tomcat service that hosts the TrackWise install. Then enter TrackWise. It should now be possible to enter **1** into the **Product Number** and click **Search**. The set of values that start with **1** should be returned by the service.



1001 - PR under Holmdel/Complaint Details - - Microsoft Internet Explorer provided by Sparta Systems, Inc.

http://10.0.1.162:8080/course_TrackWise/servlet/TeamAccess/PRMain?type=Details&pr_id=1001&mode=1

1001 - Hello World

Division / Project: Holmdel / Complaint Created: 06/27/2012 04:10 AM

Save Save & Exit Cancel New Check Spelling Family

Immediate Actions Taken

Product Information

Product Number: 01245 Lot Number: 12371

Search

Product Name: Advil Product Description: Pain Reliever, Ibuprofen

Manufactured Location: Peapack, NJ Expiration Date: 06/04/2012 Now

Strength: 200mg

Materials Affected (0) → Open

Row #	Material ID	Material Name	Material Lot #	Material Description
No results				

Done Internet | Protected Mode: Off 100%

5.4.4 Dependent Search Example

There are times when the input of one search operation depends on the output of another – in these scenarios, the fields act in a manner similar to Dependent Fields within TrackWise. Depending on the situation, these fields may be able to trigger each other automatically, which is more specifically referred to as a Daisy-Chain search. In this case, we will use the output of the `ProductNumber` search as an input to the `Lot Number` search, forming a dependency between the two fields.

The design of the `fetchLotNumber` operation will be similar to the `fetchProductNumber` operation, but will take multiple inputs.



In the next exercise we will configure the `fetchLotNumber()` Operation In TW , get `productNumber` and `lotNumber` (prefix) as parameters, and return valid lot numbers as outputs

1. Locate the `fetchLotNumber()` operation within the skeleton.
2. Parse the child elements `productNumber` and `lotNumber` from the parent Request object.
3. Use the `CourseUtil.fetchLotNumber()` operation to determine the available lot numbers.
4. Create the parent `Response` object to be returned.
5. Create the `lotNumber` object to be returned.
6. Add the available values to be returned.
7. Return the parent `Response` object.
8. Configure the `FetchLotNumber` operation in WebAdmin.

This is How the mapping Of `fetchLotNumberResponse` should look like, Notice to the Detailed Message Complex :

ProductServiceSkeleton

Operation: `fetchLotNumber` > `fetchLotNumberResponse`

Add OK Cancel					
Delete	Name	Type	Value	Preview Screen	Auto Select
☐	lotNumber	String	DataFields Lot Number (Single Selection)		☒
☐	detailedMessage	Complex			☐

ProductServiceSkeleton

Operation: `fetchLotNumber` > `fetchLotNumberResponse` > `detailedMessage`

Add OK Cancel					
Delete	Name	Type	Value	Preview Screen	Auto Select
☐	message	String	Detailed Message		☐
☐	isSuccess	String	Success Indicator		☐

5.5 Populate

Unlike the search operation that you have been using that can only return one parameter back to the form, the populate operation can return multiple. In this case, we will search for the lot number, which is the unique identifier for the remainder of the product attributes – once the lot number is chosen, we can automatically fill in the remainder of the product information fields from data that we know.

5.5.1 WSDL Definition

The WSDL will describe an operation, `fetchProductDetails`, which takes both the Product Number and Lot Number and returns a set of details into other Data Fields, populating those fields.

Binding

```
<wsdl:operation name="FetchProductDetails">
  <soap:operation soapAction="http://ows.sparta.com/FetchProductDetails"/>
  <wsdl:input>
    <soap:body use="literal"/>
  </wsdl:input>
  <wsdl:output>
    <soap:body use="literal"/>
  </wsdl:output>
</wsdl:operation>
```

Port Type

```
<wsdl:operation name="FetchProductDetails">
  <wsdl:input message="tns:FetchProductDetailsRequest">
  </wsdl:input>
  <wsdl:output message="tns:FetchProductDetailsResponse">
  </wsdl:output>
</wsdl:operation>
```

Messages

```
<wsdl:message name="FetchProductDetailsRequest">
  <wsdl:part name="parameters" element="tns:FetchProductDetailsRequest">
  </wsdl:part>
</wsdl:message>
<wsdl:message name="FetchProductDetailsResponse">
  <wsdl:part name="parameters" element="tns:FetchProductDetailsResponse">
  </wsdl:part>
</wsdl:message>
```

Data Types

```
<xs:element name="FetchProductDetailsRequest">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="productNumber" type="xs:string"/>
      <xs:element name="lotNumber" type="xs:string"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>
<xs:element name="FetchProductDetailsResponse">
  <xs:complexType>
    <xs:sequence>
      <xs:element minOccurs="0" name="productName" type="xs:string"/>
      <xs:element minOccurs="0" name="strength" type="xs:string"/>
      <xs:element minOccurs="0" name="productDescription"
        type="xs:string"/>
      <xs:element minOccurs="0" name="expirationDate" type="xs:string"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

```
<xs:element minOccurs="0" name="materialsAffected"
type="xs:string"/>
<xs:element minOccurs="0" name="detailedMessage"
type="tns:DetailedMessage"/>
</xs:sequence>
</xs:complexType>
</xs:element>
```

Note that while `materialsAffected` is included in the WSDL, this component will be added in the following section and should not yet be configured.

5.5.2 Code

Our `ProductServiceSkeleton` contains an operation `fetchProductDetails`, defined by the signature below.

```
public OMElement fetchProductDetails(OMElement fetchProductDetailsRequest)
```

This method will perform the following logic:

- Parse the parameter to obtain the supplied `ProductNumber` and `LotNumber`
- Perform a look-up of the `ProductDetails` based on the Product and Lot Numbers.
- Turn the Product Details information into an `OMElement` structured response.
- Alternatively (in case of no products found), provide a Detailed Message to the client instead.

The supplied `CourseUtil.fetchProductDetails()` will perform the lookup of the Product Details from the external system, however this time at most 1 `ProductDetails` object will be returned as Product and Lot are unique identifiers for the Product information being searched. The `ProductDetails` object can then be read via accessors (getters) to build the return web service message.

For each field in the `ProductDetailsResponse`, there is a corresponding field in the `ProductDetails` object:

```
// ProductName
String name = productDetails.getName();
OMElement nameOM = factory.createOMElement("productName", nullNS);
nameOM.setText(name);
fetchProductDetailsResponse.addChild(nameOM);
```

Please refer to the included solved codebase as a guide if needed, observing that the `MaterialAffected` can be omitted at this time.

5.5.3 Configuration



Workbook Exercises

In the following exercise we will configure and implement the `fetchProductDetails` operation that has `productNumber` and `lotNumber` as input parameters and return complex contains all product details, including the “materials affected” Grid data.

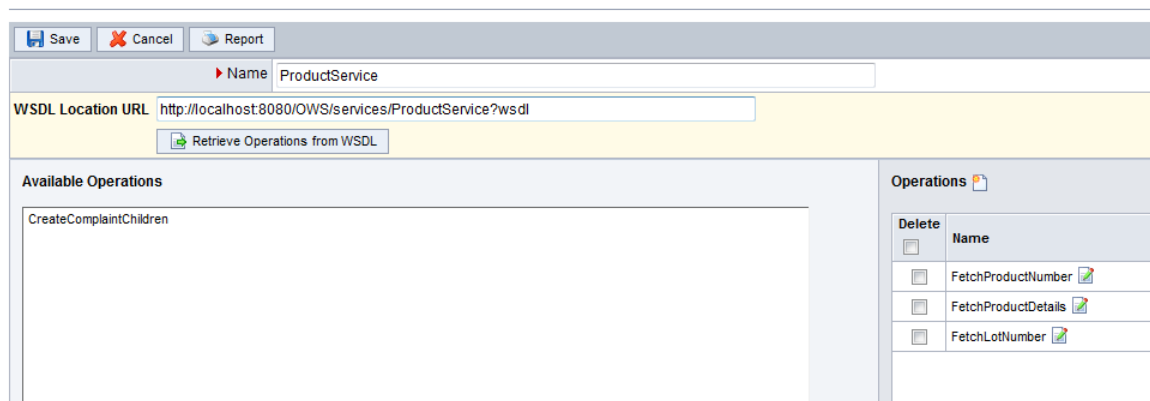
Note: This exercise simulates a real life situation. The FetchProductDetails is a service provider which returns data based on the input parameters it client sends in. However, the service provider can't tell its client how and where the returned results shall be used.

In this case, TrackWise is the client. It is up to the TrackWise developer (you) to decide where and how the results are used. In this exercise, we decided to display the results on the main form.

Steps:

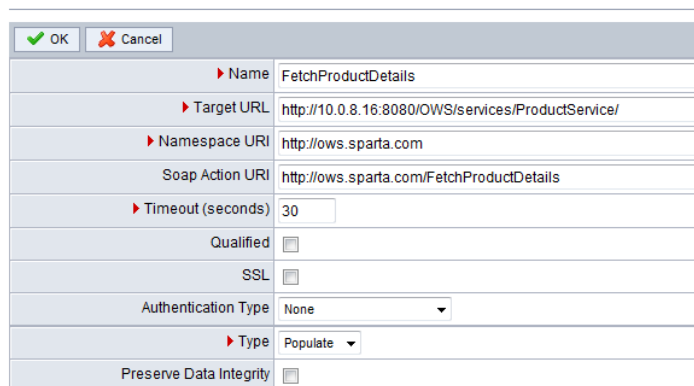
- Returning to the Outbound Web Service screen, add the `FetchProductDetails` Operation to the configuration.

ProductService



- To configure a Populate operation, select **Populate** in the **Type** field.

ProductService



- Setting the input and output parameters accordingly (omitting Materials Affected).

ProductServiceSkeleton

Operation: fetchProductDetails > fetchProductDetailsRequest

Add OK Cancel					
Delete	Sequence	Name	Type	Data Field	Require Trigger
☐	1	productNumber	String	Product Number (Single Selection)	☐
☐	2	lotNumber	String	Lot Number (Single Selection)	☐

Parameters

Output Parameters (From Web Service)

Name	Type	Value	Preview Screen	Auto Select
productName	String	Product Name (Single Selection)	☐	☒
strength	String	Strength (String)	☐	☒ Select All ☐
productDescription	String	Product Description (String)	☐	☒ Select All ☐
manufacturedLocation	String	Manufactured Location (Single Selection)	☐	☒ Select All ☐
expirationDate	String	Expiration Date (Date)	☐	☒ Select All ☐
materialsAffected	String		☐	☒ Select All ☐
detailedMessage	Complex			

- Click **OK**, then click **Save** to save the changes to the Outbound Web Service.
- Configure the service to be Triggered Upon Value Selection at Lot Number Field:

Data Field Details: Lot Number

Details
Selection Values
Dependent Field

Save
Save+
Cancel

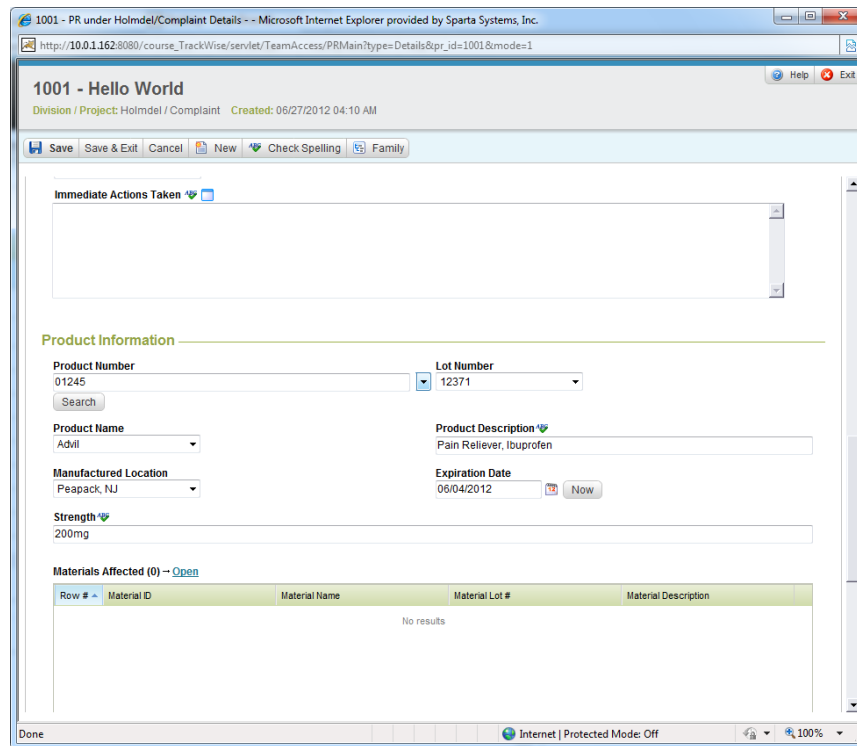
System Settings

Field Name - Customized	Lot Number	Highlight Change	☐
Field Type	Single Selection	Meaning Comment Req'd	☐
Field Category	PR (User Defined)	Selection Type	☒ Drop Down List ☐ Radio Button Group ☐ Table-Driven Selection
View Name	TW_V_LOT_NUMBER	Is Visible	☒
Seq. No	45	Selection Values Included In Configuration	No
Sorted by	Seq. No	Is Virtual	☐
Copy To Child	☐	Audit Trail Activity Type Triggered	<No Options Found>
Reverse Parent Field		Is External	☐
		Trigger Outbound Web Services	☒

Outbound Web Services Configuration

Search Operation	fetchLotNumber
Populate Operation	fetchProductDetails

6. Restart Apache Tomcat. Login to TrackWise and create a new Complaint Record. Choose a **Product Number** and **Lot Number**, which should trigger the Populate call for the `FetchProductDetails` operation. The **Product Name**, **Strength**, **Description**, and **Expiration Date** fields can now be populated.



5.6 Populate into Grid

Working with grids using OWS is more complicated than just working with normal TrackWise fields that are directly on the form. This section will step through an example of how to populate the Materials Affected grid as an expansion on the previous populate call for the Product Details.

5.6.1 Grid Fields

TrackWise Outbound WebServices is able to send and receive GridFields in many of the same ways that traditional data fields are handled, however additional XML markup must be used to define the grid structure. As an example of the XML structure used, the Materials Affected grid would be transported in the following manner.

```
<materialsAffected>
  <![CDATA[
    <gridfield>
      <row>
        <column>
          <name>Material ID</name>
          <value>1234</value>
        </column>
        <column>
          <name>Material Name</name>
```

```

        <value>matonetwothreefour</value>
      </column>
    <column>
      <name>Material Lot #</name>
      <value>110747</value>
    </column>
    <column>
      <name>Material Description</name>
      <value>A description of the material</value>
    </column>
  </row>
  <row>
    <column>
      <name>Material ID</name>
      <value>5678</value>
    </column>
    <column>
      <name>Material Name</name>
      <value>Secondmat</value>
    </column>
    <column>
      <name>Material Lot #</name>
      <value>2615948</value>
    </column>
    <column>
      <name>Material Description</name>
      <value>Desc of second material</value>
    </column>
  </row>
</gridfield>
  ]]>
</materialsAffected>

```

Note that after the element tag, the entire grid contents are then wrapped in a CDATA tag.

5.6.2 WSDL Definition

Defining Grid Fields within the context of the WSDL is straightforward as Grids are identified as a string type, with the inner XML structure being processed as a secondary element.

5.6.3 Code

The `fetchProductDetails` operation can be expanded to include the Material Affected grid structure. The provided `ProductDetails` object includes a method to transport the fetched grid data as List of key-value pairs which contain the respective column name and value information.

The signature to obtain the grid:

```

ProductDetails productDetails = CourseUtil.fetchProductDetails(productNumber,
    lotnumber);
List<Map<String, String>> materialsAffected = productDetails.getMaterialAffected()

```

Each element in the list represents a row (will require a `<row>` OMElement) and will then contain `<column>` and subsequently `<name>` and `<value>` children.

Please refer to the solved codebase for additional insight into building the grid object.

5.6.4 Configuration

To configure the newly added grid, return to the `fetchProductDetails` configuration details screen. Mapping the `materialsAffected` element to the Materials Affected DataField will allow the Grid to be seen by TrackWise.

Parameters				
Output Parameters (From Web Service)				
Name	Type	Value	Preview Screen	Auto Select
productName	String	Product Name (Single Selection)		☒
strength	String	Strength (String)	☐	☒ Select All ☐
productDescription	String	Product Description (String)	☐	☒ Select All ☐
manufacturedLocation	String	Manufactured Location (Single Selection)	☐	☒ Select All ☐
expirationDate	String	Expiration Date (Date)	☐	☒ Select All ☐
materialsAffected	String	Materials Affected (Grid)	☐	☒ Select All ☐
detailedMessage	Complex			

After restarting Apache Tomcat, triggering the `fetchProductDetails` call should now also return the grid element.

5.7 Search and Populate Inside of Grid

Another use case with Grids that is commonly requested is having functionality within the grid that is similar to the search and populates operations on the main form that we coded for earlier. In the case that we will examine, there is a Product Issue record that captures and evaluates an event from another system which may contain multiple products. Since there can be multiple products associated to a Product Issue the product information is captured entirely within a grid. Ideally, we would like to capture and validate the product information from the external system in the same way. Using OWS, we can accomplish this use case.

For this example, we will be able to reuse the `fetchProductNumber`, `fetchLotNumber`, and `fetchProductDetails` operations, however the operations will be mapped to a different set of DataFields. Please see the earlier descriptions of the Search and Populate examples for insight into the WSDL and Code structure.

5.7.1 Configuration



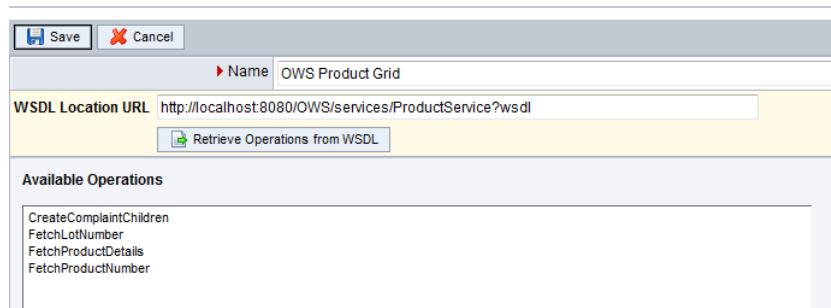
Note: As mentioned in previous sections, ProductService is considered as a service provider. It returns data based on the input from its client. But it can't tell where and how the returned results shall be used.

In the previous exercise, we decided to display the result on the main form. In this exercise, we still use the same web service as the data source; however, we decide to display the results in a grid. In really life, this may be different ways of using the same service in the same project or a different project.

In this exercise we will configure all the Operations we used before to be called inside a Grid.

1. In WebAdmin, proceed to the Outbound Web Services configuration screen. For this exercise, we will choose to create a New configuration.

<New Record>



Save Cancel

Name OWS Product Grid

WSDL Location URL http://localhost:8080/OWS/services/ProductService?wsdl

Retrieve Operations from WSDL

Available Operations

- CreateComplaintChildren
- FetchLotNumber
- FetchProductDetails
- FetchProductNumber

2. Bring the `FetchLotNumber`, `FetchProductDetails`, and `FetchProductNumber` operations to the right-hand screen.
3. Configure the `FetchProductNumber` search operation, this time using the **Product Number – Grid** field.

OWS Product Grid

Operation: fetchProductNumber

OK Cancel	
Name: fetchProductNumber	
Target URL: http://localhost:8081/OWS/services/ProductServiceSkeleton.ProductServiceSkeletonHttpSoap11Endpoint/	
Namespace URI: http://ows.sparta.com	
Soap Action URI: urn:fetchProductNumber	
Timeout (seconds): 30	
Qualified: ☒	
SSL: ☐	
Authentication Type: None	
Shared File Location:	
Type: Search	

Parameters					
Input Parameters (To Web Service)					
Delete	Sequence	Name	Type	Data Field	Require Trigger
☐	1	fetchProductNumberReq	Complex		☐

Output Parameters (From Web Service)					
Name	Type	Value	Preview Screen	Auto Select	Preserve Data Integrity
fetchProductNumberRes	Complex				☐

<New Record>

Operation: fetchProductNumber > fetchProductNumberRequest

Add OK Cancel					
Delete	Sequence	Name	Type	Data Field	Require Trigger
☐	1	productNumber	String	Product Number- Grid (Single Selection)	☐

<New Record>

Operation: fetchProductNumber > fetchProductNumberResponse

Add OK Cancel						
Delete	Name	Type	Value	Preview Screen	Auto Select	Preserve Data Integrity
☐	productNumber	String	DataFields Product Number- Grid (Single Selection)		☒	☐

- Map the **Product Number – Grid** data field to trigger the Outbound operation.

Data Field Details: Product Number- Grid

Details		Selection Values	Dependent Field
Save Save+ Cancel			
System Settings			
Field Name - Customized	Product Number- Grid		
Field Type	Single Selection		
Field Category	PR (User Defined)		
Is Visible	☒		
Selection Values Included In Configuration	No		
Is Virtual	☐		
Is External	☐		
Trigger Outbound Web Services	☒		
Outbound Web Services Configuration			
Search Operation	FetchProductNumber		
Populate Operation	OWS Product Grid FetchProductNumber		

5. Configure the FetchLotNumber search operation, this time using the Lot Number – Grid field.

OWS Product Grid

OWS Product Grid					
OK Cancel					
Name FetchLotNumber					
Target URL http://localhost:8081/OWS/services/ProductServiceSkeleton.ProductServiceSkeletonHttpSoap11Endpoint/					
Namespace URI http://ows.sparta.com					
Soap Action URI urn:fetchLotNumber					
Timeout (seconds) 30					
Qualified ☒					
SSL ☐					
Authentication Type None					
Shared File Location					
Type Search					
Parameters					
Input Parameters (To Web Service)					
Add					
Delete ☐	Sequence 1	Name FetchLotNumberRequest	Type Complex	Data Field	Require Trigger ☐
Output Parameters (From Web Service)					
Name FetchLotNumberResponse	Type Complex	Value	Preview Screen	Auto Select ☐	Preserve Data Integrity ☐

OWS Product Grid

Operation: fetchLotNumber > fetchLotNumberRequest

OWS Product Grid					
Add OK Cancel					
Delete ☐					
Sequence 1	Name productNumber	Type String	Data Field Product Number- Grid (Single Selection)	Require Trigger ☐	
Sequence 2	Name lotNumber	Type String	Data Field Lot Number - Grid (Single Selection)	Require Trigger ☐	

OWS Product Grid

Operation: fetchLotNumber > fetchLotNumberResponse

Add OK Cancel				
Delete	Name	Type	Value	
☐	lotNumber	String	DataFields Lot Number - Grid (Single Selection)	Preview Screen ☐
☐	detailedMessage	Complex		Auto Select ☒
				Preserve Data Integrity ☐

- Map the **Lot Number – Grid** data field to trigger the outbound call.

Data Field Details: Lot Number - Grid

Details		Selection Values	Dependent Field
Save Save+ Cancel			
System Settings			
Field Name - Customized	Lot Number - Grid		
Field Type	Single Selection		
Field Category	PR (User Defined)		
Is Visible	☒		
Selection Values Included In Configuration	No		
Is Virtual	☐		
Is External	☐		
Trigger Outbound Web Services	☒		
Outbound Web Services Configuration			
Search Operation	FetchLotNumber		
Populate Operation	FetchProductDetails		

- Finally configure the `FetchProductDetails` populate operation, using the respective - **Grid** fields. Note the **Material Affected** can be omitted for this exercise as Grids cannot be contained inside another Grid field.
- Save the mapping.

OWS Product Grid

OK Cancel	
Name: fetchProductDetails	
Target URL: http://localhost:8081/OWS/services/ProductServiceSkeleton-ProductServiceSkeletonHttpSoap11Endpoint/	
Namespace URI: http://ows.sparta.com	
Soap Action URI: urn:fetchProductDetails	
Timeout (seconds): 30	
Qualified: ☒	
SSL: ☐	
Authentication Type: None	
Shared File Location:	
Type: Populate	

Parameters				
Input Parameters (To Web Service)				
Add				
Delete	Sequence	Name	Type	Data Field
☐	1	fetchProductDetailsReq	Complex	
Output Parameters (From Web Service)				
Name	Type	Value	Preview Screen	Auto Select
fetchProductDetailsRes	Complex			

OWS Product Grid

Operation: fetchProductDetails > fetchProductDetailsRequest

Add OK Cancel				
Delete	Sequence	Name	Type	Data Field
☐	1	productNumber	String	Product Number - Grid (Single Selection)
☐	2	lotNumber	String	Lot Number - Grid (Single Selection)

- After restarting Apache Tomcat, the Outbound Web Service operations will now be available from within the Product Information Grid on the Product Issue record.

5.8 OWS Limitations

- Timeout: there are different timeouts for each Service call: Apache level ,Axis2 level and TW level , This timeouts can be extended, but the service running time is an important parameter at OWS call.
- XML parsing issues at TW side
- Data returned by OWS will be saved to the DB only when the user will save the Record.
- Before TW 8.5: Mapping values that are not valid, for Example: map to a single selection value that is not valid can break the Snapshot.

5.9 OWS Security

When performing OWS over the Internet, security should be considered to protect confidential information, and assure information is being sent from reputable sources. As explained in the Microsoft Best Practices and Patterns document, "Improving Web Services" found at: <http://wcfsecurityguide.codeplex.com/>, the Foundations of Security are Authentication, Authorization, Auditing, Confidentiality, Integrity, and Availability.

The current goal of the OWS Authentication and Security module is to provide enough options to sufficiently meet the foundation's requirements. By providing the ability to configure the

Secure Socket Layer (SSL) channel with UserName/Password/Timestamp verification, or message Encryption and Signing, either transport or message layer security is assured. These security types should protect from eavesdropping, impersonation, replaying, tampering, and denial of service.

6. Putting it all Together (IWS/OWS)

6.1 Introduction

Once it is known how to properly create applications to leverage IWS and OWS individually, we can start to create more complex program interactions. OWS and IWS can be coupled together to perform functionality that neither would be able to accomplish individually. An OWS transfer operation can be triggered to occur on the performance of an activity, which will push information out of TrackWise via OWS for processing. Unlike the search and populate operations, the transfer operation does not accept a return parameter – it only sends information out. Using transfer, we can push information from TrackWise to a program that takes some other action on it – like passing it to an IWS client for processing.

A scenario where this functionality is particularly useful is the case where children need to be generated from a record based on rows located in a grid. In our example Complaints system, we have a Product Issue record that can capture multiple products in a grid, and a Complaints record that only takes one product. The business need is to have multiple Complaints generated if the Product Issue is determined to be a complaint – something that TrackWise coordinator cannot do. Instead, we can build a program that uses OWS to push the product information grid in the Product Inquiry out to a custom developed program which will process the grid and create a Complaint record as a child of the Product Issue for each row.

This chapter will focus on implementing the above use case in the example Complaints system, leveraging the OWS transfer operation.

6.2 OWS Transfer

The OWS transfer operation is the first piece of this problem that we will examine. The following examples will take you through setting up an OWS transfer operation and processing through the grid information that was passed in.

6.2.1 WSDL

The WSDL definitions for this component differ from the previous examples most notably by the fact that the operation only has a defined input and does not have an output available. This structure, used in our example `CreateComplaintChildren`, has a corresponding code signature of a void method.

Binding

```
<wsdl:operation name="CreateComplaintChildren">
  <soap:operation soapAction="http://ows.sparta.com/CreateComplaintChildren"/>
  <wsdl:input>
    <soap:body use="literal"/>
  </wsdl:input>
</wsdl:operation>
```

Port Type

```
<wsdl:operation name="CreateComplaintChildren">
  <wsdl:input message="tns:CreateComplaintChildrenRequest">
    </wsdl:input>
  </wsdl:operation>
```

Messages

```
<wsdl:message name="CreateComplaintChildrenRequest">
  <wsdl:part name="parameters" element="tns:CreateComplaintChildrenRequest">
    </wsdl:part>
  </wsdl:message>
```

Data Types

```
<xs:element name="CreateComplaintChildrenRequest">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="recordId" type="xs:int"/>
      <xs:element name="eventDate" type="xs:string"/>
      <xs:element name="awarenessDate" type="xs:string"/>
      <xs:element name="description" type="xs:string"/>
      <xs:element name="customersWords" type="xs:string"/>
      <xs:element name="initialReporter" type="xs:string"/>
      <xs:element name="initialReporterCompany" type="xs:string"/>
      <xs:element name="productInformationGrid" type="xs:string"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

Notice that included in the list of Data Types is the `productInformationGrid` – as was the case in the previous examples, grid fields are treated as `String` fields when being transferred via Outbound Web Services (recall they have their own internal XML schema encapsulated within a `CDATA` block).

6.2.2 Code

Our `ProductServiceSkeleton` has the `CreateComplaintChildren` method ready to be implemented. We need to parse the field values from the input message, including parsing of the Product Information grid.

As can be seen in the WSDL file, the following values are to be parsed from the input `OME` element:

- `recordId`
- `eventDate`
- `awarenessDate`
- `customersWords`
- `initialReporter`
- `initialReporterCompany`
- `productInformation` (GridField with the hierarchy as follows)
 - `gridField`
 - `row`
 - `column`
 - `name`
 - `value`

6.2.3 Configuration

The configuration of the `CreateComplaintChildren` operation follows a similar pattern as the previous examples, however this time we will bind the operation to an activity.



1. Return to the Outbound Web Service Configuration screen.
2. Add the `CreateComplaintChildren` operation to the right-hand side of the screen.
3. Edit the details of the `CreateComplaintChildren` operation.
4. Select the Type Transfer. Observe only the Input Parameters are available for configuration.
5. Map the service parameters to the data fields, including the Product Information Grid.
6. Choose OK and Save the record.
7. Move to the configuration screen for the **Activity Type** “Confirm Complaint”.
8. Add the **Trigger Outbound Operation** “`CreateComplaintChildren`”. Save the changes.
9. Restart Apache Tomcat that hosts TrackWise.

6.3 IWS Child Creation

The second major piece of the puzzle for the problem that we are attempting to solve is the creation of children records. This section will focus on building an IWS to create children records.

Now that the OWS `CreateComplaintChildren` operation has been called and the parameters parsed, it is time to use this information to create new children records. We will create the children records in a similar manner to the way records were created in Chapter 4, however we will repeat this for each row in the Product Information Grid. After all of the children records are created, we will then post the follow-up “Complaints Generated” activity against the parent Product Issue record.



The create children part of the program should take on the following logic:

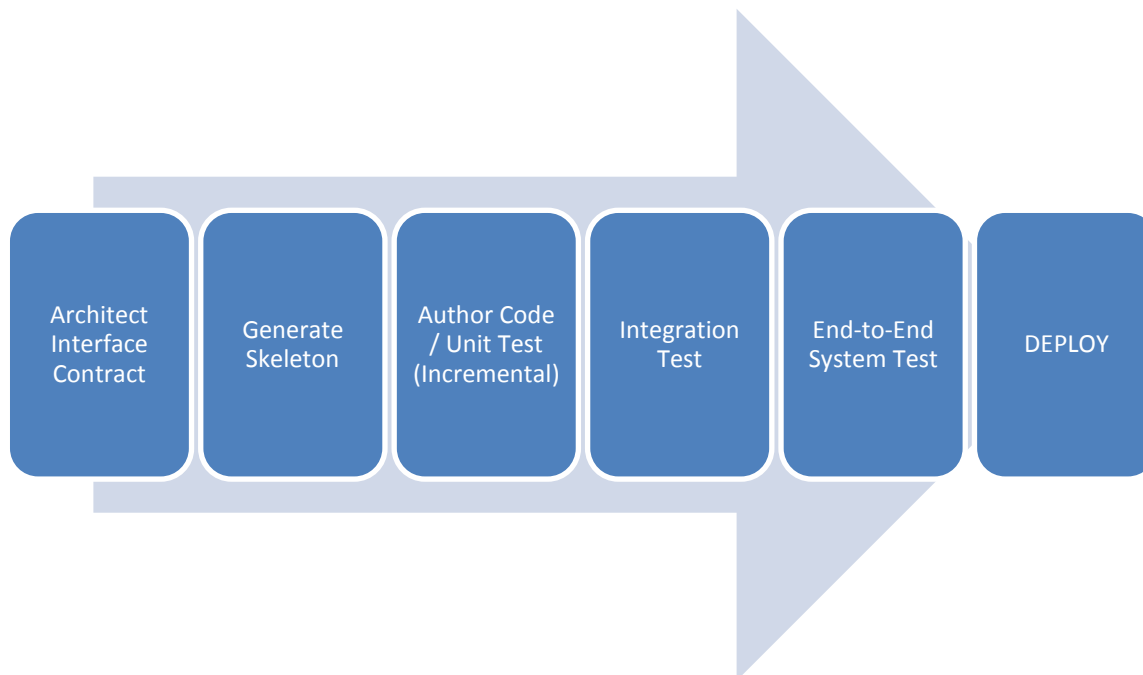
1. Connect to TrackWise Web Service.
2. For each row in the **Product Information Grid**:
 - a. Obtain an empty `PR` (Holmdel / Complaint)
 - b. Populate the following Data Fields:
 - i. **Parent ID** (Record ID from the input)
 - ii. **Event Date**
 - iii. **Awareness Date**
 - iv. **Customers Words**
 - v. **Initial Reporter** (Query by name)

- vi. **Initial Reporter Company** (Query by name)
 - vii. **Product Number** (from Product Number – Grid)
 - viii. **Lot Number** (From Lot Number – Grid)
 - ix. **Product Name** (From Product Name – Grid)
 - x. **Product Description** (From Product Description – Grid)
 - xi. **Manufactured Location** (From Manufactured Location – Grid)
 - xii. **Expiration Date** (From Expiration Date – Grid)
 - xiii. **Strength** (From Strength – Grid)
 - xiv. **Affected Materials** (From `CourseUtil.fetchAffectedMaterials()`)
- c. Create the child Complaint record using `tw.createPR()`
- 3. Post the **Complaints Generated** activity against the parent record (from Record Id).

7. Repeatable Process for Development

7.1 Introduction

Throughout this course we have looked at the detailed level of how to code web services. This chapter aims to provide a repeatable process for how to approach a problem requiring web services, from planning through code generation. A diagram is shown below for the general steps that are required to generate any given web service:



7.2 Process

Architect Interface Contract

Before coding is started on any web service, an agreement must be reached about what the web service is doing. In the case of an OWS, this involves generating a WSDL to specify the operations and data that will be exchanged. In the case of an IWS, this involves identifying what functionality exposed by the TrackWise API will be leveraged.

Generate Skeleton

From the WSDL used – the custom developed OWS WSDL – you will then be able to generate code stubs to fill in the methods of the program.

Author Code / Unit Test (Incremental)

This phase involves writing and testing the actual code as functionality is added. It is important to unit test methods, algorithms, functions, etc. to avoid a much larger troubleshooting effort later.

Integration Test

Integration testing involves testing communication between the various systems involved.

End-to-End System Test

Test the functionality and validate that the written code / service functions in the system as desired. End-to-End testing should ensure that the program and interactions all function as desired in the production environment.

DEPLOY

Deploy the code to the production environment.

8. TrackWise Web Services Efforts

So, one of the questions often asked is “what do I have to have in order to build web services with my own team?” What are the skills required the estimated time frame for a common use case, and how difficult it is to design and test the services, etc. The bottom-line is that writing the code may not be that difficult, however, there are many factors to be considered, and such as the scalability (e.g. would your code still work if TW is upgraded? Do you expect to add more service in future?), best practice (based on Sparta’s experience, certain ways are better than the others), industry knowledge, etc. So developing web services is not just writing the code, it’s more of an IT project that comes with cost and risks.

This is an Estimation example of common use cases developed by Sparta:

- **Import File into Grid And creates record for each Row – 20 day’s.**
- **Input records from flat/xml files (By CRM) and output properly formatted TrackWise records And Records Synchronizations. – 50 day’s.**
- **Master Data lookup , example :from CRM/ERP – 3 day’s**
- **Calculate and manipulate Fields : Columns summary (2 day’s) , Rate Calculation (4 day’s).**
- **Custom Utilities : tool that synchronizes Login Accounts / Persons between 2 TW Environments – 6 day’s.**

9. Troubleshooting Web Services

9.1 Introduction

To become proficient in web services, one must also learn how to trace, identify, diagnose and fix errors in the program. This involves not only syntactical errors but logic errors as well. In this chapter we will introduce several techniques for troubleshooting and error resolution.

One of the key points that should be taken from this chapter is the importance of documenting your program, enabling logging where it makes sense and testing your code.

9.2 Inbound Web Services

Under the IWS paradigm, the service interface is the provided API in which custom clients are written, providing much more control to the client for debugging and troubleshooting.

9.2.1 Client Logging

The first component to troubleshooting is to include or add logging to the client application, verifying the values of data being provided to the service as well the stack traces of any errors thrown. Use of detailed logging is one of the most effective ways to capture the flow of control and isolate issues.

9.2.2 Server Log Files

While the server side items are not nearly as controllable as the client side, the server –side log files occasionally reveal information that isn't always sent back the client. Within the TWS interface service directory (default "axis2")/WEB-INF/classes directory the Apache Log4j configuration file will indicate the logging level and the location of the log files (Default to Tomcat\logs\). Verify no stack traces or additional messages are available to review.

9.2.3 Connections

One issue that can cause pain to the TWS developer is through connection management. If the connections and sessions are not handled carefully, errors during reconnect, or even potentially session resets can occur. It is suggested that each domain control their connections to ensure that multiple programs do not attempt to use the same credentials at the same time. Also, if "Already Logged In" errors are appearing, it is likely the sessions are not being ended correctly. Verify through logging and monitoring that sessions are being closed after program cycle/execution.

9.2.4 Querying

Another area that may require troubleshooting are during attempts to query for records – often these are due to data field conflicts or permissions issues. If, for example, there are UDFs named identical to System Fields (a known issue), queries will fail with a cryptic error message. If this is the case, confirm the data field naming issue is not occurring.

9.3 Outbound Web Services

When working with Outbound Web Services, TrackWise behaves as the web service client while the service is custom developed, therefore the first place to troubleshoot tends to be server-side.

9.3.1 Server Log Files

As custom code is written on the server side during OWS, it is wise to include descriptive logging statements to provide insight into the status of objects and flow of control during processing. It is suggested that upon the calling of an operation, the invoking message immediately be written to the logs. Similarly, prior to returning the operation should write the outgoing message to the logs as well. Additional logging should be utilized as needed.

9.3.2 Client Log Files

In the case where TrackWise is the client, log files still may be used to help provide insight into the status of the objects and operations being called. In order to see these log statements, TrackWise must have its logging mode set to `DEBUG` by changing the logging level of the `log4j.properties` file contained within the `<TrackWise Install Directory>\WEB-INF\classes` folder. A restart of Tomcat is required after modifying this value.

Once in debug mode, the TrackWise log files are more verbose – using a Find tool search for the term “SendBlocking” – the messages labeled `SendBlocking.payload` and `SendBlocking.response` are the SOAP envelopes being pushed out of / received into TrackWise. Verify the messages are as expected.

9.3.3 Client Replacements

Another option for troubleshooting OWS issues would be to utilize a “client replacement” for the service to interact with. By either writing a web-service client to consume your service and/or using tools such as SoapUI, the calling of operations as well as the message structure provided can be controlled more accurately.

9.3.4 Common Errors

Some of the more common errors seen with OWS development are related to initializing the connections or returned element formatting. Often if an OWS operation fails, the culprit is likely either TrackWise failing to connect to the external service (in which case verify the configured addresses are correct and the wsdl can be reached from the server) or that message returned from the service cannot be parsed by the TrackWise parser. In this latter case, ensure the message structure conforms to the WSDL standard and that aside from the top-level element, no internal namespaces are qualified (including aliases).

10. Appendix

10.1 Glossary of Terms

Activity

An Activity contains the information of a particular action that was performed (or is scheduled) on behalf of a particular PR. Posting Activities against a PR, will in effect report on what was done or is about to be done for that PR.

API (Application Program Interface)

A well-defined programmatic interface to a server-side implementation of business and data logic. Many APIs accept arguments as input and return with a data response. Generally utilized by client application programs.

Entity

An Entity is a value such as a company, business or organization.

FlexField

FlexFields are Data Fields which can contain either a “free-text” description or an embedded (OLE) document. A user can define an unlimited number of FlexFields.

HTTP (Hypertext Transfer Protocol)

Industry-standard internet protocol capable of transferring data and requests through firewalls.

Person

A Person is an individual user of TrackWise.

PR (Problem Report)

Is the basic, or default name, for a record within the TrackWise system. This term can be configured to meet the system users’ needs, for example, AI for “Action Item”, etc. Each PR has a unique number, which is generated by the system. The PR number is the unique identifier (primary key) for all e-records in the system.

Session

An interactive exchange of requests (from a single application client) and responses (from application server) associated with a single client authentication / login.

SOAP (Simple Object Access Protocol)

Industry-standard lightweight protocol intended for exchanging structured information in a decentralized, distributed environment. This protocol normally “wraps” its XML content in a structured format that is generally understood to be used for data transfers in request/response situations.

System Field

A Data Field that is present with the initial configuration of the system. Most system Fields will have default names that can be changed by the system administrator.

TCP/IP

Industry-standard internet protocol for transferring low-level data packets.

Web Service

A server-hosted collection of APIs generally called from client application programs with arguments. Web Services, by definition, are not tied to specific programming languages, operating systems, network topologies, or hardware platforms.

WSDL (Web Services Definition Language)

Industry-standard XML format used to describe in detail the input arguments and return values of a particular Web Service without imposing any binding or coupling of this interface to any particular programming language, operating system, network protocol, or hardware platform.

XML (eXtensible Markup Language)

Industry-standard language used to describe data structures and attributes in a platform- and language-independent manner.

10.2 TrackWise Field WSDL Definitions for IWS

10.2.1 StringField

The Fields in the String Field Class will store values of type **String**. The following XML code shows the node for this Field as represented in the WSDL:

```
<complexType name="StringField">
  <complexContent>
    <extension base="tns4:Queryable">
      <sequence>
        <element name="value" nillable="true" type="xsd:string" />
      </sequence>
    </extension>
  </complexContent>
</complexType>
```

The first thing to notice is that the String Field Class is denoted by an object called `StringField`. Next is the “extension” node, which indicates that a `StringField` is a child of the `Queryable` class and can be queried on. The last thing is the element tag, which indicates that a `StringField` object’s value can be null and is of type string. Remember that the `Queryable` class is a child of `Field`, so `StringField` also has a name and metadata.

10.2.2 NumberField

The Fields in the Number Field Class will store values of type **Integer**. The following XML code shows the node for this Field as represented in the WSDL:

```
<complexType name="NumberField">
  <complexContent>
    <extension base="tns4:Queryable">
      <sequence>
        <element name="value" nillable="true" type="soapenc:int" />
      </sequence>
    </extension>
  </complexContent>
</complexType>
```

The `NumberField` is very similar to `StringField` except its value is of type integer.

10.2.3 DecimalField

The Fields in the Decimal Field Class will store values of type **Double**. The following XML code shows the node for this Field as represented in the WSDL:

```
<complexType name="DecimalField">
  <complexContent>
    <extension base="tns4:Queryable">
      <sequence>
        <element name="value" nillable="true" type="xsd:double" />
      </sequence>
    </extension>
  </complexContent>
</complexType>
```

10.2.4 DateField

The Fields in the Date Field Class will store values of type `Date`. The following XML code shows the node for this Field as represented in the WSDL:

```
<complexType name="DateField">
  <complexContent>
    <extension base="tns4:Queryable">
      <sequence>
        <element name="value" nillable="true" type="xsd:date" />
      </sequence>
    </extension>
  </complexContent>
</complexType>
```

10.2.5 DateTimeField

The Fields in the DateTime Field Class will store values of type `DateTime`. The following XML code shows the node for this Field as represented in the WSDL:

```
<complexType name="DateTimeField">
  <complexContent>
    <extension base="tns4:Queryable">
      <sequence>
        <element name="value" nillable="true" type="xsd:dateTime" />
      </sequence>
    </extension>
  </complexContent>
</complexType>
```

*Note that TWS supports Time Zone localization for all DateTime Fields.

10.2.6 PersonField

The Fields in the Person Field Class will store values of type `Person`. The following XML code shows the node for this Field as represented in the WSDL:

```
<complexType name="PersonField">
  <complexContent>
    <extension base="tns4:Queryable">
      <sequence>
        <element name="person" nillable="true" type="tns4:Person" />
      </sequence>
    </extension>
  </complexContent>
</complexType>
```

As shown, a `PersonField` object stores a `ComplexType` object named `Person`, which is a Data Type that is inherent to TWS and will be described in more detail later in this document.

10.2.7 EntityField

The Fields in the Entity Field Class will store values of type `Entity`. The following XML code shows the node for this Field as represented in the WSDL:

```
<complexType name="EntityField">
  <complexContent>
    <extension base="tns4:Queryable">
      <sequence>
        <element name="entity" nillable="true" type="tns4:Entity" />
      </sequence>
    </extension>
  </complexContent>
</complexType>
```

Similar to the `PersonField`, the `EntityField` object stores a `ComplexType` object named `Entity`, which is a Data Type that is also inherent to TWS.

10.2.8 SelectionField

The Fields in the Selection Field Class will store values of type `selection`. The following XML code shows the node for this Field as represented in the WSDL:

```
<complexType name="SelectionField">
  <complexContent>
    <extension base="tns4:Queryable">
      <sequence>
        <element name="selection" nillable="true" type="tns4:Selection" />
      </sequence>
    </extension>
  </complexContent>
</complexType>
```

```
</sequence>
</extension>
</complexContent>
</complexType>
```

A `SelectionField` is similar to the previous two Field Classes except it stores a `Selection` object. Like the `Person` and `Entity` object, a `Selection` object is an inherent Data Type in TWS.

10.2.9 MultiPersonField

The Fields in the `MultiPerson` Field Class will store values of type `Person`. The following XML code shows the node for this Field as represented in the WSDL:

```
<complexType name="MultiPersonField">
  <complexContent>
    <extension base="tns4:Queryable">
      <sequence>
        <element name="persons" nillable="true" type="impl:ArrayOf_tns4_Person"/>
      </sequence>
    </extension>
  </complexContent>
</complexType>
```

A `MultiPersonField` has the ability to store multiple `Person` objects in a single instance. These objects are stored in an Array inside the `MultiPersonField` object. As mentioned previously in this document, when querying on any multi Field, the IN operator is required.

10.2.10 MultiEntityField

The Fields in the `MultiEntity` Field Class will store values of type `Entity`. The following XML code shows the node for this Field as represented in the WSDL:

```
<complexType name="MultiEntityField">
  <complexContent>
    <extension base="tns4:Queryable">
      <sequence>
        <element name="entities" nillable="true" type="impl:ArrayOf_tns4_Entity"/>
      </sequence>
    </extension>
  </complexContent>
</complexType>
```

A `MultiEntityField` has the ability to store multiple `Entity` objects in a single instance. These objects are stored in an Array inside the `MultiEntityField` object. As mentioned previously in this document, when querying on any multi Field, the IN operator is required.

10.2.11 MultiSelectionField

The Fields in the MultiSelection Field Class will store values of type `selection`. The following XML code shows the node for this Field as represented in the WSDL:

```
<complexType name="MultiSelectionField">
  <complexContent>
    <extension base="tns4:Queryable">
      <sequence>
        <element name="selections" nillable="true" type="impl:ArrayOf_tns4_Selection"/>
      </sequence>
    </extension>
  </complexContent>
</complexType>
```

A `MultiSelectionField` has the ability to store multiple `Selection` objects in a single instance. These objects are stored in an Array inside the `MultiSelectionField` object. As mentioned previously in this document, when querying on any multi Field, the IN operator is required.

10.2.12 MultiTextField

The Fields in the MultiText Field Class will store values of type `string`. The following XML code shows the node for this Field as represented in the WSDL:

```
<complexType name="MultiTextField">
  <complexContent>
    <extension base="tns4:Queryable">
      <sequence>
        <element name="values" nillable="true" type="impl:ArrayOf_xsd_string"/>
      </sequence>
    </extension>
  </complexContent>
</complexType>
```

A `MultiTextField` has the ability to store multiple strings in a single instance. These strings are stored in an Array inside the `MultiTextField` object. As mentioned previously in this document, when querying on any multi Field, the IN operator is required.

10.2.13 GridField

The Fields in the Grid Field Class will store values of type `GridRow`. The following XML code shows the node for this Field as represented in the WSDL:

```
<complexType name="GridField">
  <complexContent>
    <extension base="tns4:Field">
      <sequence>
        <element name="gridRows" nillable="true" type="impl:ArrayOf_tns4_GridRow"/>
      </sequence>
    </extension>
  </complexContent>
</complexType>
```

A `GridField` has the ability to store multiple `GridRow` objects in a single instance. These objects are stored in an Array inside the `GridField` object. `Grid` is a special Field Class because it is the only class that stores Fields (`GridRow`), rather than values or Data Types.

10.2.14 GridRow

The Fields in the `GridRow` Field Class will store values of type `Field`. The following XML code shows the node for this Field as represented in the WSDL:

```
<complexType name="GridRow">
  <complexContent>
    <extension base="tns4:Field">
      <sequence>
        <element name="Fields" nillable="true" type="impl:ArrayOf_tns4_Field"/>
      </sequence>
    </extension>
  </complexContent>
</complexType>
```

A `GridRow` has the ability to store multiple `Field` objects in a single instance. Essentially each `GridRow` object corresponds to a single row in a `GridField`. The `Field` objects that are stored in a `GridRow` are configurable by the user in the TrackWise Administrator. Each `GridRow` must abide by the column layout and sequence created for the `Grid` in the Administrator.

10.3 TrackWise Fields XML Structure for OWS

10.3.1 Multi-value Fields

The XML format for multi-value fields will be a series of <value>value</value> sets indicating each of the selected values. This format applies to the following multi-value fields: MultiSelection, MultiPerson, MultiEntity, Ref Record, MultiTraining Req, File Pointer, MultiText.

```
<multivalue>
  <value>
    Bob
  </value>
  <value>
    Mike
  </value>
  <value>
    Peter
  </value>
</multivalue>
```

10.3.2 Grid

```
<![CDATA[
  <gridfield>
    <replace>true</replace>
    <row>
      <column>
        <name>String Column</name>
        <value>Alpha</value>
      </column>
      <column>
        <name>Decimal Column</name>
        <value>1.28</value>
      </column>
    </row>
    <row>
      <column>
        <name>String Column</name>
        <value>Beta</value>
      </column>
      <column>
        <name>Decimal Column</name>
        <value>2.28</value>
      </column>
    </row>
  </gridfield>
]>
```

10.3.3 URL

```
<urlfield>
  <urlvalue>
    <description>
      Google
    </description>
    <url>
      http://www.google.com
    </url>
  </urlvalue>
</urlfield>
```

10.3.4 Questions

```
<questionfield>
  <questionvalue>
    <question>
      Where was it found?
    </question>
    <response>
      In office D2
    </response>
  </questionvalue>
</questionfield>
```

10.3.5 Person

```
<personfield>
  <personvalue>
    <pid>65551</pid>
    <firstname>Jim</firstname>
```

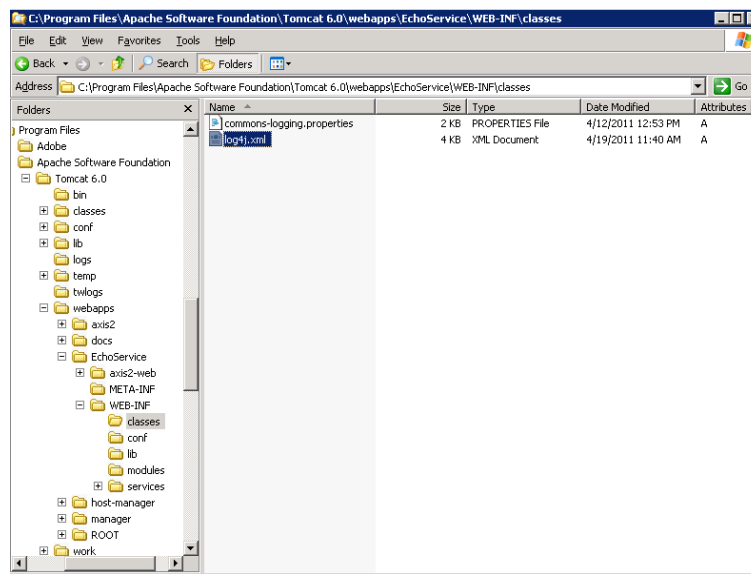
```
<lastname>Smith</lastname>
<salutation>Mr.</salutation>
<customer>no</customer>
<customerid></customerid>
<title>Unknown</title>
<entity>Sparta Systems</entity>
<department />
<manager></manager>
<active>yes</active>
<address>
  <street>2137 Hwy 35</street>
  <line2>def line2 street</line2>
  <city>Holmdel</city>
  <state>NJ</state>
  <country>USA</country>
  <zip>07733</zip>
  <zipfour></zipfour>
  <postalcode></postalcode>
  <usaddress>yes</usaddress>
</address>
<room></room>
<phone>7322030400</phone>
<ext>8888</ext>
<fax>1010101010</fax>
<mobile>def pager</mobile>
<pin>def pin</pin>
<email>JSmith@spartasystems.com</email>
<classified>no</classified>
<externalid></externalid>
</personvalue>
</personfield>
```

For more information please refer to the Web Admin Manual

10.4 Using Log4J to write to a log file

With Log4J you may specify what you'd like to be written to a log file in your Java source code.

1. Place the **log4j.xml** file into the **<Tomcat Folder>\webapps\<Web Service folder>\WEB-INF\classes**. An example of a **log4j.xml** can be found in **M:\Solutions Support\Web Services**.



2. The location that the log file will be written is specified in the log4j.xml

```
<log4j:configuration xmlns:log4j="http://jakarta.apache.org/log4j/" debug="true">
  <!-- file appender -->
  <appender name="fileAppender" class="org.apache.log4j.RollingFileAppender">
    <!-- Unix Logs -->
    <param name="File" value="C:/Program Files/Apache Software Foundation/Tomcat 6.0/logs/ws.log"/>
    <param name="MaxBackupIndex" value="5"/>
    <param name="MaxFileSize" value="5MB"/>
    <layout class="org.apache.log4j.PatternLayout">
      <!-- The default pattern: Date Priority [Category] Thread Message -->
      <param name="ConversionPattern" value="%d{ISO8601} [%-8t] [%-5p] %c{2}: %m%n"/>
    </layout>
  </appender>
```

3. You may create an appender from the package level down to the class level. In this example, we are using a class level appender.

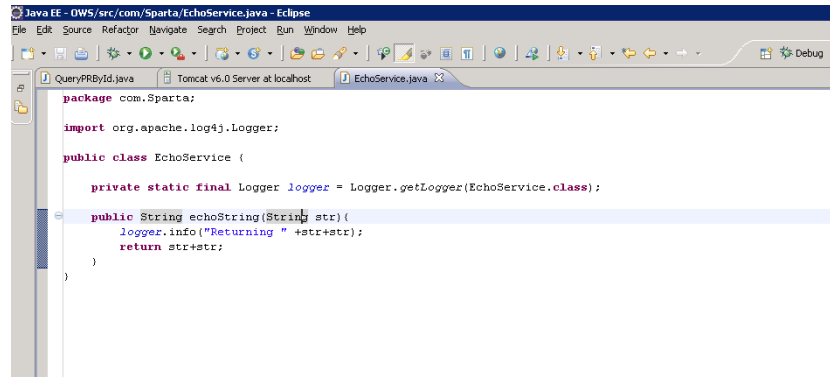
```
<category name="com.Sparta.EchoService">
  <priority value="DEBUG"/>
  <appender-ref ref="fileAppender"/>
  <appender-ref ref="consoleAppender"/>
</category>
```

4. To enable logging in your java code, add the lines:

```

import org.apache.log4j.Logger; //This is used to import the Logger class
Private static final Logger logger = Logger.getLogger(yourclassname.class);
//This is used to create the Logger instance
logger.info(); // what you place in the parenthesis is what will be written to the
log file.

```



5. In this example we are using log4J .properties file:

```
log4j.logger.<appname>logger=DEBUG, C, fileappender
```

```
log4j.additivity.<appname>logger=false
```

```
log4j.appender.C=org.apache.log4j.ConsoleAppender
```

```
log4j.appender.C.layout=org.apache.log4j.PatternLayout
```

```
#basic pattern
```

```
log4j.appender.C.layout.ConversionPattern=[%c] [%d{dd MMM yyyy - hh:mm:ss}]
```

```
%5p - %m %n
```

```
#advanced pattern (slow)
```

```
#log4j.appender.C.layout.ConversionPattern=[%c] [%d{dd MMM yyyy - hh:mm:ss}]
```

```
%5p - %m - in %M() at line %L of class %C %n
```

```
log4j.appender.fileappender=org.apache.log4j.RollingFileAppender
```

```
log4j.appender.fileappender.File=${appRootPath}WEB-INF/logs/<appname>.log
```

```
log4j.appender.fileappender.MaxFileSize=500KB
```

```
## Keep one backup file
```

```
log4j.appender.fileappender.MaxBackupIndex=3
```

```
log4j.appender.fileappender.layout=org.apache.log4j.PatternLayout
```

```
log4j.appender.fileappender.layout.ConversionPattern=%p %t %c - %m%n
```

```
#log4j.appender.C.layout.ConversionPattern=[%c] [%d{dd MMM yyyy - hh:mm:ss}]
```

```
%5p - %m %n
```

10.5 Reading parameters from a file

It may be necessary to store parameter values, such as database connection information, in a file so that it may easily be updated. The benefit to storing parameters in an external file is that the source code does not have to be updated and recompiled if the values need updating. The following code, highlighted in blue, can be used to read parameters stored in the services.xml file.

1. Import the Message Context object.

```
import org.apache.axis2.context.MessageContext;
```

2. Create a `getParameter()` method within your class.

```
private String getParameter(String paramName)
{
    String retStr =
    MessageContext.getCurrentMessageContext().getAxisService().getParameter(pa
    ramName).getParameterElement().getText();
    if (retStr != null)
        return retStr;
    else
        return null;
}
```

```
package com.Sparta;

import org.apache.log4j.Logger;
import org.apache.axis2.context.MessageContext;

public class EchoService {

    private static final Logger logger = Logger.getLogger(EchoService.class);

    public String echoString(String str) {
        logger.info("Returning " + str + str);
        return str + str;
    }

    private String getParameter(String paramName)
    {
        String retStr = MessageContext.getCurrentMessageContext().getAxisService().getP
        arameter(paramName).getP
        if (retStr != null)
            return retStr;
        else
            return null;
    }
}
```

3. Use the `getParameter()` method to assign the parameter value to a variable.

```
String dbUser = getParameter("dbUser");
String dbPass = getParameter("dbPass");
```

Here is an example of parameters stored in the services.xml file. Again, the **services.xml** file can be found in the following location: **C:\<apache Tomcat folder>\webapps\<web content folder name>\WEB-INF\services\<web service name>\META-INF\service.xml**

```

1 <service name="SAPLookup" >
2   <Description>
3     Please Type your service description here
4   </Description>
5   <messageReceivers>
6     <messageReceiver mep="http://www.w3.org/2004/08/wsdl/in-only" class="org.apache.axis2.rpc.receivers.RPCInOnlyMess
7     <messageReceiver mep="http://www.w3.org/2004/08/wsdl/in-out" class="org.apache.axis2.rpc.receivers.RPCMessageRe
8   </messageReceivers>
9   <parameter name="ServiceClass" locked="false">com.mln.xi.pm.get_trackwise_data_sap.SAPLookup</parameter>
10  <parameter name="dbUser">sysadm</parameter>
11  <parameter name="dbPass">sysadm</parameter>
12 </service>
13

```

10.6 Code Example : Convert date format

```

// string containing date in one format
String strDate = "12/12/07";

// create SimpleDateFormat object with source string date
// format
SimpleDateFormat sdfSource = new SimpleDateFormat("dd/MM/yy");

// parse the string into Date object
Date date = sdfSource.parse(strDate);

// create SimpleDateFormat object with desired date format
SimpleDateFormat sdfDestination = new SimpleDateFormat(
    "MM-dd-yyyy hh:mm:ss");

// parse the date into another format
strDate = sdfDestination.format(date);

```

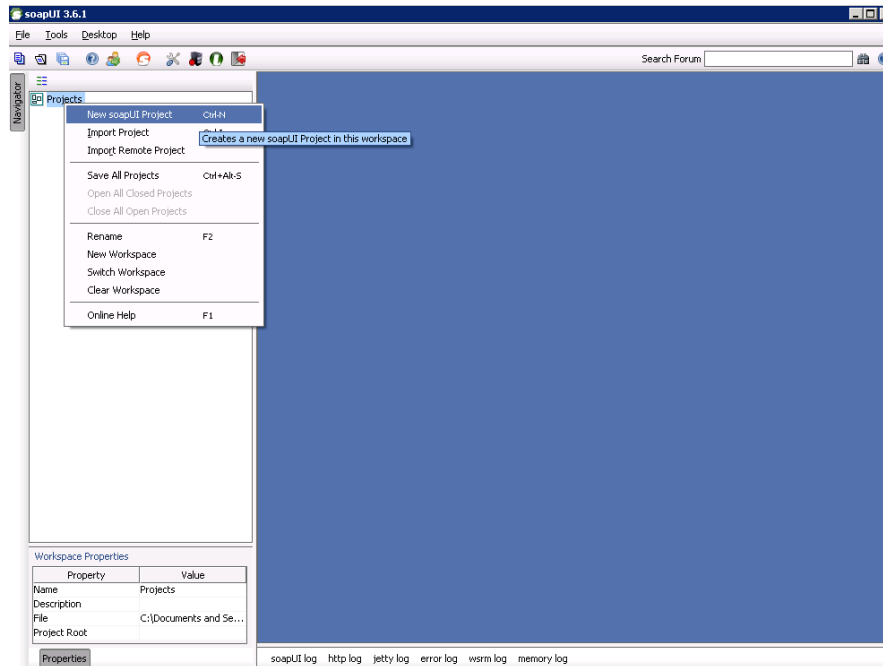
10.7 SoapUI

SoapUI is a tool that you may use to test your web service outside of TrackWise.

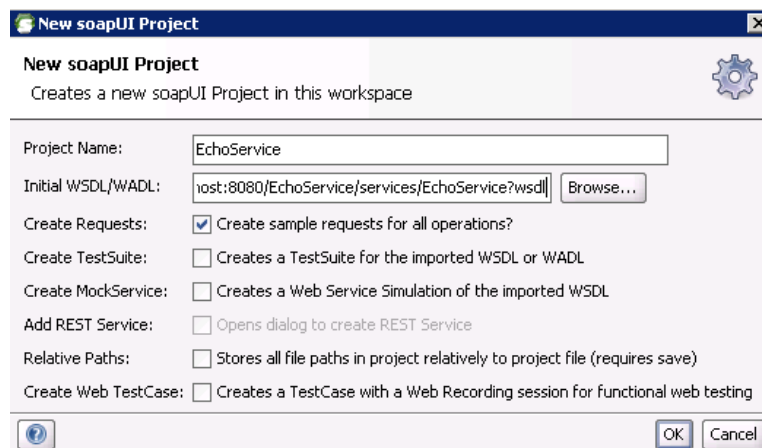
Leave all of the default settings when installing SoapUI.

10.7.1 Testing Web Service in SoapUI

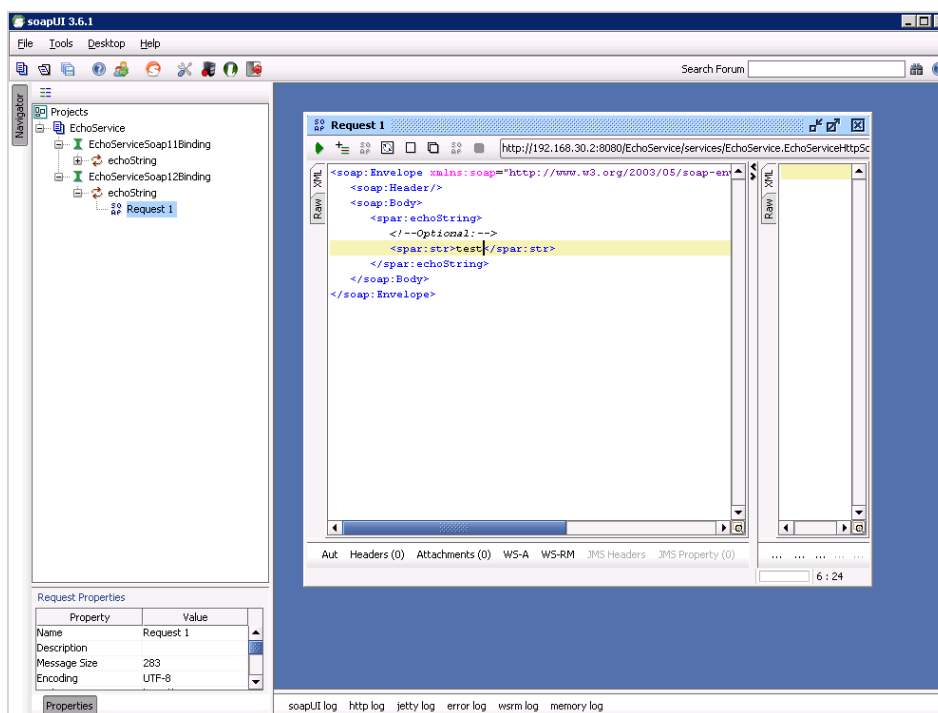
1. Right click **Projects** and select **New soapUI Project**.



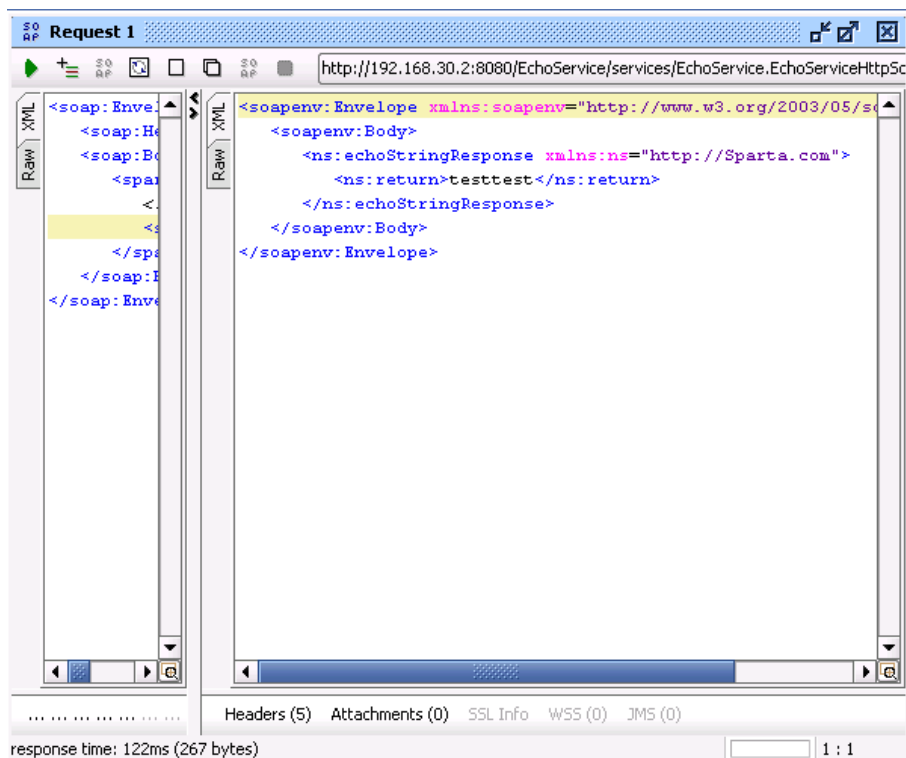
2. Name the project and specify the wsdl location in the **Initial WSDL/WADL** field.



3. Double click **Request 1** to open a SOAP request window. Enter values into the parameter fields. In this example we have one parameter field, <spar: str>.



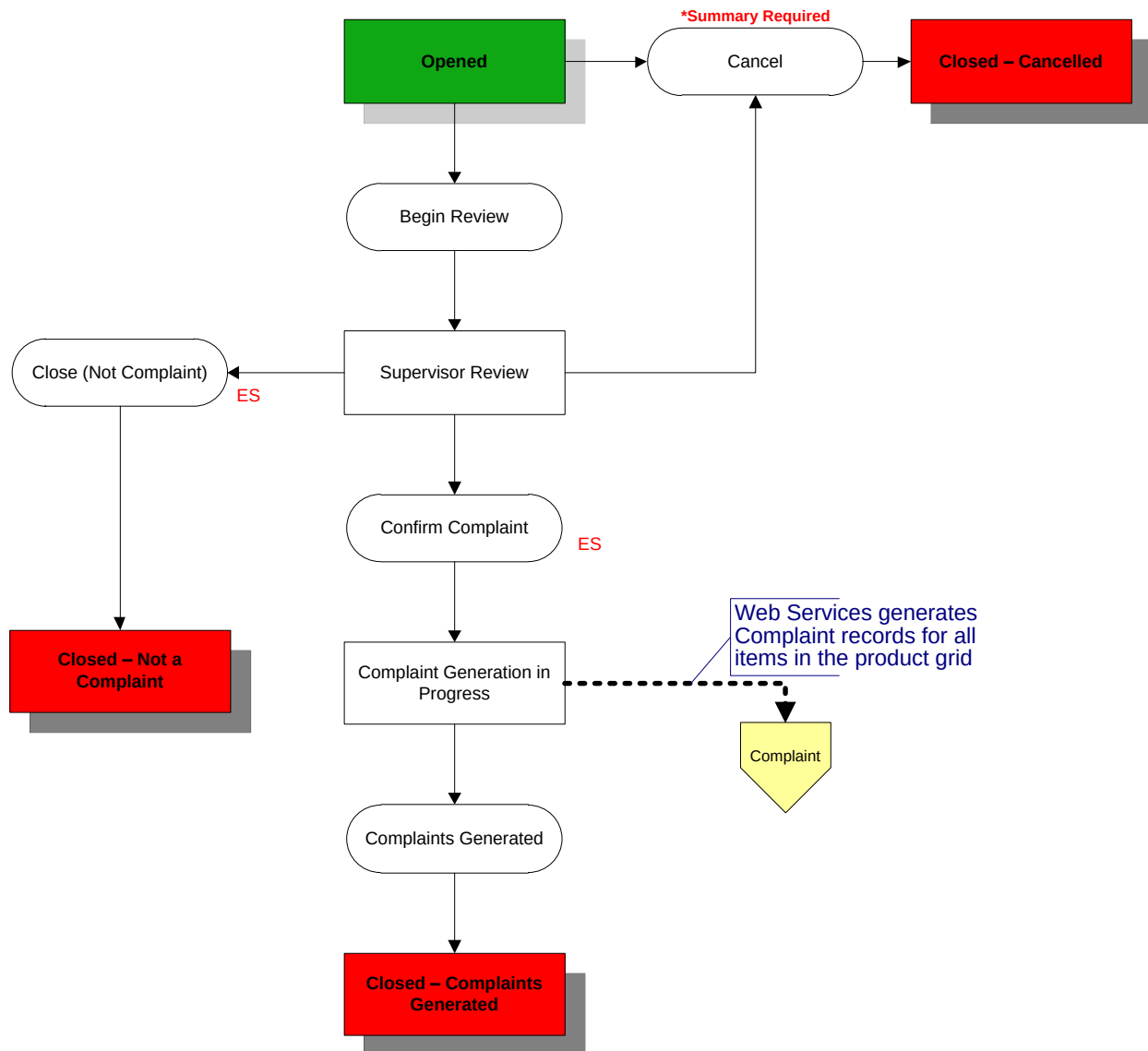
- Click **Submit** and confirm that the desired results are successfully returned.



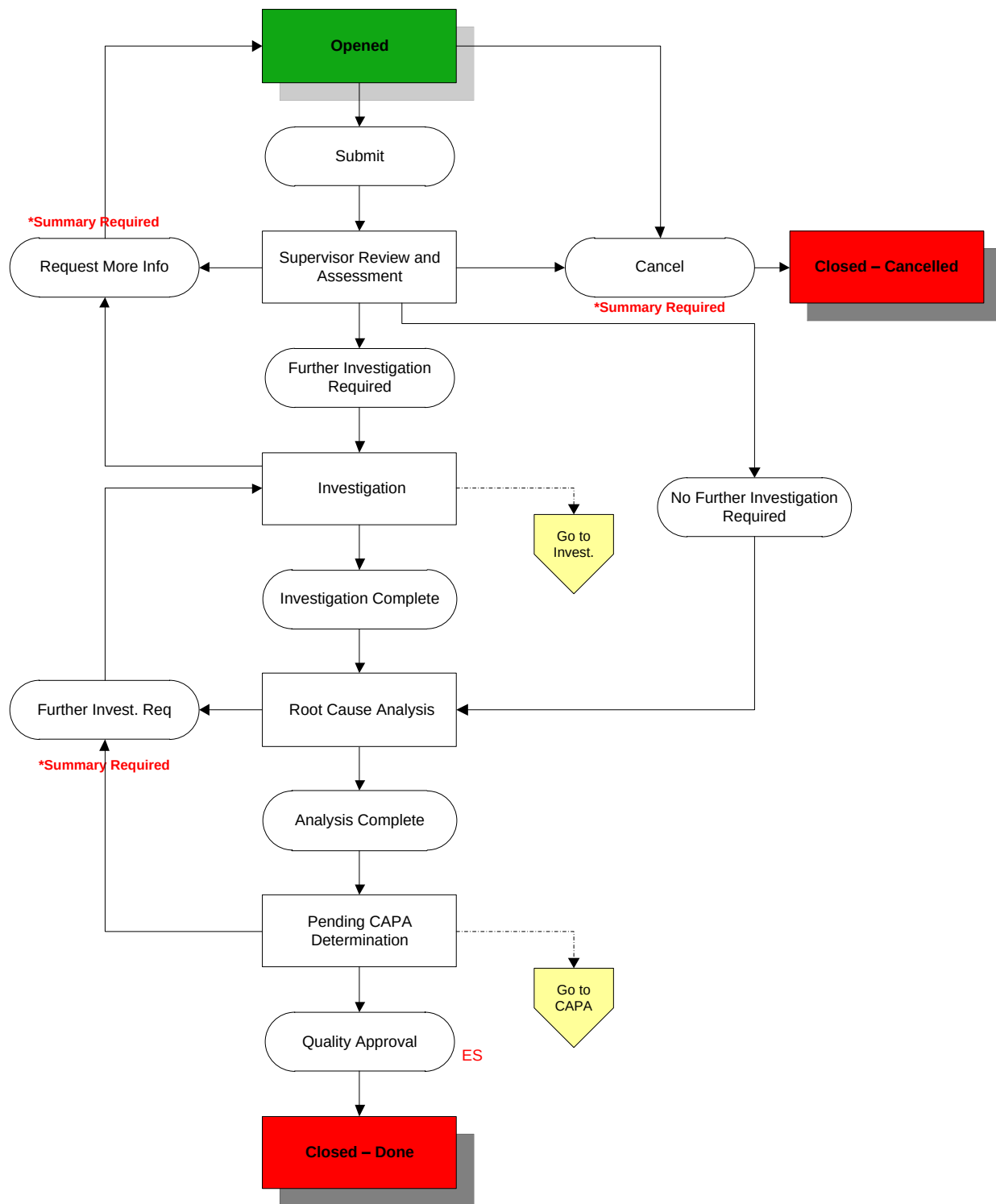
10.8 Workflows

The workflows configured for the purposes of this course are shown below:

Product Issue



Complaint



10.9 Course ProductService.wsdl File

```
<?xml version="1.0" encoding="UTF-8"?>
<wsdl:definitions name="ProductService" targetNamespace="http://ows.sparta.com"
  xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/"
  xmlns:tns="http://ows.sparta.com"
  xmlns:xs="http://www.w3.org/2001/XMLSchema"
  xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/">
  <wsdl:documentation>A TrackWise Outbound Web Service (OWS) to fetch Product Issue
    Related Data for TWS Training Course.</wsdl:documentation>
  <wsdl:types>
    <xs:schema attributeFormDefault="unqualified" elementFormDefault="unqualified"
      targetNamespace="http://ows.sparta.com">
      <xs:element name="FetchProductNumberRequest">
        <xs:complexType>
          <xs:sequence>
            <xs:element name="productNumber" type="xs:string"/>
          </xs:sequence>
        </xs:complexType>
      </xs:element>
      <xs:element name="FetchProductNumberResponse">
        <xs:complexType>
          <xs:sequence>
            <xs:element minOccurs="0" name="productNumber"
              type="xs:string"/>
            <xs:element minOccurs="0" name="detailedMessage"
              type="tns:DetailedMessage"/>
          </xs:sequence>
        </xs:complexType>
      </xs:element>
      <xs:element name="FetchLotNumberRequest">
        <xs:complexType>
          <xs:sequence>
            <xs:element name="productNumber" type="xs:string" />
            <xs:element name="lotNumber" type="xs:string"/>
          </xs:sequence>
        </xs:complexType>
      </xs:element>
      <xs:element name="FetchLotNumberResponse">
        <xs:complexType>
          <xs:sequence>
            <xs:element minOccurs="0" name="lotNumber" type="xs:string"/>
            <xs:element minOccurs="0" name="detailedMessage"
              type="tns:DetailedMessage"/>
          </xs:sequence>
        </xs:complexType>
      </xs:element>
      <xs:element name="FetchProductDetailsRequest">
        <xs:complexType>
          <xs:sequence>
            <xs:element name="productNumber" type="xs:string"/>
            <xs:element name="lotNumber" type="xs:string"/>
          </xs:sequence>
        </xs:complexType>
      </xs:element>
      <xs:element name="FetchProductDetailsResponse">
        <xs:complexType>
          <xs:sequence>
            <xs:element minOccurs="0" name="productName" type="xs:string"/>
            <xs:element minOccurs="0" name="strength" type="xs:string"/>
            <xs:element minOccurs="0" name="productDescription"
              type="xs:string"/>
            <xs:element minOccurs="0" name="expirationDate"
              type="xs:string"/>
            <xs:element minOccurs="0" name="materialsAffected"
              type="xs:string"/>
            <xs:element minOccurs="0" name="detailedMessage"
              type="tns:DetailedMessage"/>
          </xs:sequence>
        </xs:complexType>
      </xs:element>
    </xs:schema>
  </wsdl:types>

```

```

        </xs:sequence>
      </xs:complexType>
    </xs:element>
    <xs:element name="CreateComplaintChildrenRequest">
      <xs:complexType>
        <xs:sequence>
          <xs:element name="recordId" type="xs:int"/>
          <xs:element name="eventDate" type="xs:string"/>
          <xs:element name="awarenessDate" type="xs:string"/>
          <xs:element name="description" type="xs:string"/>
          <xs:element name="customersWords" type="xs:string"/>
          <xs:element name="initialReporter" type="xs:string"/>
          <xs:element name="initialReporterCompany" type="xs:string"/>
          <xs:element name="productInformationGrid" type="xs:string"/>
        </xs:sequence>
      </xs:complexType>
    </xs:element>
    <xs:complexType name="DetailedMessage">
      <xs:sequence>
        <xs:element name="message" nillable="false" type="xs:string"/>
        <xs:element name="isSuccess" nillable="true" type="xs:boolean"/>
      </xs:sequence>
    </xs:complexType>
  </xs:schema>
</wsdl:types>
<wsdl:message name="FetchProductDetailsRequest">
  <wsdl:part name="parameters" element="tns:FetchProductDetailsRequest">
  </wsdl:part>
</wsdl:message>
<wsdl:message name="FetchLotNumberResponse">
  <wsdl:part name="parameters" element="tns:FetchLotNumberResponse">
  </wsdl:part>
</wsdl:message>
<wsdl:message name="CreateComplaintChildrenRequest">
  <wsdl:part name="parameters" element="tns:CreateComplaintChildrenRequest">
  </wsdl:part>
</wsdl:message>
<wsdl:message name="FetchProductNumberRequest">
  <wsdl:part name="parameters" element="tns:FetchProductNumberRequest">
  </wsdl:part>
</wsdl:message>
<wsdl:message name="FetchLotNumberRequest">
  <wsdl:part name="parameters" element="tns:FetchLotNumberRequest">
  </wsdl:part>
</wsdl:message>
<wsdl:message name="FetchProductNumberResponse">
  <wsdl:part name="parameters" element="tns:FetchProductNumberResponse">
  </wsdl:part>
</wsdl:message>
<wsdl:message name="FetchProductDetailsResponse">
  <wsdl:part name="parameters" element="tns:FetchProductDetailsResponse">
  </wsdl:part>
</wsdl:message>
<wsdl:portType name="ProductServicePort">
  <wsdl:operation name="FetchProductNumber">
    <wsdl:input message="tns:FetchProductNumberRequest">
    </wsdl:input>
    <wsdl:output message="tns:FetchProductNumberResponse">
    </wsdl:output>
  </wsdl:operation>
  <wsdl:operation name="FetchLotNumber">
    <wsdl:input message="tns:FetchLotNumberRequest">
    </wsdl:input>
    <wsdl:output message="tns:FetchLotNumberResponse">
    </wsdl:output>
  </wsdl:operation>
  <wsdl:operation name="FetchProductDetails">
    <wsdl:input message="tns:FetchProductDetailsRequest">
    </wsdl:input>
  </wsdl:operation>

```

```

    <wsdl:output message="tns:FetchProductDetailsResponse">
    </wsdl:output>
  </wsdl:operation>
  <wsdl:operation name="CreateComplaintChildren">
    <wsdl:input message="tns:CreateComplaintChildrenRequest">
    </wsdl:input>
  </wsdl:operation>
</wsdl:portType>
<wsdl:binding name="ProductServiceBinding" type="tns:ProductServicePort">
  <soap:binding style="document"
    transport="http://schemas.xmlsoap.org/soap/http"/>
  <wsdl:operation name="FetchProductNumber">
    <soap:operation soapAction="http://ows.sparta.com/FetchProductNumber"/>
    <wsdl:input>
      <soap:body use="literal"/>
    </wsdl:input>
    <wsdl:output>
      <soap:body use="literal"/>
    </wsdl:output>
  </wsdl:operation>
  <wsdl:operation name="FetchLotNumber">
    <soap:operation soapAction="http://ows.sparta.com/FetchLotNumber"/>
    <wsdl:input>
      <soap:body use="literal"/>
    </wsdl:input>
    <wsdl:output>
      <soap:body use="literal"/>
    </wsdl:output>
  </wsdl:operation>
  <wsdl:operation name="FetchProductDetails">
    <soap:operation soapAction="http://ows.sparta.com/FetchProductDetails"/>
    <wsdl:input>
      <soap:body use="literal"/>
    </wsdl:input>
    <wsdl:output>
      <soap:body use="literal"/>
    </wsdl:output>
  </wsdl:operation>
  <wsdl:operation name="CreateComplaintChildren">
    <soap:operation soapAction="http://ows.sparta.com/CreateComplaintChildren"/>
    <wsdl:input>
      <soap:body use="literal"/>
    </wsdl:input>
  </wsdl:operation>
</wsdl:binding>
<wsdl:service name="ProductService">
  <wsdl:port name="ProductServicePort" binding="tns:ProductServiceBinding">
    <soap:address location="http://ows.sparta.com/">
    </wsdl:port>
  </wsdl:service>
</wsdl:definitions>

```

10.10 Course tws-course.properties File :

```
# This is the complete URL to the TrackWise Web Service
# (Also known as the Endpoint Reference)
#tws.url=http://localhost/axis2/services/TrackWise
tws.url=http://10.0.1.162:8080/axis2/services/TrackWise

# This specifies how many threads can run on the server side
tws.maxconnections=5

# This specifies how long the client will wait to timeout in seconds
(minimum of 5)
tws.timeout=30

# Name of the TrackWise user
#tws.user=tws
tws.user=tws
# Password of the TrackWise user
#tws.pass=tws
tws.pass=tws
# TimeZone of the TrackWise user location
tws.timezone=America/New_York

# QueryPR related attributes
querypr.prid=1001
querypr.shortdesc=Hello World
querypr.divison=Holmdel
querypr.project=Complaint
querypr.assigned.firstname=Brian
querypr.assigned.lastname=Brzezinski

# UpdatePR related attributes
updatepr.prid=1001
updatepr.assignedtoid=1
updatepr.selectionfield.value=Updated Selection Field Value
updatepr.fileattachment.name=update.jpg
updatepr.fileattachment.fileuploadlocation=C:/sample2.jpg
updatepr.stringgridfield.value=Sample String Field Value
updatepr.decimalgridfield.value=1
updatepr.selectiongridfield.value=A
updatepr.updaterowindex=1
updatepr.deleterowindex=2

# CreatePR related attributes
createpr.division=Holmdel
createpr.project=Complaint
createpr.parentid=1001
createpr.shortdesc=New complaint from external
createpr.assigned.firstname=Brian
createpr.assigned.lastname=Brzezinski
```